

Contents

第五册简介：大模型训练全流程	11
本书定位	11
为什么需要这本书	12
适合读者	12
学完后应具备的能力	12
阅读建议	12
与其他书的关系	12
第五册：大模型训练全流程	12
本书定位	12
章节文件	12
核心问题	13
第一章：训练总览	13
本章目标	13
核心议题	13
典型训练流程	13
面试重点	13
为什么训练总览很重要	13
一句话看完整训练链路	14
训练总览的资料边界	14
训练项目的关键公式口径	14
训练阶段总览	15
从目标定义开始	15
Base Model、Instruct Model、Chat Model、Reasoning Model	16
数据、模型、系统、评估、成本的联动	16
数据影响模型能力	16
模型规模影响训练策略	16
系统影响可执行性	17
评估影响方向判断	17
成本影响方案边界	17
一个训练项目的关键产物	17
预训练在做什么	17
后训练在做什么	18
训练监控要看什么	18
Checkpoint 不是越后越好	19
训练失败常见来源	19
成本意识	19
面试回答模板：如何训练一个大模型	22
一个常见错误回答	23
本章小结	23
第二章：数据工程	23
本章目标	23
核心议题	23
面试重点	23
为什么数据工程是训练起点	24
数据工程的资料边界	24
1. 数据来源有哪些	24
2. 数据处理流水线	24
2.1 采集	25
2.2 格式统一	25
3. 清洗和过滤	26
3.1 质量打分	26

3.2 过滤不能过头	26
4. 去重与近似去重	27
4.1 Exact dedup	27
4.2 Near dedup	27
4.3 为什么近似去重重要	27
5. Benchmark contamination	28
5.1 污染形式	28
5.2 怎么检测	28
5.3 为什么面试官爱问	28
6. 数据配比	28
6.1 为什么要配比	28
6.2 采样方式	29
6.3 面试表达	29
7. 质量评估与数据版本管理	29
7.1 统计什么	29
7.2 为什么要版本管理	30
8. 一个最小数据处理示例	30
9. 常见坑	35
9.1 只看数量，不看质量	35
9.2 过滤规则过拟合某一批数据	35
9.3 没有污染检测	35
9.4 没有保留元数据	35
9.5 数据配比固定不变	35
10. 面试官会怎么问	35
问法 1: 为什么数据工程这么重要?	35
问法 2: 你怎么做数据去重?	35
问法 3: 怎么判断训练集污染了 benchmark?	36
问法 4: 数据配比怎么定?	36
11. 小结	36

第三章: Tokenizer 与数据格式

36

本章目标	36
核心议题	36
面试重点	36
为什么 tokenizer 是训练协议	36
Tokenizer 与数据格式的资料边界	37
关键公式口径	37
1. Tokenizer 影响哪些训练问题	37
2. 常见 tokenizer 方法	37
2.1 BPE	38
2.2 SentencePiece	38
2.3 Byte-level tokenizer	38
3. 词表大小如何权衡	39
3.1 大词表的好处	39
3.2 大词表的代价	39
3.3 小词表的代价	39
4. 压缩率和 tokenizer 效率	39
5. Special token	40
5.1 EOS 很重要	40
5.2 PAD 不能参与 loss	40
6. Chat template	41
7. Label mask	41
7.1 为什么不对 user 部分算 loss	42
7.2 常见错误	42
8. 预训练数据格式	42
8.1 文档边界	42

8.2 Packing	42
9. SFT 数据格式	43
10. 偏好数据格式	43
11. Tool calling 数据格式	44
12. 多模态 token	44
13. Tokenizer 扩展	45
14. 一个最小 label mask 示例	45
15. 常见坑	49
15.1 训练和推理 tokenizer 不一致	49
15.2 忘记 EOS	49
15.3 label mask 错误	49
15.4 截断策略错误	49
15.5 多语言切分不均衡	49
15.6 特殊 token 被当普通字符串	50
16. 面试官会怎么问	50
问法 1: 为什么 tokenizer 会影响训练成本?	50
问法 2: 词表越大越好吗?	50
问法 3: SFT 中为什么要做 label mask?	50
问法 4: 新增 special token 要注意什么?	50
问法 5: 多模态里的 image token 是什么?	50
17. 本章小结	50
第四章: 模型架构与初始化	50
本章目标	50
核心议题	51
面试重点	51
为什么训练前要先定架构	51
模型架构与初始化的资料边界	51
架构设计的关键变量	51
1. 为什么 decoder-only 成为 LLM 主流	52
1.1 训练目标统一	52
1.2 工程实现简单	52
1.3 什么时候不是最佳选择	52
2. 模型规模怎么定	52
2.1 层数和宽度	53
2.2 head 数和 head_dim	53
2.3 FFN size	53
3. 参数量、token 数和算力预算要匹配	53
4. 现代 LLM 的常见架构组件	54
4.1 RoPE	54
4.2 RMSNorm 和 Pre-Norm	54
4.3 SwiGLU	55
4.4 MQA 和 GQA	55
5. 上下文长度如何设计	55
6. Dense model 和 MoE model	56
6.1 MoE 的优势	56
6.2 MoE 的训练难点	56
6.3 面试表达	56
7. 初始化为什么重要	56
7.1 常见初始化直觉	56
7.2 新增 token 的初始化	57
7.3 从已有 checkpoint 继续训练	57
8. 训练稳定性相关架构选择	57
8.1 Pre-Norm 更稳	57
8.2 残差尺度要受控	57
8.3 Attention logits 尺度	57

8.4 激活函数和数值精度	58
9. 架构选择和推理成本	58
10. 一个训练前架构决策 checklist	58
11. 常见坑	61
11.1 参数越大越好	61
11.2 只改上下文长度配置	61
11.3 忽略推理成本	61
11.4 MoE 只看总参数量	61
11.5 初始化和 checkpoint 不匹配	61
12. 面试官会怎么问	61
问法 1: 如果让你设计一个 7B 模型, 你会考虑什么?	61
问法 2: 为什么现代 LLM 多用 decoder-only?	61
问法 3: GQA 解决什么问题?	62
问法 4: MoE 和 dense model 的核心 trade-off 是什么?	62
问法 5: 初始化为什么会影响训练稳定性?	62
13. 本章小结	62
第五章: 预训练目标与训练循环	62
本章目标	62
核心议题	62
面试重点	62
为什么预训练目标看起来简单但很关键	63
预训练目标与训练循环的资料边界	63
1. 自回归语言模型的概率分解	63
2. 输入和标签如何错位	63
3. Cross entropy loss 与最大似然	64
4. Attention mask 和 causal mask	65
Padding mask	65
5. Packing 与 sequence length	65
Packing 的好处	65
Packing 的风险	66
6. 一个最小训练 batch	66
7. 标准训练循环	66
8. Gradient accumulation	67
9. Mixed precision 与 loss scaling	68
10. Validation loop	68
11. 日志应该记录什么	69
12. Checkpoint 保存与恢复	69
13. Checkpoint 选择不是只看最后一步	70
14. 常见训练 bug	73
14.1 输入标签没有错位	73
14.2 causal mask 错误	73
14.3 PAD 参与 loss	73
14.4 accumulation loss 没有缩放	73
14.5 train/eval 模式没切换	73
14.6 数据重复或污染	74
15. 一个面试级回答模板	74
16. 本章小结	74
第六章: 优化器与学习率策略	74
本章定位	74
优化器与学习率策略的资料边界	75
本章目标	75
1. 优化器在训练中解决什么问题	75
2. 优化器谱系总览	75
3. SGD 与 Momentum	76

4. Adam	76
5. AdamW: 大模型训练默认强基线	77
6. Adafactor: 省显存的 Adam 替代	77
7. Lion: 更少状态的符号更新	78
8. Sophia: 利用二阶信息的高效近似	78
9. Muon: 矩阵参数更新的几何约束	79
9.1 Muon 和 AdamW 的关系	79
9.2 Muon 的优势	80
9.3 Muon 的风险和边界	80
10. 参数分组策略	80
11. Learning Rate Warmup	81
12. Cosine Decay	81
13. Weight Decay	82
14. Gradient Clipping	82
15. Batch Size Scaling	82
16. 常见优化器配置示例	85
16.1 预训练 AdamW 配置	85
16.2 SFT 配置	86
16.3 LoRA 微调配置	86
16.4 Muon 探索配置	86
17. 训练不稳定时如何排查	86
17.1 学习率是否过大	86
17.2 warmup 是否太短	86
17.3 beta2 是否不合适	87
17.4 weight decay 是否过强	87
17.5 参数分组是否错误	87
17.6 新优化器是否没有重新调参	87
18. 面试高频问法	87
问法 1: Adam 和 AdamW 有什么区别?	87
问法 2: 为什么大模型训练常用 AdamW?	87
问法 3: Muon 和 AdamW 有什么不同?	87
问法 4: Adafactor 解决什么问题?	87
问法 5: 训练 loss spike 时如何从优化器角度排查?	88
19. 本章小结	88
第七章: 分布式训练	88
本章目标	88
核心议题	88
面试重点	88
为什么大模型必须分布式训练	88
分布式训练的资料边界	89
训练状态显存口径	89
1. 分布式训练要解决哪些瓶颈	89
2. Data Parallelism	89
2.1 优点	90
2.2 缺点	90
3. DDP	90
3.1 all-reduce 是什么	90
3.2 DDP 的瓶颈	91
4. ZeRO	91
4.1 ZeRO-1	91
4.2 ZeRO-2	91
4.3 ZeRO-3	91
5. FSDP	92
5.1 FSDP 的优势	92
5.2 FSDP 的代价	92

6. Tensor Parallelism	92
6.1 为什么需要 TP	92
6.2 TP 的通信	93
6.3 面试表达	93
7. Pipeline Parallelism	93
7.1 为什么需要 PP	93
7.2 Pipeline bubble	93
7.3 面试表达	93
8. Sequence Parallelism 与 Context Parallelism	93
8.1 Sequence Parallelism	93
8.2 Context Parallelism	94
9. 3D Parallelism	94
10. Expert Parallelism	94
10.1 MoE 的分布式难点	95
11. 通信开销与计算重叠	95
12. Global batch size	95
13. Activation checkpointing	96
14. 分布式训练常见故障	96
14.1 吞吐不随 GPU 数增加	96
14.2 某些 GPU 显存明显更高	96
14.3 训练 hang 住	96
14.4 loss 不一致	97
15. 如何选择并行策略	97
16. 面试官会怎么问	99
问法 1: DDP、ZeRO、FSDP 有什么区别?	99
问法 2: Tensor Parallel 和 Pipeline Parallel 切分什么?	100
问法 3: ZeRO-3 为什么省显存但通信更大?	100
问法 4: 为什么 MoE 分布式训练难?	100
问法 5: 训练吞吐低怎么排查?	100
17. 本章小结	100

第八章：显存优化与数值精度

100

本章目标	100
核心议题	100
面试重点	101
为什么显存是训练中的硬约束	101
显存优化与数值精度的资料边界	101
1. 训练显存由哪些部分构成	101
2. 一个粗略显存估算	102
3. 激活显存为什么大	102
激活显存的特点	103
4. FP32、FP16、BF16、FP8	103
4.1 FP32	103
4.2 FP16	103
4.3 BF16	103
4.4 FP8	103
5. Mixed precision training	104
5.1 Loss scaling	104
5.2 常见数值风险	104
6. Activation checkpointing	104
7. Gradient accumulation 与显存	105
8. Optimizer state sharding	105
9. 参数高效微调的显存优势	106
10. FlashAttention 与 attention 显存	106
11. OOM 排查顺序	106
12. NaN / Inf 与精度问题	107

13. 显存优化策略对比	108
14. 面试官会怎么问	111
问法 1: 为什么 AdamW 训练显存这么大?	111
问法 2: FP16 和 BF16 有什么区别?	111
问法 3: activation checkpointing 解决什么问题?	111
问法 4: OOM 你怎么排查?	111
问法 5: NaN 怎么排查?	111
15. 本章小结	111
第九章: 训练稳定性与 Debug	112
本章目标	112
核心议题	112
Debug 顺序	112
面试重点	112
为什么训练 debug 很难	112
训练稳定性 Debug 的资料边界	113
训练健康指标口径	113
1. 先建立健康训练的基线	113
2. Debug 总顺序	114
3. Loss spike	114
3.1 可能原因	114
3.2 排查方法	114
3.3 处理方法	115
4. NaN 和 Inf	115
4.1 常见原因	115
4.2 排查顺序	115
4.3 处理方法	115
5. 梯度爆炸和梯度消失	116
5.1 梯度爆炸	116
5.2 梯度消失或过小	116
6. 数据异常导致训练异常	117
6.1 怎么检查 batch	117
6.2 数据 bug 的典型现象	117
7. Loss mask 和 label 错误	117
8. 学习率和 optimizer 问题	118
8.1 学习率过大	118
8.2 学习率过小	118
8.3 optimizer state 恢复错误	118
9. 混合精度溢出	119
10. 分布式同步错误	119
10.1 常见原因	119
10.2 排查方法	119
11. Checkpoint 恢复问题	120
12. 最小可运行训练健康审计 demo	120
13. 一个实用 debug checklist	123
14. 面试官会怎么问	123
问法 1: 训练 loss 突然 spike, 你怎么排查?	123
问法 2: NaN 怎么定位?	123
问法 3: 为什么 train loss 下降但模型效果很差?	124
问法 4: 分布式训练 hang 住怎么办?	124
问法 5: 恢复 checkpoint 后 loss 突然异常怎么办?	124
15. 本章小结	124
第十章: Checkpoint、评估与实验管理	124
本章目标	124
核心议题	124

面试重点	124
为什么 checkpoint 和实验管理很重要	124
Checkpoint 与实验管理的资料边界	125
实验对象与总门禁	125
1. Checkpoint 应该保存什么	125
2. 保存频率怎么定	126
3. Checkpoint 存储成本	127
4. 训练恢复和容错	127
5. 为什么不能只看最后一个 checkpoint	128
6. Validation loss 和 perplexity	128
7. Per-domain loss	129
8. 下游 benchmark	130
9. Regression eval	130
10. 实验追踪应该记录什么	131
11. 实验命名和版本管理	131
12. 可复现性	132
13. 实验对比的基本原则	132
14. Checkpoint 选择流程	133
15. 最小可运行实验管理审计 demo	133
16. 面试官会怎么问	136
问法 1: checkpoint 里应该保存什么?	136
问法 2: 如何选择最好的 checkpoint?	136
问法 3: 为什么 train loss 降低不代表模型更好?	136
问法 4: 如何保证实验可复现?	137
问法 5: 两个实验怎么公平比较?	137
17. 本章小结	137

第十一章：后训练与对齐强化训练

137

本章定位	137
本章目标	138
为什么后训练是模型变成助手的关键阶段	138
后训练与对齐训练的资料边界	138
后训练目标的统一口径	138
1. 后训练总体流程	139
1.1 SFT: 建立基本指令遵循能力	139
1.2 Reward Modeling: 学习偏好信号	139
1.3 RLHF: 用 RL 优化策略模型	140
1.4 Offline Preference Optimization: 离线偏好优化	140
1.5 Reasoning RL: 面向可验证任务的强化训练	140
1.6 Safety Alignment: 安全与行为边界	140
2. 方法谱系总览	140
3. PPO-RLHF	141
4. DPO: 直接偏好优化	142
5. GRPO: Group Relative Policy Optimization	143
6. GSPO: 序列级策略优化	144
7. DAPO: 面向长链推理的高级策略优化	145
8. DCPO: 偏好、对比与约束优化	145
9. PPO、DPO、GRPO、GSPO、DAPO、DCPO 怎么选	146
9.1 如果目标是通用聊天偏好	146
9.2 如果目标是数学和代码 reasoning	146
9.3 如果目标是安全和格式约束	146
9.4 如果资源有限	146
9.5 如果资源充足且任务可验证	147
10. 数据要求对比	147
11. 常见训练风险	147
11.1 Reward Hacking	147

11.2 Length Bias	147
11.3 KL Collapse	147
11.4 Over-Optimization	147
11.5 Catastrophic Forgetting	148
11.6 Safety Regression	148
12. 现代后训练 pipeline 示例	148
13. 最小可运行后训练审计 demo	149
14. 面试高频问法	152
问法 1: SFT、DPO 和 PPO 的区别是什么?	152
问法 2: 为什么 DPO 不能代表当前所有主流后训练?	152
问法 3: GRPO 相比 PPO 的核心差异是什么?	152
问法 4: 为什么 reasoning RL 需要在线采样?	152
问法 5: 后训练如何避免能力退化?	152
15. 本章小结	152

第十二章：多模态训练

153

本章目标	153
核心议题	153
面试重点	153
本章资料边界	153
为什么多模态训练和纯文本训练不一样	153
1. 多模态训练的几类主任务	154
2. CLIP 风格图文对齐	154
2.1 训练形式	154
2.2 为什么重要	155
2.3 训练难点	155
3. VLM: Vision Encoder + Connector + LLM	155
3.1 Vision encoder	155
3.2 Connector / projector	155
3.3 LLM	156
4. 多模态 instruction tuning	156
4.1 目标	156
4.2 关键问题	156
5. 图像 caption 和 VQA	157
5.1 Caption 训练	157
5.2 VQA 训练	157
5.3 常见问题	157
6. OCR 和文档理解	158
6.1 训练数据	158
6.2 训练目标	158
6.3 难点	158
7. 语音训练	158
7.1 ASR	158
7.2 语音问答	159
7.3 难点	159
8. 视频训练	159
9. Diffusion 训练	159
9.1 训练目标	160
9.2 训练流程	160
9.3 难点	160
10. 多模态数据组织	160
11. 多模态训练中的对齐问题	161
12. 多模态训练的显存和算力	161
12.1 最小可运行多模态训练审计 demo	161
13. 多模态安全和评估	165
14. 常见训练策略	165

14.1 冻结视觉编码器	165
14.2 只训练 connector	165
14.3 联合微调	165
14.4 分阶段训练	165
15. 面试官会怎么问	165
问法 1: VLM 是怎么训练的?	165
问法 2: CLIP 和 VLM 有什么区别?	166
问法 3: 多模态 instruction tuning 的重点是什么?	166
问法 4: Diffusion 训练和语言模型训练有什么不同?	166
问法 5: 多模态训练最容易出什么问题?	166
16. 本章小结	166

第十三章：训练成本与资源规划

166

本章目标	166
核心议题	166
面试重点	167
本章资料边界	167
为什么成本和资源规划很重要	167
1. 训练成本由哪些部分组成	167
2. 粗略 FLOPs 估算	167
2.1 为什么 FLOPs 估算有用	168
3. Tokens/s 和训练时长	168
3.1 影响 tokens/s 的因素	168
3.2 不能只看峰值吞吐	169
4. GPU 数量怎么估	169
4.1 从单卡显存倒推	169
4.2 从时长倒推	169
4.3 资源冗余	169
5. MFU	170
5.1 为什么 MFU 重要	170
5.2 影响 MFU 的因素	170
6. 数据规模和 token 规划	170
6.1 资源规划要回答	171
6.2 数据成本	171
7. Checkpoint 存储成本	171
7.1 需要考虑的点	171
7.2 存储生命周期	172
8. 失败和重跑成本	172
8.1 重跑风险	172
8.2 降低重跑成本的方法	172
9. 资源规划流程	172
10. 一个粗略规划示例	173
10.1 最小可运行训练成本审计 demo	173
11. 常见资源规划错误	175
11.1 只看模型参数量	175
11.2 只看峰值吞吐	175
11.3 不考虑失败率	175
11.4 不考虑存储和网络	175
11.5 不留余量	175
12. 面试官会怎么问	175
问法 1: 如果让你估算一个训练项目的成本, 你怎么做?	175
问法 2: MFU 是什么? 为什么重要?	175
问法 3: checkpoint 成本为什么不能忽略?	175
问法 4: 为什么训练成本不只是 GPU 费用?	175
问法 5: 怎么判断资源规划是否合理?	175
13. 本章小结	176

第十四章：训练面试题	176
高频问题	176
回答原则	176
本章资料边界	176
如何回答训练类面试题	176
1. 如果让你训练一个 7B 模型，你会怎么设计？	177
回答框架	177
标准回答	177
常见追问	177
2. 训练 loss 突然 spike，你怎么排查？	177
回答框架	177
标准回答	177
常见追问	178
3. 为什么训练 loss 降低不代表模型能力一定提升？	178
回答框架	178
标准回答	178
4. 如何检测 benchmark contamination？	178
回答框架	178
标准回答	178
5. 如何选择 batch size 和 learning rate？	178
回答框架	178
标准回答	178
6. ZeRO、FSDP、Tensor Parallel 各自解决什么问题？	179
标准回答	179
常见追问	179
7. 如何估算训练显存？	179
回答框架	179
标准回答	179
8. SFT 后模型变笨了怎么办？	179
回答框架	179
标准回答	180
9. 如何训练一个 VLM？	180
回答框架	180
标准回答	180
10. 如何控制大规模训练成本？	180
回答框架	180
标准回答	180
11. Base model、Instruct model、Chat model 有什么区别？	180
标准回答	180
12. DPO 和 PPO 有什么区别？	180
标准回答	180
13. 训练类面试回答 checklist	181
13.1 最小可运行面试回答自评 demo	181
14. 本章小结	182

第五册简介：大模型训练全流程

本书定位

第五册专门讲大模型训练全流程，覆盖数据、tokenizer、模型架构、预训练、优化器、学习率策略、后训练、对齐强化训练、分布式训练、数值稳定性、评估、实验管理和训练事故排查。

它回答的问题不是“某个训练概念是什么”，而是“如果让你负责训练一个大模型，你会如何设计、执行、监控、评估和 debug”。

为什么需要这本书

训练是大模型能力形成的源头。只理解 Transformer 结构还不够，算法岗候选人还需要知道数据如何进入训练系统，AdamW/Muon 等优化器如何影响收敛，loss 如何变化，checkpoint 如何选择，分布式并行如何组织，训练异常如何排查。

适合读者

1. 想准备大模型训练岗、算法岗和研究岗的读者。
2. 已理解预训练、SFT、DPO 基础，但缺少 PPO、GRPO、GSPO、DAPO、DCPO 等现代后训练方法视角的人。
3. 想把训练事故、debug 和实验管理讲清楚的候选人。

学完后应具备的能力

1. 能设计训练数据 pipeline 和 tokenizer 方案。
2. 能组织预训练、SFT、RLHF/PPO、DPO、GRPO、GSPO、DAPO、DCPO 和多模态训练流程。
3. 能解释 DDP、FSDP、ZeRO、TP、PP 等分布式策略。
4. 能比较 Adam、AdamW、Adafactor、Lion、Sophia、Muon 等优化器的目标和 trade-off。
5. 能排查 loss spike、NaN、梯度爆炸、吞吐异常和评估异常。
6. 能估算训练成本、资源需求和实验风险。

阅读建议

建议在学完第一册训练部分后阅读。本书适合和第三册微调实战、第十九册训练事故排查一起交叉学习。

与其他书的关系

第一册讲训练主线，第二册讲训练进阶原理，第三册提供训练和微调代码实战，第五册负责把这些内容组织成真实训练工程闭环。

第五册：大模型训练全流程

本书定位

本书专门介绍大模型训练相关知识，覆盖从数据、tokenizer、模型架构、预训练、后训练、对齐强化训练、分布式训练、训练稳定性、评估、实验管理到训练事故排查的完整流程。

第一册和第二册会讲训练的核心原理，但本书更偏“训练全流程”和“工程实战思维”，目标是让读者能回答：如果你负责训练一个大模型，你会如何设计、执行、监控、评估和 debug。

章节文件

1. chapters/01-训练总览.md
2. chapters/02-数据工程.md
3. chapters/03-tokenizer 与数据格式.md
4. chapters/04-模型架构与初始化.md
5. chapters/05-预训练目标与训练循环.md
6. chapters/06-优化器与学习率策略.md
7. chapters/07-分布式训练.md
8. chapters/08-显存优化与数值精度.md
9. chapters/09-训练稳定性与 debug.md
10. chapters/10-checkpoint 评估与实验管理.md
11. chapters/11-后训练与对齐训练.md
12. chapters/12-多模态训练.md

- 13. chapters/13-训练成本与资源规划.md
- 14. chapters/14-训练面试题.md

核心问题

1. 如何设计大模型训练数据 pipeline?
2. 如何选择 tokenizer、模型规模和训练 token 数?
3. 如何组织预训练、SFT、RLHF/PPO、DPO、GRPO、GSPO、DAPO、DCPO 和多模态训练?
4. 如何选择 AdamW、Adafactor、Lion、Sophia、Muon 等优化器和学习率策略?
5. 如何做分布式训练和显存优化?
6. 如何监控训练是否健康?
7. 如何处理 loss spike、NaN、梯度爆炸、吞吐低、评估异常?
8. 如何估算训练成本和资源需求?

第一章：训练总览

本章目标

建立大模型训练的全局视角，理解一个模型从原始数据到可用 checkpoint 的完整路径。

核心议题

1. 大模型训练包含哪些阶段。
2. Base model、instruct model、chat model、reasoning model 的训练差异。
3. 训练流程中的数据、模型、系统、评估、成本如何互相影响。
4. OpenAI 风格岗位为什么重视训练全流程理解。

典型训练流程

1. 明确模型目标和能力边界。
2. 收集、清洗、去重和混合训练数据。
3. 训练或选择 tokenizer。
4. 设计模型架构和规模。
5. 进行大规模预训练。
6. 评估 base model。
7. 做 SFT、PPO-RLHF、DPO、GRPO、GSPO、DAPO、DCPO、偏好优化和安全对齐。
8. 做能力、安全、鲁棒性和成本评估。
9. 选择 checkpoint 并进入部署阶段。

面试重点

如果面试官问“你如何训练一个大模型”，不要只回答 Transformer 和 loss。更好的回答结构是：目标、数据、架构、训练系统、优化策略、评估、后训练、风险和成本。

为什么训练总览很重要

大模型训练不是“写一个 Transformer，然后跑 cross entropy loss”这么简单。真实训练项目是一条很长的链路：数据决定模型见过什么，tokenizer 决定文本如何被切分，模型架构决定容量和计算路径，优化器决定如何收敛，分布式系统决定能不能跑起来，评估决定 checkpoint 能不能用，后训练决定模型是否能成为助手，部署成本决定模型是否能被业务承受。

很多候选人面试时会把训练回答成几个孤立概念：AdamW、cosine learning rate、DDP、loss curve、SFT、DPO。这些概念都重要，但如果不能把它们串成闭环，面试官会觉得你只会背点，不具备负责训练项目的的能力。

训练总览要建立的是一张地图：你知道每个阶段解决什么问题，输入输出是什么，风险在哪里，出了问题应该先看哪一层。

面试表达：训练大模型的核心能力不是记住某个技巧，而是把目标、数据、模型、系统、评估和成本组织成一个可执行、可监控、可迭代的工程闭环。

一句话看完整训练链路

一个大模型从零到可用，大致经历下面这条路径：

目标定义

- > 数据收集、清洗、去重、配比
- > tokenizer 训练或选择
- > 模型架构和规模设计
- > 预训练
- > base model 评估
- > 指令微调 SFT
- > 偏好优化和安全对齐
- > 综合评估和 checkpoint 选择
- > 压缩、部署、监控和持续迭代

每一步都会影响后面所有环节。

如果数据里代码比例不足，后面再做少量代码 SFT 很难补出强代码能力。如果 tokenizer 对中文、代码或数学符号切分很差，训练效率和下游能力都会受影响。如果预训练 checkpoint 本身推理能力弱，后训练只能改善表达和偏好，很难凭空创造深层能力。如果评估集被污染，模型看似很强，真实使用会暴露问题。

所以训练项目的核心不是某一步做到完美，而是全链路协同。

训练总览的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 scaling law、Chinchilla compute-optimal training、GPT-3 预训练、InstructGPT 后训练，以及训练监控和实验评估相关公开资料。

本章只建立训练全流程的工程地图：

目标 -> 数据 -> tokenizer -> 架构 -> 预训练 -> 后训练 -> 评估 -> checkpoint -> 部署准备

它不展开每个细节的完整推导。数据清洗、tokenizer、架构、训练循环、优化器、分布式、checkpoint、后训练和成本规划会在后续章节分别展开。本章的重点是让你能在面试中先讲清“完整训练项目由哪些变量共同决定”。

训练项目的关键公式口径

自回归预训练目标可以写成：

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t | x_{1:t-1})$$

训练时最常见的负对数似然目标是：

$$L_{\text{pre}} = \frac{1}{T} \sum_{t=1}^T -\log p_{\theta}(x_t | x_{1:t-1})$$

如果训练 token 总数是 D，模型参数数量是 N，dense Transformer 训练 FLOPs 常用粗略估算：

$$C_{\text{train}} \approx 6ND$$

这个公式不是精确 profiler，而是训练方案早期做量级估算的工具。它可以帮助你判断“参数量、token 数和算力预算是否在同一个量级上”。

global batch token 数可以写成：

$$B_{\mathrm{tok}}=B_{\mathrm{seq}}\cdot T_{\mathrm{seq}}\cdot A$$

其中 B_{seq} 是每步样本数, T_{seq} 是序列长度, A 是 gradient accumulation 或等效累积倍数。训练步数是:

$$S=\lceil D/B_{\mathrm{tok}}\rceil$$

如果集群每秒处理 R_{tok} 个 token, 总训练时长近似为:

$$T_{\mathrm{train}}=\frac{D}{R_{\mathrm{tok}}}$$

GPU 小时数为:

$$H_{\mathrm{gpu}}=T_{\mathrm{train}}\cdot G$$

其中 G 是 GPU 数量。checkpoint 存储也要估算。只保存权重时:

$$M_{\mathrm{weight}}\approx N\cdot b_w$$

一个简化训练方案门禁可以写成:

$G_{\mathrm{train}}=$
 $g_{\mathrm{goal}}\backslash,$
 $g_{\mathrm{data}}\backslash,$
 $g_{\mathrm{model}}\backslash,$
 $g_{\mathrm{compute}}\backslash,$
 $g_{\mathrm{monitor}}\backslash,$
 $g_{\mathrm{eval}}\backslash,$
 g_{risk}

这些 g 都是 0/1 检查项: 目标清楚、数据有配比、模型规模和 token 数匹配、算力预算可执行、监控指标齐全、评估集能定位方向、风险和回滚有设计。面试中如果能把这些变量串起来, 就比只背 “AdamW + Transformer” 更有说服力。

训练阶段总览

大模型训练可以分为预训练、后训练、评估和部署前优化几大阶段。

阶段	输入	输出	解决的问题
数据工程	原始网页、书籍、代码、论文、对话、多模态数据	高质量训练语料	决定模型能学到什么
Tokenizer	清洗后的文本语料	tokenizer vocab 和编码规则	决定文本如何变成 token
架构设计	目标能力、算力预算、推理约束	模型配置	决定容量、速度和可扩展性
预训练	大规模通用语料	base model	学语言、知识、代码、数学
Base 评估	checkpoint 和 benchmark	能力报告	判断预训练是否健康
SFT	指令-回答数据	instruct model	学会按指令回答
偏好优化	chosen/rejected、reward、规则或反馈	aligned model	更符合人类偏好和安全要求
安全对齐	安全数据、红队样本、策略规则	safer model	降低有害输出和越权行为
部署前优化	目标模型	量化、蒸馏、推理配置	降低延迟和成本

这些阶段不是严格一次性完成。真实项目经常反复迭代: 评估发现数学弱, 回到数据配比; 部署发现延迟高, 回到架构和推理优化; 红队发现安全漏洞, 回到安全 SFT 或偏好优化。

从目标定义开始

训练项目第一步不是选模型结构, 而是定义目标。

目标至少要回答以下问题:

问题	示例
模型用途是什么	通用助手、代码模型、数学模型、企业知识助手、语音助手
目标用户是谁	普通用户、开发者、客服、研究员、内部员工
主要能力是什么	对话、代码、数学、长上下文、多语言、多模态、工具调用
约束是什么	训练预算、推理延迟、显存、部署环境、安全要求
成功标准是什么	benchmark、人工评测、业务指标、成本指标

如果目标是训练通用 base model，重点是大规模数据覆盖、稳定预训练和通用能力评估。如果目标是企业助手，可能更重视 RAG、指令跟随、安全拒答和私有知识可靠性。如果目标是代码模型，需要提高代码数据比例、设计代码评估和执行反馈。如果目标是端侧模型，需要从一开始考虑参数规模、量化和推理延迟。

面试表达：训练方案必须从目标倒推。不同目标会改变数据配比、模型规模、上下文长度、后训练方式和评估指标。

Base Model、Instruct Model、Chat Model、Reasoning Model

面试中经常会问这些模型有什么区别。

类型	训练重点	行为特点	常见用途
Base Model	预训练 next token prediction	会补全文本，但不一定听指令	继续训练、研究、作为后训练起点
Instruct Model	SFT 指令微调	能按指令完成任务	问答、摘要、抽取、格式化输出
Chat Model	多轮对话和安全对齐	更像助手，能维持对话状态	通用聊天、客服、Copilot
Reasoning Model	推理数据和强化学习/偏好优化	更擅长复杂问题分解和验证	数学、代码、科学推理、规划

Base model 学到的是语言和知识分布，它可能续写论文、代码和网页，但未必会回答“请一步步解释”。Instruct model 通过指令数据学会输入输出格式和任务意图。Chat model 进一步强调多轮交互、拒答、安全和语气。Reasoning model 通过推理轨迹、可验证奖励或强化学习等方式强化复杂问题求解能力。

常见误区：把 instruct model 当成 base model。很多公开模型已经做过 SFT 或对齐，不再是纯 base。面试中如果讨论预训练 loss、困惑度和 scaling law，要注意你说的是 base model；如果讨论 helpfulness、harmlessness 和多轮对话，要注意你说的是 chat model。

面试表达：base model 解决“学会语言和知识”，instruct/chat model 解决“按人类意图交互”，reasoning model 进一步解决“复杂问题求解和自我验证”。

数据、模型、系统、评估、成本的联动

训练项目里没有孤立决策。任何一个选择都会牵动其他部分。

数据影响模型能力

数据是能力来源。模型能否写代码、做数学、懂中文、理解法律文本，首先取决于训练中是否有足够高质量相关数据。

数据不只是越多越好。低质量网页、重复内容、模板垃圾、模型生成垃圾、污染 benchmark 的数据都会带来问题。重复数据会浪费 token 预算，低质量数据会降低学习效率，污染数据会让评估失真。

模型规模影响训练策略

模型越大，容量越强，但训练更不稳定，显存和通信成本更高。小模型可能受容量限制，大模型可能受数据和系统限制。

如果算力有限，与其盲目扩大参数，不如合理选择模型大小、训练 token 数和数据质量。Chinchilla 风格结论提醒我们：参数量和训练 token 数要匹配，训练不足的大模型不一定比充分训练的小模型好。

系统影响可执行性

一个方案理论上可行,不代表系统上能跑。大模型训练需要考虑 GPU 数量、显存、网络带宽、存储吞吐、checkpoint 保存、容错恢复和监控。

如果数据加载跟不上, GPU 会空转。如果通信瓶颈严重, 增加 GPU 不一定提升吞吐。如果 checkpoint 保存太慢, 训练会频繁暂停。如果没有良好监控, loss spike 和 NaN 可能很晚才被发现。

评估影响方向判断

没有可靠评估, 就不知道训练是否变好。预训练阶段要看 validation loss、benchmark、per-domain loss 和下游任务。后训练阶段要看指令跟随、偏好、安全、事实性、格式稳定和人工评测。

评估也会误导。如果 benchmark 被污染, 分数会虚高。如果只看平均分, 会掩盖某些关键能力退化。如果只看自动指标, 会漏掉风格、安全和幻觉问题。

成本影响方案边界

训练成本包括 GPU 时间、数据处理、存储、工程人力、失败实验和推理成本。很多方案论文上有效, 但在真实项目中因为成本不可接受而不能采用。

面试表达: 训练方案要考虑端到端成本。算法选择不能只看效果, 还要看训练吞吐、显存、通信、checkpoint、推理延迟和维护复杂度。

一个训练项目的关键产物

如果你负责训练一个模型, 最终不是只交一个 checkpoint。你需要交付一组工程产物。

产物	作用
数据说明	数据来源、清洗规则、配比、去重和污染检测
Tokenizer 方案	vocab、特殊 token、覆盖语言和兼容性
模型配置	参数量、层数、hidden size、head、上下文长度
训练配置	batch size、学习率、warmup、optimizer、精度、并行策略
监控面板	loss、grad norm、学习率、吞吐、显存、NaN、数据速度
Checkpoint 策略	保存频率、保留策略、恢复验证、最优选择标准
评估报告	自动评估、人工评测、安全评测、退化分析
风险记录	已知问题、失败实验、异常处理和回滚方案
部署建议	量化、推理配置、上下文长度、硬件需求和成本估算

这些产物体现了训练工程的可复现性。没有这些记录, 即使某次训练跑出了好模型, 也很难解释为什么好, 下一次也难以复现。

预训练在做什么

预训练通常使用 next token prediction。给定前文 token, 模型预测下一个 token。

```
input: x_1, x_2, ..., x_{t-1}
target: x_t
loss: cross entropy(predicted distribution, true token)
```

这个目标看起来简单, 但在大规模数据和大模型容量下, 模型会学习语言结构、事实知识、代码模式、数学模式、推理模板和世界常识。

预训练阶段的核心问题:

问题	说明
数据够不够好	是否覆盖目标能力，是否去重和过滤
Tokenizer 是否合适	是否支持多语言、代码、数学符号和特殊格式
模型规模是否匹配	参数量和训练 token 是否匹配预算
训练是否稳定	loss 是否正常下降，有无 spike、NaN、梯度异常
系统吞吐是否高	GPU 利用率、MFU、数据加载和通信是否健康
checkpoint 如何选择	不能只看最后一步，要结合验证和下游评估

预训练的输出是 base model。它是后续 SFT、DPO、RLHF、工具调用和多模态扩展的基础。

面试表达：预训练的目标是用大规模通用数据训练一个能力底座。它不直接保证模型会当助手，但决定了模型的知识、语言、代码和推理上限。

后训练在做什么

后训练是把 base model 变成可用助手的关键阶段。

常见后训练包括 SFT、偏好优化、安全对齐和强化学习式训练。

方法	数据	目标
SFT	指令和标准回答	学会按指令输出
RLHF/PPO	偏好数据、reward model	优化人类偏好奖励
DPO	chosen/rejected 偏好对	直接优化偏好，不显式训练 reward model
GRPO/GSPO/DAPO/DCPO	可验证奖励、组内比较或改进策略目标	强化推理、代码、数学和对齐能力
Safety Tuning	安全指令、拒答样本、红队数据	降低有害和越权输出

后训练不能完全替代预训练。如果 base model 没学过足够代码，少量代码 SFT 很难把它变成强代码模型。如果 base model 缺乏数学基础，偏好优化可能让答案格式更像推理，但未必真正提高正确率。

后训练也可能带来能力退化。过度对齐可能降低开放生成能力，低质量 SFT 可能让模型变啰嗦，安全数据配比不当可能导致过度拒答。

面试表达：后训练的作用是把 base model 的能力组织成符合人类意图的行为，而不是凭空创造所有能力。它的风险是过拟合风格、能力退化和安全-有用性失衡。

训练监控要看什么

训练过程中必须持续监控。只等训练结束再评估，风险太高。

常见监控指标：

类别	指标	说明
优化	train loss、validation loss、learning rate	判断收敛是否正常
梯度	grad norm、clipping ratio	判断是否梯度爆炸或异常
数值	NaN、Inf、loss spike	判断混合精度和数据异常
系统	tokens/sec、MFU、GPU utilization	判断吞吐是否健康
显存	allocated/reserved、activation memory	判断是否接近 OOM
数据	dataloader speed、样本分布、坏样本率	判断数据 pipeline 是否稳定
checkpoint	保存耗时、恢复成功率	判断容错能力
评估	benchmark、per-domain loss	判断能力变化

不同异常对应不同层面问题。loss 突然 spike 可能是坏数据、学习率过大、梯度爆炸或数值溢出。吞吐下降可能是数据加载、网络通信、checkpoint 保存或某个节点故障。validation loss 不降可能是数据分布、学习率、模型容量或 bug。

面试表达：训练监控要同时看优化健康、数值健康、系统健康和评估健康。只看 loss 不够，因为很多系统问题不会立刻反映在 loss 上。

Checkpoint 不是越后越好

很多初学者会默认训练最后一步的 checkpoint 最好。真实训练中不一定。

预训练中，如果学习率 schedule 设计合理，后期 checkpoint 通常更好，但也可能出现过拟合某些数据、评估退化或训练异常。后训练中，训练太久可能导致过拟合风格、指令模板化、拒答过多或通用能力下降。

选择 checkpoint 需要综合：

维度	说明
Validation loss	基础收敛指标
核心 benchmark	数学、代码、知识、多语言等目标能力
人工评测	指令质量、风格、事实性和安全性
回归测试	旧能力是否退化
安全评测	拒答、越狱、敏感内容
推理成本	延迟、显存、吞吐

面试表达：checkpoint 选择是多目标决策。不能只看 loss 或单个 benchmark，要看目标能力、安全、退化和部署成本。

训练失败常见来源

训练失败不一定是模型结构错。更常见的是系统、数据和配置问题。

问题	表现	常见原因
Loss 不下降	训练无效	learning rate 错、label 错、mask 错、数据格式错
Loss spike	突然升高	坏 batch、学习率过大、梯度爆炸、数值溢出
NaN/Inf	训练中断	混合精度不稳定、除零、softmax 溢出、梯度异常
吞吐低	GPU 利用率低	dataloader 慢、通信瓶颈、batch 太小、checkpoint 阻塞
评估异常	分数不可信	benchmark 污染、prompt 格式错、评估脚本 bug
能力退化	某些任务变差	后训练数据偏、过拟合、安全数据过强
恢复失败	checkpoint 不能用	保存不完整、版本不一致、optimizer state 丢失

排查顺序通常是：先确认数据和 label，再看 loss/mask，再看学习率和 optimizer，再看精度和梯度，再看分布式同步，再看评估脚本。

面试表达：训练 debug 要分层排查。不要一上来就改模型结构，先确认数据、目标、mask、学习率、精度和系统状态。

成本意识

大模型训练成本很高，所以任何训练方案都要有成本意识。

成本包括：

成本	示例
算力	GPU 小时、集群占用、通信网络
数据	采集、清洗、标注、存储、版权审核
工程	训练框架、监控、容错、评估平台
实验	失败 run、消融实验、超参搜索

成本	示例
推理	显存、吞吐、延迟、KV cache
维护	模型更新、安全修复、回归评估

训练前应该估算 token 数、模型参数量、batch size、训练步数、GPU 数量、预期吞吐、总训练时长和失败重跑风险。

一个简单估算思路：

训练步数 = 总训练 token 数 / global batch token 数

总时长 = 训练步数 / 每秒训练 step 数

总 GPU 小时 = 总时长小时数 * GPU 数量

真实项目还要考虑 checkpoint、评估、故障恢复、数据处理和实验重跑带来的额外成本。

面试表达：负责训练不能只说“用更多 GPU 跑更大模型”。要能估算预算、吞吐、失败风险和最终部署成本。

下面给一个 0 依赖训练规划审计 demo。它不训练模型，只检查一个训练方案是否把核心变量讲清楚。

import math

```
plan = {
    "goal": "train a 1.3B domain assistant base model candidate",
    "params_b": 1.3,
    "train_tokens_b": 120.0,
    "seq_len": 2048,
    "micro_batch": 2,
    "num_gpus": 8,
    "grad_accum": 16,
    "tokens_per_sec_per_gpu": 1800,
    "weight_bytes": 2,
    "checkpoint_keep": 5,
    "data_mix": {
        "web": 0.45,
        "code": 0.20,
        "math": 0.15,
        "domain": 0.15,
        "safety": 0.05,
    },
    "monitor": [
        "train_loss",
        "valid_loss",
        "grad_norm",
        "lr",
        "tokens_per_sec",
        "gpu_memory",
        "nan_inf",
        "dataloader_wait",
    ],
    "evals": ["general", "code", "math", "domain", "safety"],
    "risks": ["data_contamination", "loss_spike", "oom", "checkpoint_failure"],
}
```

```
def audit_training_plan(plan):
    params = plan["params_b"] * 1_000_000_000
```

```

tokens = plan["train_tokens_b"] * 1_000_000_000

global_batch_tokens = (
    plan["micro_batch"]
    * plan["num_gpus"]
    * plan["grad_accum"]
    * plan["seq_len"]
)
steps = math.ceil(tokens / global_batch_tokens)
total_tokens_per_sec = plan["tokens_per_sec_per_gpu"] * plan["num_gpus"]
train_seconds = tokens / total_tokens_per_sec
gpu_hours = train_seconds / 3600 * plan["num_gpus"]
train_flops = 6 * params * tokens
weight_gib = params * plan["weight_bytes"] / 1024**3
checkpoint_gib = weight_gib * plan["checkpoint_keep"]
data_mix_sum = sum(plan["data_mix"].values())

required_monitor = {
    "train_loss",
    "valid_loss",
    "grad_norm",
    "lr",
    "tokens_per_sec",
    "gpu_memory",
    "nan_inf",
}
monitor_missing = sorted(required_monitor - set(plan["monitor"]))
required_evals = {"general", "code", "math", "domain", "safety"}
eval_missing = sorted(required_evals - set(plan["evals"]))
required_risks = {"data_contamination", "loss_spike", "oom", "checkpoint_failure"}
risk_missing = sorted(required_risks - set(plan["risks"]))

gates = {
    "goal": bool(plan["goal"]),
    "data_mix": abs(data_mix_sum - 1.0) < 1e-9,
    "compute": steps > 0 and train_seconds > 0 and train_flops > 0,
    "monitor": not monitor_missing,
    "eval": not eval_missing,
    "risk": not risk_missing,
}

return {
    "global_batch_tokens": global_batch_tokens,
    "steps": steps,
    "train_days": round(train_seconds / 86400, 2),
    "gpu_hours": round(gpu_hours, 2),
    "train_flops": f"{train_flops:.2e}",
    "weight_gib": round(weight_gib, 2),
    "checkpoint_gib": round(checkpoint_gib, 2),
    "data_mix_sum": round(data_mix_sum, 4),
    "monitor_missing": monitor_missing,
    "eval_missing": eval_missing,
    "risk_missing": risk_missing,
    "gates": gates,
    "gate_pass": all(gates.values()),
}

```

```
}
```

```
report = audit_training_plan(plan)

print("global_batch_tokens=", report["global_batch_tokens"])
print("steps=", report["steps"])
print("train_days=", report["train_days"])
print("gpu_hours=", report["gpu_hours"])
print("train_flops=", report["train_flops"])
print("weight_gib=", report["weight_gib"])
print("checkpoint_gib=", report["checkpoint_gib"])
print("data_mix_sum=", report["data_mix_sum"])
print("monitor_missing=", report["monitor_missing"])
print("eval_missing=", report["eval_missing"])
print("risk_missing=", report["risk_missing"])
print("gates=", report["gates"])
print("gate_pass=", report["gate_pass"])
```

预期输出类似：

```
global_batch_tokens= 524288
steps= 228882
train_days= 96.45
gpu_hours= 18518.52
train_flops= 9.36e+20
weight_gib= 2.42
checkpoint_gib= 12.11
data_mix_sum= 1.0
monitor_missing= []
eval_missing= []
risk_missing= []
gates= {'goal': True, 'data_mix': True, 'compute': True, 'monitor': True, 'eval': True, 'risk': True}
gate_pass= True
```

这个 demo 的重点是训练方案审计，而不是追求硬件精确估价。真实项目还要把 optimizer states、activation memory、checkpoint 写盘、评估开销、失败重跑和推理成本加进去。但如果面试中能先给出这种量级估算，说明你知道训练不是一句“多用 GPU”就能解决。

面试回答模板：如何训练一个大模型

如果面试官问“给你资源，你如何训练一个大模型”，可以按下面结构回答。

第一步，明确目标和约束。我会先确定模型用途、目标能力、参数规模、训练预算、推理场景和安全要求。例如是通用助手、代码模型还是领域模型。

第二步，设计数据 pipeline。我会收集网页、书籍、代码、论文、多语言、数学和领域数据，做清洗、去重、质量过滤、安全过滤、benchmark contamination 检测和数据配比。

第三步，确定 tokenizer 和模型架构。我会根据语言覆盖、代码和数学符号需求选择 tokenizer，根据预算和目标选择参数量、上下文长度、RoPE、GQA、FFN 结构和初始化方案。

第四步，组织预训练。我会设置 optimizer、learning rate schedule、batch size、混合精度、gradient clipping、分布式并行策略和 checkpoint 策略，同时监控 loss、grad norm、吞吐、显存、NaN 和数据速度。

第五步，做阶段性评估。我会定期评估 validation loss、per-domain loss、知识、数学、代码、多语言、长上下文和安全基础指标，选择候选 checkpoint。

第六步，做后训练和对齐。我会先 SFT 学指令，再根据目标选择 DPO、PPO/RLHF、GRPO 或其他偏好/强化学习方法，加入安全数据和拒答策略，同时监控能力退化。

第七步，做最终评估和部署准备。我会用自动 benchmark、人工评测、红队测试、业务 golden set、推理压测和成本评估选择最终 checkpoint，并准备量化、蒸馏或部署优化。

最后，我会强调风险控制：数据污染、训练不稳定、系统故障、后训练退化、安全漏洞和成本超预算都需要提前设计监控和回滚方案。

一个常见错误回答

错误回答：我会用 Transformer decoder-only 架构，用 AdamW 优化器，训练 next token prediction，然后做 SFT 和 RLHF。

这个回答不是完全错，但太粗。问题在于没有说明数据怎么来、tokenizer 怎么选、模型规模如何定、分布式怎么跑、训练是否稳定、怎么评估、checkpoint 怎么选、后训练有什么风险、成本如何控制。

更好的回答是：Transformer、AdamW 和 next token prediction 只是训练方案中的一部分。完整训练方案必须覆盖目标、数据、架构、系统、优化、评估、后训练、安全和成本。

本章小结

大模型训练是一项全链路工程。预训练负责形成能力底座，后训练负责把能力组织成可用行为，评估负责判断方向是否正确，系统工程负责让训练稳定高效地跑完，成本管理决定方案是否现实。

本章最重要的结论如下：

1. 训练方案必须从目标和约束出发，而不是从模型结构出发。
2. 数据、tokenizer、模型、优化器、分布式系统、评估和成本互相影响。
3. Base model、instruct model、chat model 和 reasoning model 的训练目标不同。
4. 后训练能改善交互和偏好，但不能完全弥补预训练能力缺口。
5. 训练监控要同时看 loss、梯度、数值、吞吐、数据和评估。
6. checkpoint 选择是多目标决策，不是默认最后一步最好。
7. 面试中回答训练全流程，要体现工程闭环和风险意识。

下一章，我们进入数据工程，讨论大模型训练数据从哪里来、如何清洗、如何去重、如何配比，以及为什么数据质量往往比模型技巧更关键。

第二章：数据工程

本章目标

理解数据是大模型能力的核心来源，掌握数据收集、清洗、去重、过滤、配比和质量评估。

核心议题

1. 数据来源：网页、书籍、论文、代码、对话、数学、多语言、多模态数据。
2. 数据清洗：格式修复、乱码处理、语言识别、质量打分、安全过滤。
3. 数据去重：exact dedup、near dedup、document-level dedup、benchmark contamination 检测。
4. 数据配比：通用文本、代码、数学、多语言和专业领域数据如何混合。
5. 数据质量：高质量数据为什么可能比更多低质量数据更重要。

面试重点

训练大模型时，数据问题通常比模型结构问题更难。面试回答要体现数据质量、污染检测、分布覆盖和训练目标之间的联系。

为什么数据工程是训练起点

很多人把大模型训练理解成“把数据丢进去跑 loss”，但真实情况是：数据工程决定模型见过什么、学得什么、会不会被污染、会不会偏科。

如果数据出问题，后面再调学习率、换优化器、加更多 GPU，通常只能缓解，不能根治。

数据工程至少要回答四个问题：

- 1. 数据从哪里来。
- 2. 数据质量如何保证。
- 3. 数据之间如何混合。
- 4. 如何证明数据没有污染评估集。

面试表达：训练系统里，数据不是前置步骤，而是能力上限的主要决定因素之一。

数据工程的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 T5 / C4 数据清洗、GPT-3 预训练数据混合、The Pile 数据集构建、训练数据去重论文、多语言温度采样，以及 Hugging Face Datasets 的 dataset card / DatasetInfo 资料。

本章聚焦训练数据工程的最小闭环：来源、schema、清洗、质量打分、去重、污染检测、数据配比、版本和报告。它不展开爬虫法务、版权治理、分布式数据湖、在线标注平台、数据合成系统和生产级 LSH 索引的完整实现；这些内容会在第九册数据工程和后续平台章节继续展开。

1. 数据来源有哪些

大模型常见训练数据大致可以分成下面几类：

数据类型	作用	常见风险
Web 文本	覆盖广泛知识和表达方式	噪声大、重复多、营销噪声多
书籍和论文	结构化、质量较高	版权、领域偏差
代码	提升编程能力和结构化推理	许可问题、测试污染
对话数据	提升指令遵循和多轮交互	模板化、低信息密度
数学数据	提升推理和符号计算	格式不统一、答案泄漏
多语言数据	提升跨语言能力	语言识别错误、配比失衡
多模态数据	提升图文、音频、视频能力	对齐错误、标注噪声

不同目标需要不同数据。通用 base model 更重视广覆盖和去重；代码模型要提高代码和单元测试相关数据比例；企业助手更重视高质量业务文档、FAQ、对话和安全策略；reasoning model 更重视可验证推理、数学和代码执行反馈。

2. 数据处理流水线

一个典型的数据 pipeline 可以写成：

```
raw data
-> 采集
-> 格式统一
-> 语言识别
-> 清洗过滤
-> 去重
-> 污染检测
-> 质量打分
-> 数据配比
-> shard / pack
```


-> training dataloader

每一步都可能影响最终训练质量。

可以把原始语料写成：

$D_{\{\mathrm{raw}\}} = \{d_i\}_{i=1}^n$

每条样本至少包含文本和元数据：

$d_i = (x_i, m_i)$

其中 x_i 是正文内容， m_i 是 source、language、domain、timestamp、license、quality_score、filter_reason 等元数据。数据工程的目标不是只得到一堆文本，而是得到可追踪、可过滤、可混合、可复现的训练语料：

$D_{\{\mathrm{train}\}} = F_{\{\mathrm{mix}\}}(F_{\{\mathrm{contam}\}}(F_{\{\mathrm{dedup}\}}(F_{\{\mathrm{clean}\}}(D_{\{\mathrm{raw}\}}))))$

这个写法强调一件事：每个阶段都应该留下统计和版本信息，否则训练后发现某个 benchmark 或能力退化时，很难定位到底是哪批数据造成的。

2.1 采集

采集阶段关注的是“拿到什么数据”。

常见来源包括爬虫、公开语料、内部文档、代码仓库、人工标注、合成数据和历史日志。

采集阶段的常见问题：

1. 来源不可靠。
2. 格式不统一。
3. 元数据缺失。
4. 采样分布偏向热门网页。

面试时不要只说“我抓了很多网页”，而要说清楚来源、许可、字段和风险。

2.2 格式统一

原始数据通常格式很乱，可能是 HTML、Markdown、JSON、CSV、PDF 抽取文本或对话日志。

统一格式的目标是把数据整理成训练系统可消费的 schema，例如：

```
{
  "id": "sample_001",
  "text": "...",
  "source": "web",
  "language": "zh",
  "domain": "general",
  "quality_score": 0.87
}
```

对于对话数据，还可能包含：

```
{
  "messages": [
    {"role": "user", "content": "..."},
    {"role": "assistant", "content": "..."}
  ]
}
```

格式统一的目的不是好看，而是让后续清洗、统计、混合和追踪都能自动化。

3. 清洗和过滤

清洗的目标是去掉明显无效或有害的数据。

常见过滤项：

1. 乱码、空文本、重复空行。
2. 过短或过长的异常样本。
3. 营销噪声、灌水、模板页。
4. 语言不匹配。
5. 低质量自动翻译。
6. 敏感或违法内容。
7. 明显抄袭或模型生成垃圾。

3.1 质量打分

实际系统中，常会给样本打一个质量分，再按阈值过滤或分层采样。

质量分可能来自规则、分类器、困惑度、重复度、信息密度、结构性、长度分布、领域匹配度等特征。

一个简单思路是：

```
q_i =
w_{\mathrm{lang}}s_{\mathrm{lang},i}
+w_{\mathrm{struct}}s_{\mathrm{struct},i}
+w_{\mathrm{domain}}s_{\mathrm{domain},i}
+w_{\mathrm{info}}s_{\mathrm{info},i}
-w_{\mathrm{noise}}s_{\mathrm{noise},i}
-w_{\mathrm{dup}}s_{\mathrm{dup},i}
```

其中 q_i 是样本质量分， s_{lang} 是语言识别置信度， s_{struct} 是结构完整性， s_{domain} 是领域匹配度， s_{info} 是信息密度， s_{noise} 是噪声分， s_{dup} 是重复风险分。权重 w 不应该拍脑袋定完就不变，而要结合目标能力、人工抽检、验证集和坏例分析迭代。

过滤门禁可以写成：

```
g_i =
\mathbf{1}[q_i \geq \tau_q],
\mathbf{1}[\ell_i \in L_{\mathrm{allow}}],
\mathbf{1}[r_i \leq \tau_r],
\mathbf{1}[a_i = 0]
```

其中 τ_q 是质量阈值， ℓ_i 是语言， L_{allow} 是允许语言集合， r_i 是重复或模板化风险， a_i 是高风险内容标记。清洗后的集合是：

$$D_{\mathrm{clean}} = \{d_i \in D_{\mathrm{raw}} \mid g_i = 1\}$$

这里的重点不是把规则写成复杂公式，而是说明你知道质量评估是多维度的，并且每条被过滤的样本都应该能解释原因。

3.2 过滤不能过头

过滤过度也有风险。

如果规则太严，会把代码片段、表格、公式、口语表达、论坛讨论等有价值内容一起删掉。对某些领域数据来说，低频但高价值的样本尤其重要。

所以清洗不是“越干净越好”，而是“对目标任务最有帮助的分布最优先”。

面试表达：数据清洗的核心不是追求绝对纯净，而是在噪声、覆盖率和成本之间做合适 trade-off。

过滤阶段至少要记录两个比例：

$$r_{\mathrm{keep}} = \frac{|D_{\mathrm{clean}}|}{|D_{\mathrm{raw}}|}$$

$$r_{\{\mathrm{drop}\},c}=\frac{|\{d_i\in D_{\{\mathrm{raw}\}}\mid c_i=c\}|}{|D_{\{\mathrm{raw}\}}|}$$

第一个比例告诉你整体保留率，第二个比例告诉你每类过滤原因 c 的占比。若某次规则更新让某个语言、领域或长度桶突然大量下降，必须先抽检再进入训练。

4. 去重与近似去重

去重是训练数据工程里的关键环节。

原因很简单：重复数据会浪费 token 预算，还会让模型过度记忆某些文本模式，降低泛化。

4.1 Exact dedup

exact dedup 是完全相同文本的去重，例如 hash 相同就认为重复。

适合去掉完全复制粘贴、重复网页镜像和数据管线重复导入的问题。

最小形式可以写成：

$$h_i=H(N(x_i))$$

其中 N 是规范化函数，例如统一大小写、空白和标点， H 是 hash 函数。保留集合可以写成：

$$D_{\{\mathrm{exact}\}}=\{d_i\mid h_i\notin\{h_1,\ldots,h_{i-1}\}\}$$

面试里要补一句：hash 去重只处理“规范化后完全一样”的文本，不等价于语义去重，也不能替代污染检测。

4.2 Near dedup

near dedup 用于处理“内容基本相同但字面不同”的样本，例如：

1. 不同页面的转载。
2. 模板页只改了标题。
3. 同一代码仓库的 fork。
4. 轻微改写的重复问答。

常见方法包括 SimHash、MinHash、shingling、embedding 相似度和局部敏感哈希。

教学上最容易理解的是 shingling + Jaccard。把样本 x_i 切成 k -gram 集合：

$$A_i=\mathrm{shingle}_k(x_i)$$

两个样本的近似相似度是：

$$J_{\{ij\}}=\frac{|A_i\cap A_j|}{|A_i\cup A_j|}$$

如果：

$$J_{\{ij\}}\geq \tau_{\{\mathrm{near}\}}$$

就可以把 d_i 和 d_j 视作近似重复候选，再结合来源、时间、质量分和人工抽检决定保留哪一条。大规模系统不会两两比较全部样本，通常会用 MinHash、SimHash 或 LSH 先召回候选，再做精排或抽检。

4.3 为什么近似去重重要

如果只做 exact dedup，很多重复内容仍会保留下来。Web 数据尤其如此，因为同一段内容会在多个站点转载、镜像或生成分页。

近似去重做得好，通常能明显改善训练 token 的有效利用率。

面试表达：去重不是数据清洁的附属步骤，而是控制训练效率、降低记忆偏差和减少 benchmark 污染的核心手段。

5. Benchmark contamination

污染检测是训练数据工程里最容易被低估的问题之一。

如果训练数据里包含评估集或其近似变体，模型在 benchmark 上可能“看起来很强”，但其实只是见过题目。

5.1 污染形式

1. 评估题原文出现在训练语料里。
2. 题目改写后仍高度相似。
3. 答案或解析泄漏到训练数据中。
4. 代码题的测试样例、题面或解答被爬入训练集。

5.2 怎么检测

常见办法有：

1. 字符串或 n-gram 匹配。
2. 近似相似度检索。
3. 题目与训练语料的 embedding 检索。
4. 关键词和结构模式匹配。
5. 人工抽检高风险集合。

可以把 benchmark 集合写成：

$$B = \{b_j\}_{j=1}^m$$

对训练样本 d_i ，计算它和所有 benchmark 题目的最大相似度：

$$c_i = \max_{1 \leq j \leq m} s(x_i, b_j)$$

污染过滤门禁是：

$$g_{\{\mathrm{contam}\}, i} = \mathbf{1}[c_i < \tau_c]$$

这里的 s 可以是字符串相似度、n-gram Jaccard、MinHash 估计、embedding 余弦相似度或多路召回后的综合分。高风险 benchmark 不应该只依赖单一阈值，最好保留高相似样本列表并做人工复核。

5.3 为什么面试官爱问

因为污染检测直接关系到你如何相信评估结果。

如果你不能说明污染控制方法，面试官会怀疑你对训练数据和 benchmark 的关系理解不够。

面试表达：数据工程不仅是准备训练集，还要确保评估集不被污染，否则所有指标都不可信。

6. 数据配比

数据配比决定模型“偏什么科”。

常见做法不是把所有数据混在一起，而是先按 domain、质量和任务分类，再按比例采样。

6.1 为什么要配比

不同数据的学习信号不同。

1. 通用网页数据提供语言和知识覆盖。
2. 代码数据提供结构化推理和程序模式。
3. 数学数据提供符号推理。
4. 对话数据提供指令跟随。
5. 专业领域数据提供垂直能力。

如果代码数据太少，模型代码能力弱；如果对话数据太多，base model 可能过早偏向助手风格；如果低质量网页比例太高，模型会学到大量噪声。

6.2 采样方式

常见采样方式包括：

1. 固定比例采样。
2. 温度采样。
3. 按质量分层采样。
4. 按阶段动态调整配比。

温度采样通常用于避免大数据集完全淹没小而高价值的数据集。

假设第 k 个数据桶有 n_k 个 token 或样本，温度式平滑采样可以写成：

$$p_k = \frac{n_k^\alpha}{\sum_{j=1}^K n_j^\alpha}$$

其中 $0 < \alpha \leq 1$ 。当 $\alpha=1$ 时，采样比例接近原始规模；当 α 变小，小数据桶会被相对上采样，大数据桶不会完全淹没代码、数学、多语言或高质量领域数据。

如果计划训练 D 个 token，第 k 个桶的目标 token 数可以估算为：

$$D_k = \text{round}(p_k D)$$

真实训练中还要结合质量分、去重率、验证集表现和目标能力动态调整，而不是只按原始数据大小采样。

6.3 面试表达

训练数据配比的核心不是“越多越好”，而是“把有限 token 花在最能提升目标能力的数据上”。

7. 质量评估与数据版本管理

训练数据不应该是一次性产物。

一个成熟的数据工程流程应该有版本管理、数据统计和质量报告。

7.1 统计什么

常见统计项：

1. 样本数。
2. token 数。
3. 语言分布。
4. 领域分布。
5. 长度分布。
6. 重复率。
7. 过滤率。
8. 高风险样本比例。

一份数据质量报告至少应该能回答下面几个量：

$$R_{\text{dup}} = \frac{|D_{\text{exact}}| - |D_{\text{dedup}}|}{|D_{\text{exact}}|}$$

$$P_{\text{lang}}(\ell) = \frac{|\{d_i \in D_{\text{train}} \mid \ell_i = \ell\}|}{|D_{\text{train}}|}$$

$$P_{\text{domain}}(k) = \frac{|\{d_i \in D_{\text{train}} \mid k_i = k\}|}{|D_{\text{train}}|}$$

R_{dup} 看重重复率， P_{lang} 看语言分布， P_{domain} 看领域分布。训练 loss 或 benchmark 变化时，这些统计能帮助你判断是模型问题、优化问题，还是数据分布变了。

7.2 为什么要版本管理

如果没有版本号，你很难回答：

1. 这次训练和上次差在哪里。
2. 哪批数据导致指标上升或下降。
3. 某个 bug 是不是由数据变更引起的。

数据版本管理的作用，和代码版本管理一样，都是为了可复现。

面试表达：优秀的数据工程不仅有过滤规则，还要有数据版本、统计报告和可回滚能力。

一个简化的数据门禁可以写成：

```
G_{\mathrm{data}}=
g_{\mathrm{schema}}\backslash,
g_{\mathrm{quality}}\backslash,
g_{\mathrm{dedup}}\backslash,
g_{\mathrm{contam}}\backslash,
g_{\mathrm{mix}}\backslash,
g_{\mathrm{report}}\backslash,
g_{\mathrm{version}}
```

这些 g 都是 0/1 检查项：schema 字段齐全、质量阈值通过、重复率可控、污染检测通过、配比满足目标、质量报告完整、数据版本可追踪。面试中这样回答，比只说“我会清洗数据”更有工程可信度。

8. 一个最小数据处理示例

下面这个 demo 用标准库模拟一个最小数据审计 pipeline。它演示五件事：质量打分、exact dedup、near dedup、benchmark contamination 检测、数据配比。真实系统会更复杂，但核心检查口径类似。

```
import hashlib
import re
from collections import Counter

records = [
    {
        "id": "web_001",
        "text": (
            "Transformer attention lets each token compare with previous tokens. "
            "This article explains training data quality, deduplication, and "
            "evaluation boundaries for language models."
        ),
        "source": "web",
        "language": "en",
        "domain": "general",
    },
    {
        "id": "web_001_dup",
        "text": (
            "Transformer attention lets each token compare with previous tokens. "
            "This article explains training data quality, deduplication, and "
            "evaluation boundaries for language models."
        ),
        "source": "mirror",
        "language": "en",
        "domain": "general",
    },
]
```

```

    "id": "web_002_near",
    "text": (
        "Transformer attention lets every token compare with previous tokens. "
        "This note explains training data quality, near deduplication, and "
        "evaluation boundaries for language models."
    ),
    "source": "web",
    "language": "en",
    "domain": "general",
},
{
    "id": "code_001",
    "text": (
        "def add(a, b):\n"
        "    return a + b\n\n"
        "Unit tests should cover edge cases and document expected behavior "
        "for simple functions."
    ),
    "source": "code_repo",
    "language": "en",
    "domain": "code",
},
{
    "id": "math_001",
    "text": (
        "A clean math solution states assumptions, derives the recurrence, "
        "checks boundary cases, and verifies the final answer with a small example."
    ),
    "source": "math_notes",
    "language": "en",
    "domain": "math",
},
{
    "id": "bench_leak",
    "text": (
        "Benchmark question: What is the capital of France? Answer: Paris. "
        "This evaluation item should never appear in training data."
    ),
    "source": "web",
    "language": "en",
    "domain": "general",
},
{
    "id": "zh_001",
    "text": " 大模型数据工程需要保留来源、语言、领域、时间戳和过滤原因，方便后续复盘训练指标。",
    "source": "internal_doc",
    "language": "zh",
    "domain": "general",
},
{
    "id": "bad_001",
    "text": "BUY NOW BUY NOW BUY NOW cheap course limited offer limited offer limited offer",
    "source": "web",
    "language": "en",
    "domain": "general",
}

```

```

    },
    {
        "id": "short_001",
        "text": "too short",
        "source": "web",
        "language": "en",
        "domain": "general",
    },
]

benchmark = [
    {"id": "bench_capital_fr", "text": "What is the capital of France? Answer Paris."},
]

allowed_languages = {"en", "zh"}
quality_threshold = 0.70
near_dedup_threshold = 0.58
contamination_threshold = 0.40
noise_terms = {"buy", "cheap", "offer"}
domain_prior = {"general": 0.85, "code": 0.95, "math": 0.95}

def normalize(text):
    text = text.lower()
    text = re.sub(r"^[a-z0-9\u4e00-\u9fff]+", " ", text)
    return re.sub(r"\s+", " ", text).strip()

def tokens(text):
    return normalize(text).split()

def shingles(text, k=2):
    ts = tokens(text)
    if len(ts) < k:
        return set(ts)
    return {tuple(ts[i : i + k]) for i in range(len(ts) - k + 1)}

def jaccard(a, b):
    if not a and not b:
        return 1.0
    if not a or not b:
        return 0.0
    return len(a & b) / len(a | b)

def quality_score(record):
    ts = tokens(record["text"])
    counts = Counter(ts)
    language_score = 1.0 if record["language"] in allowed_languages else 0.0
    structure_score = min(len(record["text"]) / 120, 1.0)
    domain_score = domain_prior.get(record["domain"], 0.70)
    unique_ratio = len(counts) / max(len(ts), 1)
    info_score = min(unique_ratio / 0.75, 1.0)

```



```

noise_score = 1.0 if any(term in counts for term in noise_terms) else 0.0
repeat_score = max(counts.values()) / max(len(ts), 1)
score = (
    0.30 * language_score
    + 0.20 * structure_score
    + 0.25 * domain_score
    + 0.20 * info_score
    - 0.35 * noise_score
    - 0.15 * repeat_score
)
return round(max(0.0, min(score, 1.0)), 4)

for record in records:
    record["quality"] = quality_score(record)
    record["hash"] = hashlib.sha1(normalize(record["text"]).encode()).hexdigest()[:10]

quality_kept = [r for r in records if r["quality"] >= quality_threshold]
quality_drop = [r["id"] for r in records if r["quality"] < quality_threshold]

seen_hashes = set()
exact_kept = []
exact_drop = []
for record in quality_kept:
    if record["hash"] in seen_hashes:
        exact_drop.append(record["id"])
        continue
    seen_hashes.add(record["hash"])
    exact_kept.append(record)

near_kept = []
near_drop = []
for record in exact_kept:
    record_shingles = shingles(record["text"])
    best = 0.0
    best_id = None
    for kept in near_kept:
        score = jaccard(record_shingles, shingles(kept["text"]))
        if score > best:
            best = score
            best_id = kept["id"]
    if best >= near_dedup_threshold:
        near_drop.append((record["id"], best_id, round(best, 4)))
    else:
        near_kept.append(record)

bench_tokens = [(b["id"], set(tokens(b["text"]))) for b in benchmark]
contam_drop = []
final_kept = []
for record in near_kept:
    rec_tokens = set(tokens(record["text"]))
    best_id, best_sim = max(
        ((bid, jaccard(rec_tokens, btoks)) for bid, btoks in bench_tokens),
        key=lambda item: item[1],
    )

```

```

record["contam_score"] = round(best_sim, 4)
if best_sim >= contamination_threshold:
    contam_drop.append((record["id"], best_id, round(best_sim, 4)))
else:
    final_kept.append(record)

domain_counts = Counter(r["domain"] for r in final_kept)
alpha = 0.5
weights = {domain: count**alpha for domain, count in domain_counts.items()}
weight_sum = sum(weights.values())
mixture_raw = {domain: weight / weight_sum for domain, weight in sorted(weights.items())}
mixture = {domain: round(prob, 4) for domain, prob in mixture_raw.items()}

remaining_hashes = [r["hash"] for r in final_kept]
remaining_near_pairs = []
for i, left in enumerate(final_kept):
    for right in final_kept[i + 1 :]:
        sim = jaccard(shingles(left["text"]), shingles(right["text"]))
        if sim >= near_dedup_threshold:
            remaining_near_pairs.append((left["id"], right["id"], round(sim, 4)))

gates = {
    "schema": all({"id", "text", "source", "language", "domain"} <= set(r) for r in records),
    "quality": not any(r["quality"] < quality_threshold for r in final_kept),
    "dedup": len(remaining_hashes) == len(set(remaining_hashes)) and not remaining_near_pairs,
    "contamination": not any(r["contam_score"] >= contamination_threshold for r in final_kept),
    "mixture": abs(sum(mixture_raw.values()) - 1.0) < 1e-12,
}

stats = {
    "raw": len(records),
    "quality_kept": len(quality_kept),
    "exact_kept": len(exact_kept),
    "near_kept": len(near_kept),
    "final_kept": len(final_kept),
    "filter_rate": round(1 - len(final_kept) / len(records), 4),
}

print("quality_by_id=", {r["id"]: r["quality"] for r in records})
print("quality_drop=", quality_drop)
print("exact_drop=", exact_drop)
print("near_drop=", near_drop)
print("contam_drop=", contam_drop)
print("kept_ids=", [r["id"] for r in final_kept])
print("domain_counts=", dict(sorted(domain_counts.items())))
print("mixture_alpha=", alpha)
print("mixture=", mixture)
print("stats=", stats)
print("gates=", gates)
print("gate_pass=", all(gates.values()))

```

运行后应看到:

```

quality_by_id= {'web_001': 0.9057, 'web_001_dup': 0.9057, 'web_002_near': 0.906, 'code_001': 0.9225, 'math_001': 0.9232,
quality_drop= ['bad_001', 'short_001']
exact_drop= ['web_001_dup']

```

```
near_drop= [('web_002_near', 'web_001', 0.5926)]
contam_drop= [('bench_leak', 'bench_capital_fr', 0.4211)]
kept_ids= ['web_001', 'code_001', 'math_001', 'zh_001']
domain_counts= {'code': 1, 'general': 2, 'math': 1}
mixture_alpha= 0.5
mixture= {'code': 0.2929, 'general': 0.4142, 'math': 0.2929}
stats= {'raw': 9, 'quality_kept': 7, 'exact_kept': 6, 'near_kept': 5, 'final_kept': 4, 'filter_rate': 0.5556}
gates= {'schema': True, 'quality': True, 'dedup': True, 'contamination': True, 'mixture': True}
gate_pass= True
```

这个例子体现了数据处理的思路：

1. 统一入口。
2. 明确过滤规则。
3. 输出可追踪。
4. 保留每一类丢弃原因。
5. 最后用门禁判断能否进入训练。

真实系统会复杂得多，但核心逻辑类似。

9. 常见坑

9.1 只看数量，不看质量

这是最常见错误。数量大不代表有效 token 多。

9.2 过滤规则过拟合某一批数据

如果规则只针对某个来源写死，换个来源就失效。

9.3 没有污染检测

评估集被污染后，所有 benchmark 都可能失真。

9.4 没有保留元数据

没有 source、language、domain、timestamp，就很难回溯问题。

9.5 数据配比固定不变

训练不同阶段需要不同配比。预训练、继续预训练、SFT 和偏好优化的最佳数据结构通常不一样。

10. 面试官会怎么问

问法 1：为什么数据工程这么重要？

可以这样答：

因为数据决定模型学到什么。数据质量、覆盖率、重复率、污染和配比都会直接影响模型能力上限。很多训练问题看起来像模型问题，

问法 2：你怎么做数据去重？

可以这样答：

我会分两层做。第一层是 exact dedup，用 hash 去掉完全重复样本；第二层是 near dedup，用 n-gram、SimHash、MinHash 或 embedd

问法 3：怎么判断训练集污染了 benchmark?

可以这样答：

我会做多层检测：先做字符串和 n-gram 匹配，再做近似检索和 embedding 相似度检索，对高风险 benchmark 还会人工抽检。对于代

问法 4：数据配比怎么定？

可以这样答：

数据配比要从目标倒推。通用 base model 更重视广覆盖和质量平衡，代码模型要提高代码和执行反馈数据比例，企业助手要提高业务

11. 小结

本章核心结论：

1. 数据工程是训练起点，不是辅助步骤。
2. 数据来源、清洗、去重、污染检测、配比和版本管理缺一不可。
3. 高质量数据往往比单纯更多数据更重要。
4. Benchmark contamination 会严重破坏评估可信度。
5. 数据配比决定模型偏科方向。
6. 面试中要能讲清数据如何影响能力上限、泛化和评估可信度。

第三章：Tokenizer 与数据格式

本章目标

理解 tokenizer 如何影响训练效率、上下文长度、多语言能力、代码能力和特殊任务格式。

核心议题

1. BPE、SentencePiece、byte-level tokenizer。
2. 词表大小和压缩率。
3. 特殊 token：BOS、EOS、PAD、UNK、system、user、assistant、tool。
4. Chat template 和 label mask。
5. 多模态 token、图像 patch token、语音 token。

面试重点

Tokenizer 不是简单预处理，而是模型输入空间的定义。它会影响训练 token 数、推理成本、上下文利用率和跨语言表现。

为什么 tokenizer 是训练协议

大模型不能直接处理字符串。所有文本都要先经过 tokenizer，变成 token id，再进入 embedding 层。

原始文本

```
-> tokenizer  
-> token ids  
-> embedding  
-> Transformer  
-> logits over vocabulary
```

所以 tokenizer 不是一个可随便替换的小工具，而是模型输入输出协议的一部分。

如果 tokenizer 变了，token id 的含义、词表大小、特殊 token、chat template 和 embedding 矩阵都会受影响。对于已经训练好的模型，随意更换 tokenizer 通常会导致模型无法正常理解输入。

面试表达：tokenizer 定义了自然语言进入模型的离散接口，也定义了模型输出如何还原为文本。

Tokenizer 与数据格式的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 BPE 原论文、SentencePiece 论文、OpenAI tiktoken、Hugging Face Tokenizers / Transformers tokenizer 文档、Transformers chat template 文档、TRL SFTTrainer 的 assistant-only / completion-only loss 说明，以及 PyTorch CrossEntropyLoss(ignore_index) 文档。

本章聚焦 tokenizer 和训练数据格式的工程协议：分词映射、词表大小、压缩率、special token、chat template、label mask、packing、SFT / 偏好 / tool calling / 多模态格式和扩词表风险。它不展开生产级 tokenizer 训练作业、完整 BPE / Unigram LM 训练器源码、所有模型家族模板差异，也不替代框架官方接口说明。

关键公式口径

给定 tokenizer T 和文本 x ，编码结果可以写成：

$$z=T(x)=(z_1,\ldots,z_L)$$

其中每个 token id 满足：

$$z_t\in\{0,\ldots,V-1\}$$

V 是词表大小， L 是这段文本被切成的 token 数。tokenizer 效率可以用每字符 token 数衡量：

$$\rho_{\mathrm{char}}(x)=\frac{L}{|x|_{\mathrm{char}}}$$

全量训练 token 数是：

$$N_{\mathrm{tok}}=\sum_{i=1}^n|T(x_i)|$$

所以 tokenizer 会直接改变训练步数、注意力计算量、KV cache 长度和推理生成步数。面试中说 “tokenizer 影响成本”，最好能落到 L 和 N_{tok} 这两个量上。

1. Tokenizer 影响哪些训练问题

Tokenizer 至少影响五类问题。

影响维度	说明
训练 token 数	同一文本切得越碎，训练步数和计算越多
上下文利用率	8K context 是 8K token，不是 8K 字符
多语言能力	低资源语言如果切得太碎，学习效率会降低
代码能力	空格、缩进、符号和标识符切分会影响代码建模
特殊任务格式	chat、tool、多模态都依赖 special token 和模板

一个简单例子：

英文：The model is good.

中文：这个模型很好。

代码：def get_user_id(x): return x.user_id

不同 tokenizer 对这三类文本的切分效率可能差异很大。如果中文或代码被切成过多 token，同样的上下文窗口能容纳的信息就更少，训练和推理成本也更高。

2. 常见 tokenizer 方法

现代大模型主流使用子词或字节级方案，而不是传统词级分词。

2.1 BPE

BPE 的核心思想是从小单位开始，不断合并高频相邻片段。

直觉上，它会把常见词、词根、代码片段或标点组合合并成更长 token，从而减少序列长度。

第 r 轮合并时，可以把最高频相邻片段写成：

$$(a^*\backslash, b^*\backslash) = \arg\max_{\{(a,b)\}} C_r(a,b)$$

其中 $C_r(a,b)$ 是当前语料分词结果中相邻片段 (a,b) 的出现次数。合并后的新片段是：

$$u^*\backslash = a^*\backslash * b^*\backslash$$

词表更新为：

$$V_{r+1} = V_r \cup \{u^*\backslash\}$$

这个公式只表达 BPE 的核心机制：每轮把高频相邻片段合成一个更长 token。真实工程里还要处理预分词、字节级编码、特殊 token、正则规则和训练语料采样。

优点：

1. 简单有效。
2. 工程成熟。
3. 可以处理未见词，因为未见词能退化为更小片段。

缺点：

1. 合并规则来自训练语料，容易继承语料偏向。
2. 对低资源语言可能切得更碎。
3. 不保证切分符合语义边界。

2.2 SentencePiece

SentencePiece 常用于多语言模型，因为它可以直接从原始文本训练 tokenizer，不强依赖语言特定的预分词。

它常把空格也纳入模型处理，例如用特殊符号表示词边界。

优点：

1. 适合没有天然空格分词的语言。
2. 适合多语言混合语料。
3. 支持 BPE 和 Unigram LM 等算法。

面试中要注意：SentencePiece 不是单一切分算法，而是一个 tokenizer 训练和处理工具体系。

2.3 Byte-level tokenizer

Byte-level tokenizer 从字节层面保证覆盖率。任意 Unicode 字符最终都能被表示，因此几乎没有 OOV。

覆盖率可以写成：

$$\forall x, \quad T_{\{\text{byte}\}}(x) \neq \text{varnothing}$$

意思是任何输入字符串都至少能退化成字节序列再编码。这个性质对代码、脏数据、混合语言和特殊符号很重要。

优点：

1. 鲁棒性强。
2. 能处理特殊字符、emoji、代码和脏输入。
3. 不容易因为未登录字符崩掉。

代价：

1. 某些语言可能 token 数变多。
2. 部分文本的语义单位被切得很碎。

3. 上下文窗口利用率可能降低。

面试表达: byte-level 的核心价值是覆盖和鲁棒, 代价是部分文本上的 token 效率。

3. 词表大小如何权衡

词表大小不是越大越好。

假设 vocabulary size 是 V , hidden size 是 d , 那么输入 embedding 参数量大约是:

$$P_{\{\mathrm{emb}\}} = Vd$$

如果输出层没有和 embedding 共享权重, 还会多出一个同规模矩阵:

$$P_{\{\mathrm{out}\}} = Vd$$

每个位置都要预测 V 类 token, 输出 logits 的形状是:

$$\ell \in \mathbb{R}^{B \times L \times V}$$

其中 B 是 batch size, L 是序列长度。如果词表很大, embedding、输出层和 softmax 都会更重。

3.1 大词表的好处

1. 常见文本可以切得更短。
2. 多语言、代码和专业词可能有更好覆盖。
3. 推理时生成同样文本可能需要更少 steps。

3.2 大词表的代价

1. embedding 和输出层参数增加。
2. softmax 计算成本增加。
3. 低频 token 学得不充分。
4. 词表训练更依赖语料分布。

3.3 小词表的代价

小词表参数少, 但序列更长。

序列变长会带来两个问题:

1. Attention 计算和显存更高。
2. 同样上下文窗口能放下的信息更少。

所以词表大小要结合语言覆盖、训练语料、上下文长度、推理成本和模型规模综合选择。

面试表达: 词表大小是在 embedding/softmax 成本和序列长度之间做 trade-off。

4. 压缩率和 tokenizer 效率

评估 tokenizer 时, 一个重要指标是压缩率, 或者说每段文本会被切成多少 token。

例如可以比较:

每 1000 个字符对应多少 token
每 1000 个英文单词对应多少 token
每 100 行代码对应多少 token

如果某种语言被切得特别碎, 它在训练中等价于 “更贵”。

比如同样 2048 token 的上下文:

高效 tokenizer: 能放入较长文章
低效 tokenizer: 只能放入较短文章

这会影响长上下文任务、RAG、代码文件输入和多轮对话。

可以把 token 效率写成：

$$R_{\{\mathrm{tok}/\mathrm{byte}\}}(D)=\frac{\sum_{x\in D}|T(x)|}{\sum_{x\in D}|x|_{\{\mathrm{byte}\}}}$$

如果上下文窗口是 C_{tok} ，某个语料桶平均每字符 token 数是 ρ_{char} ，那么可容纳的字符数近似是：

$$C_{\{\mathrm{char}\}}\approx \frac{C_{\{\mathrm{tok}\}}}{\rho_{\{\mathrm{char}\}}}$$

这解释了为什么同样是 8K token，上下文里能放下多少中文、英文、代码或 JSON 可能差异很大。

面试中可以说：tokenizer 的效率最终会体现在有效上下文长度、训练 token 预算和推理费用上。

5. Special token

Special token 是具有控制语义的 token，不是普通字符串。

常见 special token：

token	作用
BOS	序列开始
EOS	序列结束
PAD	batch padding
UNK	未知 token
system	系统指令边界
user	用户消息边界
assistant	助手消息边界
tool	工具调用或工具结果边界
image	图像占位符
audio	音频占位符

这些 token 的处理必须在训练、微调、评估和推理中一致。

如果训练时使用一种格式，推理时使用另一种格式，模型可能无法正确理解角色、边界和停止条件。

5.1 EOS 很重要

EOS 告诉模型何时结束。

如果训练数据里 EOS 处理不一致，模型可能出现：

1. 生成停不下来。
2. 过早停止。
3. 多轮对话边界混乱。

5.2 PAD 不能参与 loss

batch 训练时常需要 padding 到相同长度。

PAD token 只是占位，不应该让模型学习预测 PAD。

所以训练 loss 通常要把 PAD 位置 mask 掉。

attention mask 可以写成：

$$\mathbf{a}_t=\mathbf{1}[z_t\neq \mathrm{pad}]$$

计算 loss 时也要忽略 PAD 对应位置：


```

y_t=
\begin{cases}
z_t, & a_t=1 \\
-100, & a_t=0
\end{cases}

```

这里 -100 是很多 PyTorch / Hugging Face 训练脚本默认使用的 ignore index。核心思想不是数字本身，而是 PAD 位置不能贡献 loss。

6. Chat template

Chat template 定义多轮对话如何序列化成模型输入。

例如一段对话：

```

system: 你是一个有帮助的助手。
user: 什么是 tokenizer?
assistant: tokenizer 是把文本转成 token 的组件。

```

序列化后可能变成：

```

<bos><system>
你是一个有帮助的助手。
<user>
什么是 tokenizer?
<assistant>
tokenizer 是把文本转成 token 的组件。<eos>

```

不同模型的 chat template 可能完全不同。即使内容一样，模板不同也会影响模型行为。

如果多轮消息是：

$$M=((r_1, c_1), \dots, (r_K, c_K))$$

其中 r_k 是 role, c_k 是内容, chat template 可以看成是一个序列化函数：

$$s=T_{\{\mathrm{chat}\}}(M)$$

再经过 tokenizer 得到：

$$z=T(s)$$

因此 SFT 的真实输入不是原始 JSON，而是 $T(T_{\mathrm{chat}}(M))$ 。如果训练和推理的 T_{chat} 不一致，模型看到的角色边界、停止 token 和回答起点都会错位。

面试表达：chat template 是 chat model 的输入协议，它把角色、轮次和输出位置显式编码进 token 序列。

7. Label mask

SFT 训练中经常不是对所有 token 都算 loss。

典型做法是只让模型学习 assistant 的回答，而不学习 user prompt。

例如：

```

system: 你是助手。
user: 解释 attention。
assistant: attention 是一种信息聚合机制。

```

训练输入包含全部 token，但 loss 只算 assistant 部分：

```

system tokens    -> ignore
user tokens      -> ignore
assistant tokens -> compute loss

```

这就是 assistant-only loss mask。

7.1 为什么不对 user 部分算 loss

如果对 user prompt 也算 loss，模型会被训练去复现用户输入，而不是专注学习如何回答。

对 SFT 来说，用户输入是条件，助手回答才是监督目标。

7.2 常见错误

1. 把 system/user/assistant 全部算 loss。
2. assistant 起始 token 没处理好。
3. 多轮对话只 mask 了最后一轮。
4. padding 位置没有忽略。
5. template 和 label mask 不匹配。

这些错误会让 loss 看起来正常，但模型行为变差。

对 causal LM 来说，assistant-only SFT loss 可以写成：

$$L_{\{\mathrm{sft}\}} = \frac{\sum_{t=1}^{L-1} m_{t+1} \left[-\log p_{\theta}(z_{t+1} \mid z_{\leq t}) \right]}{\max(\sum_{t=1}^{L-1} m_{t+1}, 1)}$$

其中 $m_{t=1}$ 表示第 t 个 token 属于 assistant 监督区域， $m_{t=0}$ 表示 system、user、tool result 或 padding 等只作为条件。分母用有效监督 token 数归一化，避免不同样本回答长度差异过大时 loss 口径混乱。

面试表达：SFT 的关键不是只把对话拼起来，还要确保 label mask 和 chat template 对齐。

8. 预训练数据格式

预训练通常不是 chat 格式，而是大量连续文本。

常见格式是：

document_1 <eos> document_2 <eos> document_3 <eos>

训练目标是 next token prediction。

8.1 文档边界

文档之间通常需要边界 token，避免模型把两个无关文档当成连续上下文。

但是否插入 EOS、如何拼接、是否允许跨文档 attention，会影响训练行为。

8.2 Packing

为了提高训练效率，短文档通常会 pack 到固定长度 block 中。

例如把多个短样本拼成 2048 token 或 4096 token 的训练块。

packing 的好处是减少 padding，提高 GPU 利用率。

风险是如果边界处理不好，模型会学习到不自然的跨样本连接。

假设每个文档编码后追加 EOS：

$$s_i = T(x_i) \oplus [z_{\{\mathrm{eos}\}}]$$

把所有文档拼成总序列：

$$S = s_1 \oplus s_2 \oplus \dots \oplus s_n$$

固定 block size 为 C 时，第 j 个训练块可以写成：

$$b_j = S_{jC:(j+1)C-1}$$

如果最后一个 block 不足 C ，再用 PAD 补齐并设置 attention mask。关键是保留 EOS 或显式边界，否则模型会把两个无关文档误当作自然连续文本。

9. SFT 数据格式

SFT 数据通常是 instruction-response 或多轮 messages。

单轮格式：

```
{
  "instruction": " 解释什么是 RoPE",
  "input": "",
  "output": "RoPE 是一种旋转位置编码..."
}
```

多轮格式：

```
{
  "messages": [
    {"role": "system", "content": " 你是一个严谨的助手。"},
    {"role": "user", "content": " 解释什么是 RoPE"},
    {"role": "assistant", "content": "RoPE 是一种旋转位置编码..."}
  ]
}
```

训练前要通过 chat template 转成 token ids，并生成 label mask。

可以把一条 SFT 样本写成：

$$d_i = (M_i, z_i, m_i)$$

其中 M_i 是 messages， $z_i = T(T_chat(M_i))$ 是 token ids， m_i 是 assistant-only loss mask。真正喂给模型的是 z_i 和 m_i ，不是 JSON 文件本身。

面试中要强调：SFT 数据格式不只是 JSON 长什么样，更重要的是最后进入模型的 token 序列和 loss mask 是否正确。

10. 偏好数据格式

DPO、RLHF、GRPO 等偏好或强化学习阶段的数据格式和 SFT 不同。

最常见的偏好数据包含：

```
{
  "prompt": " 用户问题...",
  "chosen": " 更好的回答...",
  "rejected": " 更差的回答..."
}
```

关键点：

1. chosen 和 rejected 必须基于同一个 prompt。
2. 两个回答要用同一个 chat template 编码。
3. loss 通常只关注回答部分。
4. 要避免 chosen 只是更长，而不是真的更好。

如果数据格式处理错，偏好优化可能会学到错误偏好，例如偏向长回答、模板化回答或过度拒答。

偏好样本可以写成：

$d_i = (x_i, y_i^+, y_i^-)$

其中 x_i 是同一个 prompt， y_i^+ 是 chosen， y_i^- 是 rejected。两条回答必须使用同一个 tokenizer 和同一个 chat template 编码：

$z_i^+ = T(T_{\{\mathrm{chat}\}}(x_i, y_i^+))$

$z_i^- = T(T_{\{\mathrm{chat}\}}(x_i, y_i^-))$

否则比较的不是“回答质量”，而可能是模板、截断或 tokenization 差异。

11. Tool calling 数据格式

工具调用模型需要额外表达工具 schema、工具调用和工具返回结果。

一个简化格式：

```
<user>
查一下今天北京天气
<assistant_tool_call>
{"name": "weather", "arguments": {"city": "北京"}}
<tool_result>
{"temperature": "20C", "condition": "晴"}
<assistant>
今天北京天气晴，约 20C。
```

工具调用数据的难点：

1. JSON 格式必须稳定。
2. 工具参数要合法。
3. 工具结果和最终回答要对应。
4. tool token 边界要清晰。

如果格式不稳定，模型可能生成非法 JSON，或者把工具结果当成用户输入。

工具调用数据至少要保证函数名和参数 schema 可校验。简化门禁可以写成：

```
G_{\mathrm{tool}} =
g_{\mathrm{name}} \backslash,
g_{\mathrm{json}} \backslash,
g_{\mathrm{schema}} \backslash,
g_{\mathrm{result}} \backslash,
g_{\mathrm{boundary}}
```

这些检查分别表示工具名合法、JSON 可解析、参数符合 schema、工具结果和最终回答一致、tool call / tool result / assistant 边界清楚。

12. 多模态 token

多模态模型通常需要把图片、音频或视频接入语言模型。

常见做法不是直接把图片像素当成普通文本 token，而是使用占位符或经过 encoder 的视觉表示。

例如：

```
<user>
<image>
这张图里有什么？
<assistant>
图中有一只猫。
```

这里的 <image> 可能表示：

1. 一个图像占位 token。
2. 多个视觉 patch embedding 插入位置。
3. 连接到 vision encoder 输出的桥接位置。

语音模型也可能使用 audio token、codec token 或音频 encoder 输出。

面试表达：多模态 token 的关键是对齐。文本 token、图像 patch、音频片段和位置关系必须在模型输入中有一致协议。

13. Tokenizer 扩展

有时需要给已有模型增加特殊 token，例如工具 token、图像 token、领域控制 token。

这时通常需要：

1. 修改 tokenizer vocabulary。
2. resize embedding。
3. 初始化新增 token embedding。
4. 用包含新 token 的数据继续训练或微调。
5. 确保推理侧模板同步更新。

常见风险：

1. 只改 tokenizer，不改 embedding。
2. 新 token 没训练够，模型不会用。
3. 训练和推理模板不一致。
4. 新 token 和普通文本字符串混淆。

如果原词表大小是 V ，新增 K 个 special token 后：

$$V' = V + K$$

embedding 矩阵也必须从：

$$E \in \mathbb{R}^{V \times d}$$

扩展为：

$$E' \in \mathbb{R}^{V' \times d}$$

新增 token embedding 可以随机初始化，也可以用相关 token embedding 的均值初始化，但无论哪种方式，都需要后续训练让模型真正学会这些 token 的语义。

面试表达：新增 special token 是模型协议变更，不是只改配置文件。

14. 一个最小 label mask 示例

下面这个 demo 用标准库实现一个 toy tokenizer 和数据格式审计器。它不追求复刻真实 BPE，而是把最容易出错的训练协议跑一遍：压缩率统计、chat template 序列化、assistant-only label mask、预训练 packing、PAD attention mask、EOS 边界和新增 special token 后的 embedding 行数。

```
IGNORE_INDEX = -100
SPECIAL_TOKENS = ["<bos>", "<eos>", "<system>", "<user>", "<assistant>", "<pad>", "<image>"]
BASE_PIECES = [
    "tokenizer",
    "improves",
    "attention",
    "数据工程",
    "需要",
    "保留",
    "来源",
```

```

"def",
"get_user_id",
"return",
"user_id",
"解释",
"你",
"严谨",
"助手",
"是",
"文本",
"到",
"token",
"id",
"的",
"协议",
"。",
"(",
")",
":",
".",
" ",
"x",
]

vocab = {}
id_to_token = {}

def add_token(token):
    if token not in vocab:
        idx = len(vocab)
        vocab[token] = idx
        id_to_token[idx] = token
    return vocab[token]

for token in SPECIAL_TOKENS + BASE_PIECES:
    add_token(token)

match_pieces = sorted(BASE_PIECES, key=len, reverse=True)

def fallback_piece(ch):
    return f"<U+{ord(ch):04X}>"

def tokenize_text(text):
    pieces = []
    i = 0
    while i < len(text):
        matched = None
        for piece in match_pieces:
            if text.startswith(piece, i):
                matched = piece
                break

```

```

        if matched is None:
            matched = fallback_piece(text[i])
        pieces.append(matched)
        i += len(matched) if not matched.startswith("<U+") else 1
    return pieces

def encode_pieces(pieces):
    return [add_token(piece) for piece in pieces]

def decode_ids(ids):
    return [id_to_token[i] for i in ids]

def serialize_chat(messages):
    pieces = ["<bos>"]
    assistant_mask = [0]
    for msg in messages:
        role_piece = f"<{msg['role']}>"
        pieces.append(role_piece)
        assistant_mask.append(0)

        content_pieces = tokenize_text(msg["content"])
        pieces.extend(content_pieces)
        assistant_mask.extend([1 if msg["role"] == "assistant" else 0] * len(content_pieces))

        if msg["role"] == "assistant":
            pieces.append("<eos>")
            assistant_mask.append(1)

    input_ids = encode_pieces(pieces)
    labels = [
        token_id if mask else IGNORE_INDEX
        for token_id, mask in zip(input_ids, assistant_mask)
    ]
    return input_ids, labels, assistant_mask

def pack_documents(docs, block_size):
    blocks = []
    masks = []
    current = []
    for doc in docs:
        doc_ids = encode_pieces(tokenize_text(doc) + ["<eos>"])
        if len(current) + len(doc_ids) > block_size and current:
            pad_len = block_size - len(current)
            blocks.append(current + [vocab["<pad>"]] * pad_len)
            masks.append([1] * len(current) + [0] * pad_len)
            current = []
        current.extend(doc_ids)
    if current:
        pad_len = block_size - len(current)
        blocks.append(current + [vocab["<pad>"]] * pad_len)
        masks.append([1] * len(current) + [0] * pad_len)

```

```

    return blocks, masks

texts = {
    "en": "tokenizer improves attention",
    "zh": "数据工程需要保留来源",
    "code": "def get_user_id(x): return x.user_id",
}
compression = {
    name: {
        "chars": len(text),
        "tokens": len(tokenize_text(text)),
        "tokens_per_char": round(len(tokenize_text(text)) / len(text), 3),
    }
    for name, text in texts.items()
}

messages = [
    {"role": "system", "content": "你是严谨助手。"},
    {"role": "user", "content": "解释 tokenizer"},
    {"role": "assistant", "content": "tokenizer 是文本到 token id 的协议。"},
]
input_ids, labels, assistant_mask = serialize_chat(messages)
label_tokens = [id_to_token[x] if x != IGNORE_INDEX else "IGN" for x in labels]
first_assistant_pos = assistant_mask.index(1)
prompt_label_count = sum(1 for x in labels[:first_assistant_pos] if x != IGNORE_INDEX)
assistant_label_count = sum(1 for x in labels if x != IGNORE_INDEX)

blocks, attention_masks = pack_documents(list(texts.values()), block_size=16)

old_vocab_size = len(vocab)
added = 0
for token in ["<tool_call>", "<tool_result>"]:
    before = len(vocab)
    add_token(token)
    added += len(vocab) - before
new_vocab_size = len(vocab)
embedding_rows = new_vocab_size

packed_eos_count = sum(block.count(vocab["<eos>"]) for block in blocks)
gates = {
    "specials": all(token in vocab for token in SPECIAL_TOKENS),
    "assistant_only_labels": prompt_label_count == 0 and assistant_label_count > 0,
    "pad_mask": all(
        (tok != vocab["<pad>"]) == bool(mask)
        for block, mask_row in zip(blocks, attention_masks)
        for tok, mask in zip(block, mask_row)
    ),
    "packing_shape": all(len(block) == 16 for block in blocks),
    "eos_boundary": packed_eos_count == len(texts),
    "vocab_resize": new_vocab_size == old_vocab_size + added == embedding_rows,
}

print("compression=", compression)
print("chat_tokens=", decode_ids(input_ids))

```



```

print("label_tokens=", label_tokens)
print("assistant_label_count=", assistant_label_count)
print("prompt_label_count=", prompt_label_count)
print("packed_attention=", [sum(mask) for mask in attention_masks])
print("packed_pad_count=", [mask.count(0) for mask in attention_masks])
print("packed_eos_count=", packed_eos_count)
print("old_vocab_size=", old_vocab_size)
print("new_vocab_size=", new_vocab_size)
print("gates=", gates)
print("gate_pass=", all(gates.values()))

```

运行后应看到：

```

compression= {'en': {'chars': 28, 'tokens': 5, 'tokens_per_char': 0.179}, 'zh': {'chars': 10, 'tokens': 4, 'tokens_per_ch
chat_tokens= ['<bos>', '<system>', '你', '是', '严谨', '助手', '。', '<user>', '解释', ' ', 'tokenizer', '<assistant>', '
label_tokens= ['IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'IGN', 'tokenizer', ' ', '是',
assistant_label_count= 14
prompt_label_count= 0
packed_attention= [11, 14]
packed_pad_count= [5, 2]
packed_eos_count= 3
old_vocab_size= 36
new_vocab_size= 38
gates= {'specials': True, 'assistant_only_labels': True, 'pad_mask': True, 'packing_shape': True, 'eos_boundary': True,
gate_pass= True

```

这个 demo 里有几个检查点：

1. <system>、<user>、<assistant>、<eos>、<pad> 都是 special token，不是普通字符串。
2. prompt_label_count=0 说明 system 和 user 部分没有参与 loss。
3. packed_pad_count 和 pad_mask 说明 PAD 只占位，不参与 attention。
4. packed_eos_count=3 说明三个预训练文档边界都被保留。
5. new_vocab_size=38 说明新增 tool token 后 embedding 行数也要同步变大。

但核心原则不变：输入可以包含完整上下文，loss 只在需要模型学习生成的位置计算。

15. 常见坑

15.1 训练和推理 tokenizer 不一致

这是严重错误。token id 含义不同会让模型输入完全错位。

15.2 忘记 EOS

模型可能不知道何时停止，或多轮回答边界混乱。

15.3 label mask 错误

loss 可能正常下降，但模型学到复述 prompt 或生成格式异常。

15.4 截断策略错误

如果总是截断结尾，可能把答案截掉；如果总是截断开头，可能丢失系统指令。

15.5 多语言切分不均衡

某些语言 token 数过多，会导致训练成本更高、上下文更短。

15.6 特殊 token 被当普通字符串

如果 <assistant> 只是普通字符串序列，而不是 tokenizer 中的 special token，模型可能无法稳定学习角色边界。

16. 面试官会怎么问

问法 1：为什么 tokenizer 会影响训练成本？

可以这样答：

因为训练和推理都是按 token 计算的。同一段文本如果被切成更多 token，就会增加序列长度、attention 计算、KV cache 和生成步数。

问法 2：词表越大越好吗？

可以这样答：

不一定。大词表可以减少序列长度，但会增加 embedding 和输出 softmax 的参数与计算成本，也可能导致低频 token 学不好。小词表

问法 3：SFT 中为什么要做 label mask？

可以这样答：

因为用户输入和系统提示是条件，不是模型要学习生成的目标。SFT 通常只在 assistant 回答部分计算 loss。如果把用户问题也算进

问法 4：新增 special token 要注意什么？

可以这样答：

新增 special token 后要同步修改 tokenizer、resize embedding，并用包含这些 token 的数据训练新增 embedding。同时训练、评估

问法 5：多模态里的 image token 是什么？

可以这样答：

image token 通常不是图片像素本身，而是图像输入在语言模型序列中的占位或桥接位置。真实视觉信息可能来自 vision encoder 的

17. 本章小结

本章核心结论：

1. Tokenizer 是模型输入输出协议，不是简单预处理。
2. Tokenizer 会影响训练 token 数、上下文利用率、多语言能力、代码能力和推理成本。
3. BPE、SentencePiece、byte-level tokenizer 各有覆盖率、效率和工程取舍。
4. 词表大小要在序列长度和 embedding/softmax 成本之间权衡。
5. Special token 和 chat template 定义了对话模型的结构协议。
6. SFT 训练必须正确处理 label mask。
7. 预训练、SFT、偏好优化、tool calling 和多模态训练的数据格式不同。
8. 新增 special token 需要同步 tokenizer、embedding、训练数据和推理模板。
9. 面试中要把 tokenizer 讲成训练系统的一部分，而不是字符串切分工具。

第四章：模型架构与初始化

本章目标

理解训练前如何选择模型架构、规模和初始化策略。

核心议题

1. Decoder-only Transformer 的训练优势。
2. hidden size、layer 数、head 数、FFN size 的权衡。
3. RoPE、RMSNorm、SwiGLU、GQA、MoE。
4. 参数初始化和深层稳定性。
5. 从 dense model 到 MoE model 的训练差异。

面试重点

架构设计要结合训练稳定性、推理成本、数据规模和目标能力，而不是孤立比较模型结构。

为什么训练前要先定架构

训练大模型之前，必须先确定模型结构和规模。这个决策一旦开始大规模训练后就很难回头。

如果模型太小，数据再好也可能容量不够。如果模型太大，预算不足会导致训练不充分，甚至不如一个更小但训练充分的模型。如果架构不适合目标场景，后面再做 SFT、DPO 或推理优化也很难彻底弥补。

架构设计要同时考虑：

1. 目标能力。
2. 数据规模。
3. 训练预算。
4. 推理成本。
5. 分布式训练可行性。
6. 后续微调和部署生态。

面试表达：训练前的架构选择不是论文模型对比，而是目标、数据、算力和服务约束之间的系统取舍。

模型架构与初始化的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 Transformer 原论文、LLaMA 架构报告、RoPE、RMSNorm、SwiGLU、Grouped-Query Attention、Switch Transformer / MoE 和 Chinchilla compute-optimal training 相关公开资料。

本章聚焦训练前的架构决策口径：decoder-only、层数、宽度、head、FFN、RoPE、RMSNorm、SwiGLU、GQA、MoE、上下文长度、参数初始化、训练稳定性和推理成本。它不展开完整模型源码、所有架构家族差异、长上下文扩展细节、MoE 并行系统和分布式训练实现，这些会在架构专题、训练系统和推理部署章节继续展开。

架构设计的关键变量

设 hidden size 为 d ，层数为 N_L ，query head 数为 H_q ，K/V head 数为 H_{kv} ，每个 query head 的维度为 d_h ：

$$d_h = \frac{d}{H_q}$$

词表大小为 V ，SwiGLU FFN 中间维度为 d_{ff} 。embedding 参数量是：

$$P_{\{\mathrm{emb}\}} = Vd$$

GQA 下 attention projection 参数量近似是：

$$P_{\{\mathrm{attn}\}} = d^2 + 2dH_{\{\mathrm{kv}\}}d_h + d^2$$

这里四项分别对应 Q、K、V、O projection。SwiGLU FFN 参数量近似是：

$$P_{\{\mathrm{ffn}\}} \approx 3dd_{\{\mathrm{ff}\}}$$

忽略 bias 和很小的 norm 参数后，dense decoder block 参数量可以估算为：

$$P_{\{\mathrm{block}\}} \approx P_{\{\mathrm{attn}\}} + P_{\{\mathrm{ffn}\}}$$

总参数量粗略估算为：

$$P_{\{\mathrm{model}\}} \approx P_{\{\mathrm{emb}\}} + N_{LP_{\{\mathrm{block}\}}}$$

这些公式不是替代真实配置导出的参数统计，而是训练前做量级审计：模型是不是接近目标规模，GQA 是否真的降低 K/V 成本，FFN 是否成为主要参数来源。

1. 为什么 decoder-only 成为 LLM 主流

现代通用大语言模型最常见的是 decoder-only Transformer。

它的核心训练目标是自回归 next token prediction：

给定前面的 token，预测下一个 token。

1.1 训练目标统一

文本续写、问答、代码生成、对话、工具调用，都可以统一成“给定上下文生成后续 token”。

这让预训练和后训练之间的接口非常自然：

预训练：根据前文预测后文

SFT：根据指令生成回答

偏好优化：让生成回答更符合偏好

推理：根据 prompt 逐 token 生成

1.2 工程实现简单

decoder-only 架构只有 causal self-attention，不需要 encoder-decoder 之间的 cross-attention。

这带来几个工程好处：

1. 模型结构统一。
2. 训练和推理代码路径更简单。
3. KV Cache 机制清晰。
4. 适合长文本生成和多轮对话。

1.3 什么时候不是最佳选择

decoder-only 不是所有任务的唯一答案。

Encoder-only 仍适合很多理解类任务，encoder-decoder 仍适合某些翻译、摘要和条件生成任务。但通用 LLM 追求统一生成接口，decoder-only 的训练和服务闭环最简单。

面试表达：decoder-only 的优势是把预训练、指令微调、对话和推理统一到同一个自回归生成框架里。

2. 模型规模怎么定

模型规模不是只看参数量。

一个模型的主要规模参数包括：

超参	含义
num_layers	Transformer block 层数
hidden_size	hidden state 维度
num_attention_heads	attention head 数
head_dim	每个 head 的维度
ffn_size	MLP/FFN 中间层维度
vocab_size	词表大小

超参	含义
<code>max_position_embeddings</code>	最大上下文长度或位置范围

这些参数共同决定模型容量、训练成本、显存、通信和推理延迟。

2.1 层数和宽度

增加层数会增强逐层抽象能力，但也增加训练深度和稳定性挑战。

增加 hidden size 会增强每层表示容量，但会显著增加矩阵乘法成本。

粗略看，Transformer 参数主要来自 attention projection、FFN 和 embedding，其中 FFN 通常占很大比例。

面试中不需要精确背参数公式，但要知道：

hidden size 增大通常会让参数和计算快速增加。

层数增加会线性增加 block 数量和激活显存。

2.2 head 数和 head_dim

attention head 不是越多越好。

head 数增加可以让模型在不同子空间做信息路由，但如果 hidden size 固定，head 数越多，每个 head 的维度越小。

常见设计会保持合理的 head_dim，例如 64、128 等量级，同时让 hidden size 能被 head 数整除。

基本约束是：

$d = H_{qd} \cdot h$

如果要使用 GQA，还需要 query head 可以被 K/V head 分组：

$H_q = g \cdot H_{\{kv\}}$

其中 g 是每组 query head 数。若这个整除关系不成立，工程实现通常要改 head 数、hidden size 或 K/V head 数。

2.3 FFN size

FFN 通常是 Transformer block 中参数和计算的重要来源。

很多 LLM 的 FFN 中间维度约为 hidden size 的数倍。使用 SwiGLU 后，参数形式和传统 ReLU/GELU FFN 不完全一样，但核心都是用更大的中间层提供非线性变换能力。

面试表达：模型规模设计不是只报一个 7B 或 70B，而是层数、宽度、head、FFN、词表和上下文长度共同决定。

3. 参数量、token 数和算力预算要匹配

如果训练 token 不够，大模型会 under-train。

Chinchilla 风格的结论提醒我们：在固定计算预算下，参数量和训练 token 数要匹配。盲目扩大模型而不给足训练 token，可能不如训练一个更小但更充分的模型。

训练前要回答：

1. 有多少高质量 token。
2. 有多少 GPU 和训练时间。
3. 目标模型是 base、chat、代码还是领域模型。
4. 推理端能接受多大模型。

面试表达：模型大小要从 compute budget 和 token budget 倒推，而不是越大越好。

训练 FLOPs 的常用粗估仍然可以写成：

$$C_{\mathrm{train}} \approx 6P_{\mathrm{model}}D$$

其中 D 是训练 token 数。一个简单的 token / 参数比例是：

$$r_{\mathrm{tok/param}} = \frac{D}{P_{\mathrm{model}}}$$

这个比例不能机械套用到所有模型，但它能提醒你：只报 “7B” 没有意义，还要说明准备用多少高质量 token 和多少算力把它训练充分。

4. 现代 LLM 的常见架构组件

现代 decoder-only LLM 往往不再是原始 Transformer 的直接版本，而是有一组成熟改造。

4.1 RoPE

RoPE 通过旋转 Q/K 向量注入位置信息，适合 decoder-only 自回归模型。

对每个二维子空间，可以把位置 p 上的旋转写成：

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(p\theta_i) & -\sin(p\theta_i) \\ \sin(p\theta_i) & \cos(p\theta_i) \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}$$

K 向量做同样旋转。直觉上，RoPE 不是给 token 加一个位置 embedding，而是在 Q/K 的相似度计算里注入相对位置信息。

训练视角下要注意：

1. RoPE 参数要和最大上下文长度匹配。
2. 长上下文扩展不能只改 config。
3. RoPE scaling 可能需要继续训练或专项评估。

如果 RoPE base、scaling 策略或实现不一致，模型输出可能明显退化。

4.2 RMSNorm 和 Pre-Norm

现代 LLM 常用 Pre-Norm 结构，也就是在 attention/FFN 之前做 normalization。

Pre-Norm 相比 Post-Norm 通常更利于深层 Transformer 训练稳定。

RMSNorm 是 LayerNorm 的简化形式，常见于 LLaMA 风格模型。它只使用均方根进行归一化，不显式减均值，计算更简单。

RMSNorm 可以写成：

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$
$$y_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}$$

训练视角下，normalization 的作用是稳定激活尺度和梯度流。

4.3 SwiGLU

SwiGLU 是 gated MLP 的一种形式，很多现代 LLM 用它替代传统 ReLU/GELU FFN。

直觉上，它通过门控机制让模型对不同特征通道做更灵活的选择。

一个常见写法是：

$$\mathrm{SwiGLU}(x) = \left[\mathrm{swish}(xW_g) \odot (xW_u) \right] W_d$$

其中 W_g 是 gate projection, W_u 是 up projection, W_d 是 down projection, $\odot$ 表示逐元素相乘。

训练视角下，SwiGLU 往往带来更好的表示能力，但也要考虑 FFN 参数量和计算成本。

4.4 MQA 和 GQA

MQA 和 GQA 主要是为了降低推理时 KV Cache 成本。

标准 MHA 中，每个 attention head 都有自己的 K/V。MQA 多个 query head 共享一组 K/V，GQA 则是若干 query head 共享一组 K/V。

每层每 token 的 KV cache 大小近似为：

$$M_{\{\mathrm{KV}, \mathrm{layer}\}} = 2H_{\{\mathrm{kv}\}}d_{\mathrm{hb}}$$

其中 b 是每个数值的字节数，例如 fp16 / bf16 通常是 2。整模型 batch KV cache 近似是：

$$M_{\{\mathrm{KV}\}} = B_{\{\mathrm{batch}\}}C_{\{\mathrm{ctx}\}}N_{\mathrm{LM}}_{\{\mathrm{KV}, \mathrm{layer}\}}$$

GQA 的核心收益来自 $H_{\mathrm{kv}} < H_{\mathrm{q}}$ 。如果从 MHA 改为 GQA，K/V cache 近似按下面比例下降：

$$\frac{M_{\{\mathrm{GQA}\}}}{M_{\{\mathrm{MHA}\}}} = \frac{H_{\{\mathrm{kv}\}}}{H_{\mathrm{q}}}$$

训练视角下：

1. GQA 可以在效果和推理成本之间折中。
2. 它会影响 KV Cache 显存和服务吞吐。
3. 如果目标是线上推理，架构设计阶段就要考虑它。

面试表达：GQA 不是只服务推理框架的细节，而是训练前就要确定的架构选择。

5. 上下文长度如何设计

上下文长度会影响训练成本、显存和模型能力。

长上下文并不是把 `max_position` 改大就行。

训练长上下文要考虑：

1. 训练数据中是否真的有长样本。
2. 位置编码能否支持长位置。
3. attention 显存和计算是否可承受。
4. packing 和 truncation 策略是否合理。
5. 评估是否覆盖长文检索和长文推理。

常见错误是训练几乎都用短样本，却声称模型支持很长上下文。这种模型可能接收长输入，但不能可靠利用远端信息。

如果上下文长度是 C_{ctx} ，标准全量 attention 的注意力矩阵规模随下面量级增长：

$$M_{\{\mathrm{attn}\}} \propto B_{\{\mathrm{batch}\}}H_{\mathrm{q}}C_{\{\mathrm{ctx}\}}^2$$

decode 阶段 KV cache 则随上下文长度线性增长：

$$M_{\{\mathrm{KV}\}} \propto B_{\{\mathrm{batch}\}}C_{\{\mathrm{ctx}\}}N_{\mathrm{LH}}_{\{\mathrm{kv}\}}d_{\mathrm{h}}$$

所以长上下文训练和长上下文推理是两个不同成本问题：训练主要受 attention 二次复杂度影响，推理主要受 KV cache 显存和带宽影响。

面试表达：上下文长度是数据分布、位置编码、attention 成本和评估共同决定的能力，不是单个配置项。

6. Dense model 和 MoE model

Dense model 每个 token 都经过同一组 FFN 参数。

MoE model 则让每个 token 路由到部分 expert，从而增加总参数量，但每个 token 激活的参数数量较少。

如果共有 E 个 expert，每个 token 选择 k 个 expert，每个 expert 的 FFN 参数量是 P_{expert} ，那么 MoE FFN 总参数量是：

$$P_{\{\mathrm{moe}, \mathrm{total}\}} = E P_{\{\mathrm{expert}\}}$$

每个 token 激活的 FFN 参数量近似是：

$$P_{\{\mathrm{moe}, \mathrm{active}\}} = k P_{\{\mathrm{expert}\}}$$

这就是 MoE 的基本取舍：总容量大，但每个 token 的激活计算只走一部分 expert。

6.1 MoE 的优势

1. 在相近激活计算下增加模型容量。
2. 对多领域、多语言、多能力可能有帮助。
3. 推理时每个 token 只激活部分 expert。

6.2 MoE 的训练难点

1. 路由器训练不稳定。
2. expert load balance 难。
3. 通信复杂度更高。
4. batch 和并行策略更复杂。
5. 某些 expert 可能训练不足。

6.3 面试表达

MoE 的核心不是“参数更多所以更强”，而是在计算预算一定时通过稀疏激活增加容量。代价是训练、路由、负载均衡和服务系统更复杂。

7. 初始化为什么重要

大模型训练初期非常脆弱。初始化如果不合适，可能导致激活尺度异常、梯度不稳定、loss spike 或收敛很慢。

初始化要解决的问题是：

让每层输入输出的尺度在训练开始时处于合理范围。

7.1 常见初始化直觉

线性层通常使用与 fan-in / fan-out 相关的初始化，让输出方差不过大也不过小。

一个常见的 Xavier 风格标准差是：

$$\sigma_{\{\mathrm{xavier}\}} = \sqrt{\frac{2}{f_{\{\mathrm{in}\}} + f_{\{\mathrm{out}\}}}}$$

对于输入输出维度都等于 d 的 projection，可以近似为：

$$\sigma \approx \frac{1}{\sqrt{d}}$$

Embedding、attention projection、FFN projection、输出层都需要合理初始化。

对于深层网络，残差分支的尺度也很重要。如果每层残差都不断放大，深层模型容易不稳定。

一种常见直觉是让残差 projection 的初始化随层数缩小：

$$\sigma_{\mathrm{res}} \approx \frac{\sigma}{\sqrt{2N_L}}$$

不同模型家族具体做法不同，但面试里要能说明：深层残差路径的尺度必须受控，初始化不是随便填随机数。

7.2 新增 token 的初始化

如果扩展 tokenizer，新增 token embedding 也要初始化。

常见做法包括：

1. 随机初始化。
2. 用已有 token 的均值初始化。
3. 用语义相关 token 的 embedding 平均初始化。

无论哪种方式，新增 token 都需要通过继续训练或微调学会稳定语义。

7.3 从已有 checkpoint 继续训练

如果不是从零训练，而是继续预训练或领域继续训练，初始化来自已有 checkpoint。

这时要特别注意：

1. tokenizer 是否一致。
2. 架构 config 是否一致。
3. RoPE 参数是否一致。
4. vocab size 是否一致。
5. optimizer state 是否恢复。

这些不一致会导致加载失败、训练异常或效果退化。

8. 训练稳定性相关架构选择

架构设计和训练稳定性直接相关。

8.1 Pre-Norm 更稳

深层 Transformer 中，Pre-Norm 通常比 Post-Norm 更容易训练。

原因是梯度更容易沿残差路径传播。

8.2 残差尺度要受控

层数越深，残差累积越明显。很多架构会通过初始化、norm 或 residual scaling 控制激活尺度。

8.3 Attention logits 尺度

$QK^T / \sqrt{d_{\text{head}}}$ 的缩放是为了防止 softmax 过于尖锐。

标准 scaled dot-product attention logits 可以写成：

$$A = \frac{QK^T}{\sqrt{d_h}}$$

如果 attention logits 尺度异常，训练可能出现梯度不稳定。

8.4 激活函数和数值精度

SwiGLU、GELU 等激活会影响激活分布。混合精度训练中，某些激活和归一化实现也会影响 NaN 风险。

面试表达：训练稳定性不是只靠调学习率，架构中的 norm、residual、初始化和 attention 尺度都会影响稳定性。

9. 架构选择和推理成本

训练时做的架构选择会影响未来服务成本。

几个典型例子：

架构选择	对推理的影响
hidden size 更大	每 token 计算更重
layer 更多	解码每步延迟更高
vocab 更大	logits 和采样成本更高
context 更长	KV Cache 更大
MHA vs GQA	KV Cache 成本差异明显
dense vs MoE	路由和 expert 并行更复杂

所以训练前要和部署目标联动。

如果目标是端侧模型，参数量、KV Cache、量化友好性和延迟很关键。如果目标是服务端高吞吐模型，GQA、batching、并行方式和显存占用更关键。如果目标是研究模型，可以更重视能力上限。

10. 一个训练前架构决策 checklist

训练前可以按下面顺序检查：

- 1. 模型目标：通用、代码、数学、领域、聊天还是多模态？
- 2. 训练 token：有多少高质量 token？
- 3. 预算：GPU 数量、训练时长、存储和失败容忍度？
- 4. 参数规模：模型大小是否和 token/compute 匹配？
- 5. 架构组件：RoPE、RMSNorm、SwiGLU、GQA、MoE 是否必要？
- 6. 上下文长度：训练数据和位置编码是否支持？
- 7. 推理约束：延迟、吞吐、显存、KV Cache 是否可接受？
- 8. 初始化：从零训练还是加载已有 checkpoint？
- 9. 生态兼容：是否支持主流训练和推理框架？
- 10. 风险预案：如果不稳定，如何缩小规模复现实验？

这比单纯讨论“哪个架构更先进”更接近真实训练项目。

下面这个 demo 用标准库做一个训练前架构审计。它检查 hidden/head 整除关系、GQA 的参数和 KV cache 收益、参数量、训练 FLOPs、token/参数比例、初始化标准差、RMSNorm / RoPE 的最小数值行为，以及 MoE 总参数和激活参数差异。

```
import math

cfg = {
    "vocab_size": 32000,
    "hidden_size": 4096,
    "num_layers": 32,
    "query_heads": 32,
    "kv_heads": 8,
    "ffn_size": 11008,
    "context_length": 4096,
```

```

    "batch_size": 4,
    "bytes_per_value": 2,
    "train_tokens": 120_000_000_000,
    "moe_experts": 8,
    "moe_top_k": 2,
}

def dense_arch_stats(c):
    d = c["hidden_size"]
    layers = c["num_layers"]
    hq = c["query_heads"]
    hkv = c["kv_heads"]
    dh = d // hq
    f = c["ffn_size"]
    vocab = c["vocab_size"]

    q_params = d * d
    k_params = d * hkv * dh
    v_params = d * hkv * dh
    o_params = d * d
    attn_params = q_params + k_params + v_params + o_params
    mha_attn_params = 4 * d * d
    ffn_params = 3 * d * f
    norm_params = 2 * d
    block_params = attn_params + ffn_params + norm_params
    embedding_params = vocab * d
    total_params = embedding_params + layers * block_params

    kv_bytes_per_token_layer = 2 * hkv * dh * c["bytes_per_value"]
    kv_cache_gib = (
        kv_bytes_per_token_layer
        * layers
        * c["context_length"]
        * c["batch_size"]
        / (1024**3)
    )
    mha_kv_cache_gib = kv_cache_gib * (hq / hkv)
    train_flops = 6 * total_params * c["train_tokens"]

    return {
        "head_dim": dh,
        "attn_params": attn_params,
        "mha_attn_params": mha_attn_params,
        "gqa_attn_ratio": attn_params / mha_attn_params,
        "ffn_params": ffn_params,
        "block_params": block_params,
        "embedding_params": embedding_params,
        "total_params": total_params,
        "kv_cache_gib": kv_cache_gib,
        "mha_kv_cache_gib": mha_kv_cache_gib,
        "train_flops": train_flops,
    }

```

```

def rms_norm(values, eps=1e-6):
    rms = math.sqrt(sum(x * x for x in values) / len(values) + eps)
    return [x / rms for x in values]

def rope_pair(x0, x1, position, theta):
    angle = position * theta
    c = math.cos(angle)
    s = math.sin(angle)
    return [c * x0 - s * x1, s * x0 + c * x1]

def init_stds(c):
    d = c["hidden_size"]
    base = math.sqrt(2.0 / (d + d))
    residual = base / math.sqrt(2 * c["num_layers"])
    return base, residual

stats = dense_arch_stats(cfg)
base_std, residual_std = init_stds(cfg)
moe_ffn_total = cfg["moe_experts"] * 3 * cfg["hidden_size"] * cfg["ffn_size"]
moe_ffn_active = cfg["moe_top_k"] * 3 * cfg["hidden_size"] * cfg["ffn_size"]
tokens_per_param = cfg["train_tokens"] / stats["total_params"]

checks = {
    "hidden_divisible_by_heads": cfg["hidden_size"] % cfg["query_heads"] == 0,
    "query_heads_divisible_by_kv": cfg["query_heads"] % cfg["kv_heads"] == 0,
    "head_dim_reasonable": stats["head_dim"] in {64, 128},
    "ffn_large_enough": cfg["ffn_size"] >= 2 * cfg["hidden_size"],
    "kv_cache_under_16gib": stats["kv_cache_gib"] < 16,
    "tokens_per_param_reasonable": 10 <= tokens_per_param <= 40,
    "residual_std_smaller": residual_std < base_std,
    "moe_topk_valid": 1 <= cfg["moe_top_k"] <= cfg["moe_experts"],
}

print("head_dim=", stats["head_dim"])
print("attn_params_m=", round(stats["attn_params"] / 1e6, 2))
print("ffn_params_m=", round(stats["ffn_params"] / 1e6, 2))
print("block_params_m=", round(stats["block_params"] / 1e6, 2))
print("total_params_b=", round(stats["total_params"] / 1e9, 3))
print("gqa_attn_ratio=", round(stats["gqa_attn_ratio"], 4))
print("kv_cache_gib=", round(stats["kv_cache_gib"], 3))
print("mha_kv_cache_gib=", round(stats["mha_kv_cache_gib"], 3))
print("tokens_per_param=", round(tokens_per_param, 2))
print("train_flops=", f"{stats['train_flops']:.2e}")
print("base_init_std=", round(base_std, 6))
print("residual_proj_std=", round(residual_std, 6))
print("rms_norm=", [round(x, 4) for x in rms_norm([1.0, 2.0, 3.0, 4.0])])
print("rope_pair=", [round(x, 4) for x in rope_pair(1.0, 0.0, position=4, theta=0.25)])
print("moe_ffn_total_m=", round(moe_ffn_total / 1e6, 2))
print("moe_ffn_active_m=", round(moe_ffn_active / 1e6, 2))
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

运行后应看到：

```
head_dim= 128
attn_params_m= 41.94
ffn_params_m= 135.27
block_params_m= 177.22
total_params_b= 5.802
gqa_attn_ratio= 0.625
kv_cache_gib= 2.0
mha_kv_cache_gib= 8.0
tokens_per_param= 20.68
train_flops= 4.18e+21
base_init_std= 0.015625
residual_proj_std= 0.001953
rms_norm= [0.3651, 0.7303, 1.0954, 1.4606]
rope_pair= [0.5403, 0.8415]
moe_ffn_total_m= 1082.13
moe_ffn_active_m= 270.53
checks= {'hidden_divisible_by_heads': True, 'query_heads_divisible_by_kv': True, 'head_dim_reasonable': True, 'ffn_large_gate_pass= True
```

这个 demo 的重点不是精确复现某个 7B 模型，而是让你训练前能把 “参数量、KV cache、token 预算、初始化和 MoE 激活参数” 放到同一张审计表里。

11. 常见坑

11.1 参数越大越好

不对。参数量要和 token 数、算力、目标和推理成本匹配。

11.2 只改上下文长度配置

只改配置不等于模型真正具备长上下文能力。

11.3 忽略推理成本

训练出一个很大的模型，如果线上不可服务，工程价值会大幅下降。

11.4 MoE 只看总参数量

MoE 要看激活参数、路由、负载均衡、通信和服务复杂度。

11.5 初始化和 checkpoint 不匹配

继续训练时，如果 tokenizer、RoPE、vocab 或 config 不一致，可能直接失败或暗中退化。

12. 面试官会怎么问

问法 1：如果让你设计一个 7B 模型，你会考虑什么？

可以这样答：

我会先从目标和预算倒推，而不是直接固定 7B。需要确定目标能力、训练 token 数、GPU 预算、上下文长度和推理约束。架构上通常 only Transformer，再确定层数、hidden size、head 数、FFN size、RoPE、RMSNorm、SwiGLU 和是否使用 GQA。最后还要看训练稳定性。

问法 2：为什么现代 LLM 多用 decoder-only？

可以这样答：

为什么预训练目标看起来简单但很关键

GPT 类大语言模型的预训练目标非常简单：根据前面的 token 预测下一个 token。

input: $x_1, x_2, \dots, x_{t-1}$
target: x_t

但这个简单目标一旦放到海量数据和大模型容量上，就会让模型学习语言结构、事实知识、代码模式、数学模式、推理模板和世界常识。

预训练阶段得到的是 base model。它通常擅长续写，但不一定会稳定遵循指令。后续 SFT、DPO、RLHF 或 reasoning RL，都是在这个底座上继续塑造行为。

面试表达：预训练不是让模型“背数据”，而是通过大规模 next-token prediction 学到可泛化的语言 and 知识表示。

预训练目标与训练循环的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 Hugging Face causal language modeling 文档、Transformers Trainer 的 gradient accumulation 说明、Hugging Face Accelerate 梯度累积说明、PyTorch CrossEntropyLoss(ignore_index) 文档，以及 PyTorch AMP / GradScaler 文档。

本章聚焦 decoder-only causal LM 的训练闭环：label shift、cross entropy、causal mask、padding mask、packing、gradient accumulation、mixed precision、validation、日志和 checkpoint。它不展开分布式并行、ZeRO/FSDP、FlashAttention、optimizer 内部状态、复杂数据加载和完整训练平台实现；这些会在后续分布式训练、显存优化和实验管理章节继续展开。

1. 自回归语言模型的概率分解

给定一个 token 序列：

$x_1, x_2, \dots, x_T$

自回归语言模型把联合概率分解为：

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{1:t-1})$$

也就是说，每个位置只根据前面的 token 预测当前 token。

在 causal LM 中，模型不能看到未来 token，所以需要 causal mask。

位置 t 只能看 $\leq t$ 的上下文

不能看 $t+1, t+2, \dots$

这个约束保证训练目标和推理过程一致：推理时模型也是从左到右逐 token 生成。

2. 输入和标签如何错位

语言模型训练里，输入 x 和标签 y 通常是同一段 token 序列错开一位。

例如 token 序列是：

[10, 20, 30, 40, 50]

如果 block size 是 4：

$x = [10, 20, 30, 40]$

$y = [20, 30, 40, 50]$

模型在每个位置预测下一个 token：

看到 10 -> 预测 20
看到 10,20 -> 预测 30
看到 10,20,30 -> 预测 40

实现时常见 shape:

input_ids: [B, T]
labels: [B, T]
logits: [B, T, V]

其中:

1. B 是 batch size。
2. T 是 sequence length。
3. V 是 vocabulary size。

如果原始训练块是:

$z=(z_1, \dots, z_T)$

那么输入和标签通常是:

$x=(z_1, \dots, z_{T-1})$

$y=(z_2, \dots, z_T)$

模型输出 logits:

$\ell \in \mathbb{R}^{B \times (T-1) \times V}$

这里最容易错的是 off-by-one: logits 的第 t 个位置预测的是 z_{t+1} , 不是 z_t 本身。

3. Cross entropy loss 与最大似然

模型输出 logits 后, 会对词表做 softmax, 得到每个 token 的概率分布。

训练目标是让真实下一个 token 的概率尽可能高。

对单个位置来说, cross entropy loss 可以写成:

$$\ell_{b,t} = -\log p_{\theta}(y_{b,t} \mid x_{b, \leq t})$$

用 logits 展开就是:

$$\ell_{b,t} = -a_{b,t,y_{b,t}} + \log \sum_{v=1}^V \exp(a_{b,t,v})$$

其中 a 是模型输出 logits, v 遍历词表。实际实现通常使用数值稳定的 `cross_entropy`, 不要先手写 softmax 再取 log。

整段序列的 loss 通常是所有有效位置的平均:

$$L = \frac{\sum_{b,t} m_{b,t} \ell_{b,t}}{\max(\sum_{b,t} m_{b,t}, 1)}$$

其中 $m_{b,t}=1$ 表示该位置参与 loss, $m_{b,t}=0$ 表示 padding 或其他忽略位置。PyTorch / Hugging Face 中常见做法是把忽略位置的 label 设为 -100。

从概率角度看, 最小化 cross entropy 等价于最大化训练数据的似然。

面试表达: next-token prediction 的训练本质是最大似然估计, cross entropy 是对应的负对数似然损失。

4. Attention mask 和 causal mask

训练 decoder-only 模型时必须使用 causal mask。

一个长度为 4 的序列，允许看的位置可以表示为：

```
1 0 0 0
1 1 0 0
1 1 1 0
1 1 1 1
```

第 3 个位置可以看前 3 个 token，但不能看第 4 个 token。

如果没有 causal mask，模型训练时会偷看答案，loss 会异常好看，但推理时完全不成立。

causal mask 可以写成：

$$C_{\{i,j\}} = \mathbf{1}[j \leq i]$$

scaled dot-product attention 加 mask 后可以理解为：

$$A = \frac{\text{softmax}\left(\frac{QK^{\text{top}}}{\sqrt{d_h}} + M\right)}{\text{softmax}}$$

其中未来位置的 $M_{i,j}$ 取一个很大的负数，使 softmax 后概率接近 0。

Padding mask

如果 batch 中有 padding，还需要 padding mask，避免模型关注 PAD token，也避免 PAD 位置参与 loss。

预训练常用 packing 减少 padding，但 SFT 或变长样本中 padding mask 依然很重要。

padding attention mask 可以写成：

$$a_{\{b,t\}} = \mathbf{1}[x_{\{b,t\}} \neq z_{\{\text{padding}\}}]$$

label mask 则通常写成：

$$m_{\{b,t\}} = \mathbf{1}[y_{\{b,t\}} \neq -100]$$

注意 attention mask 和 label mask 不是同一个东西：前者控制模型能不能看某个输入位置，后者控制某个 label 位置是否贡献 loss。

5. Packing 与 sequence length

预训练数据通常来自很多长短不同的文档。如果每篇文档单独 padding 到固定长度，会浪费大量计算。

Packing 的做法是把多个文档拼接成固定长度 block。

doc1 <eos> doc2 <eos> doc3 <eos> ... -> chunks of length T

这样可以减少 padding，提高 GPU 利用率。

Packing 的好处

1. batch shape 稳定。
2. 减少 padding 浪费。
3. 提高 tokens/sec。
4. 更适合大规模流式数据加载。

Packing 的风险

1. 文档边界处理不当。
2. EOS 插入不一致。
3. 不相关文档被错误看成连续上下文。
4. 短文档过多导致上下文结构不自然。

对普通预训练，跨文档拼接通常可以接受，但需要 EOS 或边界设计让模型知道文档结束。

设每个文档编码后追加 EOS：

$$s_i = T(d_i) \oplus [z_{\{\mathrm{eos}\}}]$$

拼成总 token 流：

$$S = s_1 \oplus s_2 \oplus \cdots \oplus s_n$$

固定 sequence length 为 T_{seq} 时，第 k 个 block 是：

$$B_k = S_{\{kT_{\{\mathrm{seq}\}}:(k+1)T_{\{\mathrm{seq}\}}-1\}}$$

如果最后一个 block 不足长度，再用 PAD 补齐，并把对应 labels 设为 ignore index。

6. 一个最小训练 batch

一个 batch 至少包含：

input_ids
labels
attention_mask

例如：

```
batch = {  
    "input_ids": input_ids,      # [B, T]  
    "labels": labels,           # [B, T]  
    "attention_mask": attention_mask, # [B, T]  
}
```

对于纯 causal LM，有些框架会在模型内部自动构造 causal mask，但 padding mask 和 label ignore 仍要正确处理。

常见 ignore index 是：

IGNORE_INDEX = -100

PyTorch 的 cross entropy 默认会忽略 label 为 -100 的位置。

7. 标准训练循环

一个简化训练循环如下：

```
for step, batch in enumerate(train_loader):  
    input_ids = batch["input_ids"].to(device)  
    labels = batch["labels"].to(device)  
    attention_mask = batch["attention_mask"].to(device)  
  
    outputs = model(  
        input_ids=input_ids,  
        attention_mask=attention_mask,  
        labels=labels,  
    )  
    loss = outputs.loss
```

```

loss.backward()
optimizer.step()
scheduler.step()
optimizer.zero_grad()

```

真实大模型训练会更复杂，还包括：

1. mixed precision。
2. gradient accumulation。
3. gradient clipping。
4. distributed data parallel。
5. checkpoint 保存。
6. 日志记录。
7. validation。
8. 异常检测。

但核心顺序不变：forward、loss、backward、optimizer step。

8. Gradient accumulation

大模型训练常常希望使用很大的 global batch size，但单张 GPU 显存放不下。

Gradient accumulation 的做法是把一个大 batch 拆成多个 micro-batch，累积梯度后再更新一次参数。

$\text{global_batch} = \text{micro_batch} * \text{accumulation_steps} * \text{data_parallel_size}$

如果 micro batch token 数是 B_{micro} ，序列长度是 T_{seq} ，累积步数是 A ，数据并行规模是 P_{dp} ，那么每次 optimizer step 对应的 token 数近似是：

$B_{\text{tok}} = B_{\text{micro}} T_{\text{seq}} A P_{\text{dp}}$

对等大小 micro-batch，梯度累积的平均梯度可以写成：

$$g = \frac{1}{\sum_{a=1}^A \|\nabla_{\theta} L_a\|}$$

如果每个 micro-batch 有效 token 数不同，更稳妥的口径是按有效 token 总数归一化：

$$g = \frac{\sum_{a=1}^A \sum_{t \in M_a} \|\nabla_{\theta} \ell_t\|}{\sum_{a=1}^A |M_a|}$$

其中 M_a 是第 a 个 micro-batch 中参与 loss 的 token 集合。

伪代码：

```
optimizer.zero_grad()
```

```

for micro_step, batch in enumerate(loader):
    loss = model(**batch).loss
    loss = loss / accumulation_steps
    loss.backward()

```

```

if (micro_step + 1) % accumulation_steps == 0:
    optimizer.step()
    scheduler.step()
    optimizer.zero_grad()

```

注意 loss 要除以 accumulation_steps，否则等价于把梯度放大了。

面试表达：gradient accumulation 用时间换显存，让多个 micro-batch 的梯度累积成一个更大的 effective batch。

9. Mixed precision 与 loss scaling

大模型训练通常使用 FP16、BF16 或混合精度来提升吞吐和降低显存。

BF16 动态范围更大，训练大模型时通常比 FP16 更稳。

FP16 训练中可能需要 loss scaling，避免梯度 underflow。

loss scaling 的核心可以写成：

$\tilde{L} = sL$

先对放大后的 loss 反向传播，再把梯度除回去：

$g =$
 $\frac{\partial \tilde{L}}{\partial \theta}$

如果检测到 Inf 或 NaN，动态 loss scaling 会跳过本次 optimizer step 并降低 scale。

常见数值风险：

1. loss 变成 NaN。
2. gradient norm 爆炸。
3. logits 过大。
4. optimizer state 出现 Inf。

这些问题不一定是模型结构错，也可能来自学习率、数据异常、精度设置或实现 bug。

10. Validation loop

训练过程中必须定期做 validation。

Validation loop 和 training loop 类似，但不反向传播，也不更新参数。

```

model.eval()
val_losses = []

with torch.no_grad():
    for batch in val_loader:
        outputs = model(**batch)
        val_losses.append(outputs.loss.item())

```

```

model.train()
val_loss = sum(val_losses) / len(val_losses)

```

Validation 的作用：

1. 判断模型是否泛化。
2. 发现过拟合或数据问题。
3. 辅助 checkpoint 选择。
4. 观察不同 domain 的 loss。

不要只看 train loss。train loss 下降不代表模型能力一定提升，尤其当数据重复、污染或评估维度不足时。
validation loss 可以按有效 token 平均：

$$L_{\mathrm{val}} = \frac{\sum_{b,t} m_{b,t} \ell_{b,t}}{\sum_{b,t} m_{b,t}}$$

困惑度是：

$$\mathrm{PPL} = \exp(L_{\mathrm{val}})$$

PPL 对 base model 有参考价值，但不能替代下游能力评估、安全评估和污染检查。

11. 日志应该记录什么

预训练日志至少要记录四类指标。

类别	指标
优化指标	train loss、val loss、learning rate、grad norm
系统指标	tokens/sec、GPU utilization、MFU、dataloader time
数值指标	NaN、Inf、loss spike、gradient clipping ratio
训练状态	step、epoch/token count、checkpoint、数据版本

如果只记录 loss，出了问题很难定位。

例如：

1. loss 不变，可能是学习率太小，也可能是数据重复或 label 错位。
2. tokens/sec 下降，可能是数据加载慢、通信慢或 checkpoint 卡住。
3. val loss 上升，可能是过拟合，也可能是训练数据和验证数据分布差异。

面试表达：训练日志要同时覆盖优化健康、系统健康、数值健康和数据健康。

12. Checkpoint 保存与恢复

大模型训练很贵，必须定期保存 checkpoint。

一个完整 checkpoint 通常包括：

1. model weights。
2. optimizer state。
3. scheduler state。
4. random seed / RNG state。
5. 当前 step。
6. tokenizer 和 config。
7. 数据迭代位置或数据版本。

只保存模型权重可以用于推理，但不一定能无缝恢复训练。

恢复训练时，如果 optimizer state、learning rate schedule 或数据位置丢失，训练曲线可能发生跳变。

一个训练 checkpoint 可以抽象为：

$$C_s = (\theta_s, o_s, r_s, u_s, d_s, c_s)$$

其中 θ_s 是模型权重， o_s 是 optimizer state， r_s 是随机数状态， u_s 是 scheduler state， d_s 是数据迭代位置或数据版本， c_s 是模型 config 和 tokenizer。缺任意关键项，都可能无法严格复现继续训练。

13. Checkpoint 选择不是只看最后一步

训练最后一步不一定最好。

选择 checkpoint 要看：

1. validation loss。
2. per-domain loss。
3. 下游 benchmark。
4. 安全和毒性评估。
5. 训练稳定性。
6. 推理成本。

对于 base model，可以重点看 validation loss 和基础能力评估。对于后训练模型，还要看指令遵循、人类偏好、安全和回归测试。

一个简化的 checkpoint 选择规则可以写成：

$$s^{\lambda} = \arg\min_s \left[L_{\text{val}}(s) + \lambda R_{\text{risk}}(s) + \mu C_{\text{serve}}(s) \right]$$

其中 R_{risk} 表示安全、污染或回归风险， C_{serve} 表示服务成本。这个公式不是说一定要真的线性加权，而是提醒你 checkpoint 选择不是只看最后一步或单一 loss。

下面这个 demo 用标准库实现一个极小 causal LM 训练步。它不训练真实 Transformer，只用一个 token 到 logits 的表模拟 next-token prediction，目的是检查 label shift、-100 ignore、causal mask、packing、gradient accumulation 缩放和一次 SGD step 后 loss 是否下降。

```
import math

PAD = 0
EOS = 1
IGNORE_INDEX = -100
vocab = {"<pad>": PAD, "<eos>": EOS, "the": 2, "cat": 3, "sat": 4, "dog": 5, "ran": 6}
id_to_token = {idx: tok for tok, idx in vocab.items()}

docs = [
    [vocab["the"], vocab["cat"], vocab["sat"]],
    [vocab["the"], vocab["dog"], vocab["ran"]],
]
block_size = 5

def pack_documents(documents, block_size):
    stream = []
    for doc in documents:
        stream.extend(doc + [EOS])
    blocks = []
    for start in range(0, len(stream), block_size):
        block = stream[start : start + block_size]
        block += [PAD] * (block_size - len(block))
        blocks.append(block)
    return blocks
```

```

def build_batch(blocks):
    input_ids = [block[:-1] for block in blocks]
    labels = []
    attention_mask = []
    for block in blocks:
        shifted = block[1:]
        labels.append([tok if tok != PAD else IGNORE_INDEX for tok in shifted])
        attention_mask.append([1 if tok != PAD else 0 for tok in block[:-1]])
    return input_ids, labels, attention_mask

def causal_mask(seq_len):
    return [[1 if j <= i else 0 for j in range(seq_len)] for i in range(seq_len)]

def valid_pairs(input_ids, labels):
    pairs = []
    for row_x, row_y in zip(input_ids, labels):
        for x, y in zip(row_x, row_y):
            if y != IGNORE_INDEX:
                pairs.append((x, y))
    return pairs

def init_logits(vocab_size):
    return [[0.03 * (i - j) for j in range(vocab_size)] for i in range(vocab_size)]

def softmax(logits):
    m = max(logits)
    exps = [math.exp(x - m) for x in logits]
    total = sum(exps)
    return [x / total for x in exps]

def loss_and_grad(table, pairs):
    vocab_size = len(table)
    grad = [[0.0] * vocab_size for _ in range(vocab_size)]
    loss_sum = 0.0
    for prev, target in pairs:
        probs = softmax(table[prev])
        loss_sum += -math.log(probs[target])
        for j, p in enumerate(probs):
            grad[prev][j] += p
        grad[prev][target] -= 1.0
    denom = max(len(pairs), 1)
    return loss_sum / denom, [[g / denom for g in row] for row in grad]

def add_grads(left, right):
    return [
        [a + b for a, b in zip(row_a, row_b)]
        for row_a, row_b in zip(left, right)
    ]

```

```

def scale_grad(grad, scale):
    return [[g * scale for g in row] for row in grad]

def grad_norm(grad):
    return math.sqrt(sum(g * g for row in grad for g in row))

def apply_sgd(table, grad, lr):
    return [
        [w - lr * g for w, g in zip(row_w, row_g)]
        for row_w, row_g in zip(table, grad)
    ]

blocks = pack_documents(docs, block_size)
input_ids, labels, attention_mask = build_batch(blocks)
pairs = valid_pairs(input_ids, labels)
logits = init_logits(len(vocab))
full_loss, full_grad = loss_and_grad(logits, pairs)

micro_batches = [pairs[:3], pairs[3:]]
accum_grad = [[0.0] * len(vocab) for _ in range(len(vocab))]
bad_grad = [[0.0] * len(vocab) for _ in range(len(vocab))]
for micro in micro_batches:
    _, micro_grad = loss_and_grad(logits, micro)
    accum_grad = add_grads(accum_grad, scale_grad(micro_grad, 1 / len(micro_batches)))
    bad_grad = add_grads(bad_grad, micro_grad)

updated_logits = apply_sgd(logits, accum_grad, lr=0.5)
after_loss, _ = loss_and_grad(updated_logits, pairs)
max_grad_diff = max(
    abs(a - b)
    for row_a, row_b in zip(full_grad, accum_grad)
    for a, b in zip(row_a, row_b)
)

print("blocks=", [[id_to_token[t] for t in block] for block in blocks])
print("input_ids=", input_ids)
print("labels=", labels)
print("attention_mask=", attention_mask)
print("causal_mask=", causal_mask(seq_len=4))
print("valid_pairs=", [(id_to_token[x], id_to_token[y]) for x, y in pairs])
print("valid_token_count=", len(pairs))
print("initial_loss=", round(full_loss, 4))
print("after_one_step_loss=", round(after_loss, 4))
print("loss_decreased=", after_loss < full_loss)
print("max_grad_diff=", round(max_grad_diff, 10))
print("full_grad_norm=", round(grad_norm(full_grad), 6))
print("accum_grad_norm=", round(grad_norm(accum_grad), 6))
print("bad_unscaled_grad_norm=", round(grad_norm(bad_grad), 6))
print("bad_grad_ratio=", round(grad_norm(bad_grad) / grad_norm(full_grad), 2))

checks = {

```



```

    "labels_shifted": labels[0][:3] == blocks[0][1:4],
    "pad_ignored": labels[1][-1] == IGNORE_INDEX,
    "causal_lower_triangular": causal_mask(4)[2] == [1, 1, 1, 0],
    "accum_matches_full": max_grad_diff < 1e-12,
    "unscaled_accum_too_large": grad_norm(bad_grad) > grad_norm(full_grad),
    "loss_decreased": after_loss < full_loss,
}
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

运行后应看到:

```

blocks= [['the', 'cat', 'sat', '<eos>', 'the'], ['dog', 'ran', '<eos>', '<pad>', '<pad>']]
input_ids= [[2, 3, 4, 1], [5, 6, 1, 0]]
labels= [[3, 4, 1, 2], [6, 1, -100, -100]]
attention_mask= [[1, 1, 1, 1], [1, 1, 1, 0]]
causal_mask= [[1, 0, 0, 0], [1, 1, 0, 0], [1, 1, 1, 0], [1, 1, 1, 1]]
valid_pairs= [('the', 'cat'), ('cat', 'sat'), ('sat', '<eos>'), ('<eos>', 'the'), ('dog', 'ran'), ('ran', '<eos>')]
valid_token_count= 6
initial_loss= 1.9427
after_one_step_loss= 1.8718
loss_decreased= True
max_grad_diff= 0.0
full_grad_norm= 0.377787
accum_grad_norm= 0.377787
bad_unscaled_grad_norm= 0.755575
bad_grad_ratio= 2.0
checks= {'labels_shifted': True, 'pad_ignored': True, 'causal_lower_triangular': True, 'accum_matches_full': True, 'unscaled_accum_too_large': True, 'loss_decreased': True}
gate_pass= True

```

这个 demo 的关键观察是: labels 必须相对 input shift 一位, PAD label 必须变成 ignore index, causal mask 必须是下三角; 在两个等大小 micro-batch 上, 正确缩放后的 accumulation gradient 和 full batch gradient 完全一致, 不缩放时梯度范数正好放大 2 倍。

14. 常见训练 bug

14.1 输入标签没有错位

如果 x 和 y 没有 shift, 模型可能学习复制当前 token, 而不是预测下一个 token。

14.2 causal mask 错误

如果模型看到未来 token, 训练 loss 会虚假变好。

14.3 PAD 参与 loss

模型会学习预测 PAD, 浪费容量并影响 loss。

14.4 accumulation loss 没有缩放

梯度会被放大, 等价于学习率异常变大。

14.5 train/eval 模式没切换

dropout、norm 或其他层行为可能不一致。

14.6 数据重复或污染

train loss 很好，但真实评估不可靠。

15. 一个面试级回答模板

如果面试官问：

请讲一下大模型预训练的训练循环。

可以这样答：

预训练通常使用 next-token prediction。数据先经过 tokenizer 变成 token ids，再通过 packing 构造固定长度序列。训练时 input

训练循环上，每个 batch 先 forward 得到 loss，再 backward 计算梯度，必要时做 gradient accumulation 和 gradient clipping，

16. 本章小结

本章核心结论：

1. 预训练目标通常是 next-token prediction。
2. 自回归语言模型把联合概率分解为逐 token 条件概率。
3. Cross entropy 等价于最大似然的负对数形式。
4. 输入和标签通常是同一序列错开一位。
5. Causal mask 防止模型偷看未来 token。
6. Packing 可以提高训练效率，但要处理好文档边界和 EOS。
7. Gradient accumulation 用多个 micro-batch 模拟更大的 global batch。
8. Validation loop、日志和 checkpoint 是训练工程不可缺少的部分。
9. 训练 bug 常来自 label shift、mask、padding、accumulation 和数据问题。
10. 面试中要能从 batch 到 loss、backward、optimizer step、validation 和 checkpoint 讲完整闭环。

第六章：优化器与学习率策略

本章定位

优化器是大模型训练中最容易被低估的部分。

很多人只知道：

训练大模型一般用 AdamW。

但真实训练中，优化器选择会影响：

1. 收敛速度。
2. 训练稳定性。
3. 显存占用。
4. 通信开销。
5. batch size scaling。
6. loss spike 和 NaN 风险。
7. 最终模型泛化能力。

近几年除了 Adam、AdamW，也出现了 Lion、Adafactor、Sophia、Muon 等优化器或优化思路。尤其是 Muon 这类方法重新强调矩阵参数更新方向的几何结构，在一些大模型训练讨论中受到关注。

本章目标不是把所有优化器公式背下来，而是建立一个面试和工程都可用的优化器谱系。

优化器与学习率策略的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 Adam、AdamW、Adafactor、Lion、Sophia、Muon 相关论文或公开说明，以及 PyTorch AdamW / clip_grad_norm_、Hugging Face Transformers warmup/cosine scheduler 和主流训练 recipe 资料。

本章聚焦训练工程中最常用的优化器和学习率策略：SGD、Momentum、Adam、AdamW、Adafactor、Lion、Sophia、Muon、参数分组、warmup、cosine decay、decoupled weight decay、gradient clipping 和 batch size scaling。它不展开每个优化器的完整收敛理论，也不把相对新的优化器写成无条件替代 AdamW 的结论；工程上仍应以 AdamW baseline、消融实验和稳定性指标为准。

本章目标

学完本章，你需要掌握：

- 1. SGD、Momentum、Adam、AdamW 的关系。
- 2. Adam 和 AdamW 的核心区别。
- 3. 为什么大模型训练长期偏爱 AdamW。
- 4. Adafactor、Lion、Sophia、Muon 等方法想解决什么问题。
- 5. Muon 的基本思想和适用边界。
- 6. warmup、cosine decay、weight decay、gradient clipping 如何配合优化器。
- 7. loss spike、NaN、训练不稳定时如何从优化器和学习率角度排查。

1. 优化器在训练中解决什么问题

神经网络训练的目标是最小化 loss：

$$\min_{\theta} L(\theta)$$

最简单的更新是梯度下降：

$$\theta_{t+1} = \theta_t - \eta g_t$$

其中 η 是学习率， g_t 是当前 mini-batch 梯度：

$$g_t = \nabla_{\theta} L(\theta_t)$$

但大模型训练里，简单 SGD 往往不够。

原因是：

- 1. 参数规模巨大。
- 2. 不同参数梯度尺度差异很大。
- 3. 训练数据噪声大。
- 4. batch size 很大。
- 5. mixed precision 容易带来数值问题。
- 6. loss landscape 复杂。

优化器本质上是在回答：

拿到当前梯度后，应该朝哪个方向走，走多大一步？

2. 优化器谱系总览

优化器	核心思想	优点	主要问题	常见场景
SGD	直接沿负梯度更新	简单、泛化好	收敛慢、调参敏感	CV、小模型
Momentum SGD	累积历史梯度方向	减少震荡、加速	仍需仔细调 lr	CNN、传统
Adam	一阶动量 + 二阶自适应	收敛快、鲁棒	weight decay 耦合不理想	NLP、Tra
AdamW	Adam + decoupled weight decay	大模型默认强基线	optimizer state 显存大	LLM 预训练
Adafactor	factored second moment	显存低	调参和稳定性更复杂	T5、大模型

优化器	核心思想	优点	主要问题	常见场景
Lion	sign-based momentum update	状态少、可能更省	需要重新调 lr/wd	部分视觉和
Sophia	利用 Hessian 近似	提高样本效率	实现和超参复杂	研究和特定
Muon	对矩阵更新做正交化/几何约束	改善矩阵参数更新方向	新方法，适用边界仍需验证	新一代大模

面试中最常被问的是 AdamW，但如果能补充 Muon、Adafactor、Lion 等方法，会显得你关注当前训练进展。

3. SGD 与 Momentum

SGD 更新公式：

$$\theta_t = \theta_{t-1} - \eta g_t$$

其中 g_t 是当前 mini-batch 梯度。

SGD 的问题是梯度噪声大，容易震荡。

Momentum 引入历史方向：

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

直觉是：

如果连续很多步都朝同一个方向，就加速；如果方向来回变，就抵消震荡。

Momentum 是理解 Adam 的前置概念。

4. Adam

Adam 同时维护一阶动量和二阶动量。

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

更新时用：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

其中：

1. m_t 表示梯度方向的滑动平均。
2. v_t 表示梯度平方的滑动平均。
3. 分母让不同参数有自适应学习率。

Adam 的优点：

1. 收敛快。
2. 对学习率相对不那么敏感。
3. 适合 Transformer 这种梯度尺度差异大的模型。

Adam 的问题：

1. optimizer state 大，需要保存 m 和 v 。
2. weight decay 和梯度更新耦合，正则效果不够理想。

3. 大规模训练中仍然需要仔细调 warmup、lr 和 clipping。

5. AdamW：大模型训练默认强基线

AdamW 的核心是 decoupled weight decay。

普通 Adam 中，L2 正则项会进入梯度项。

AdamW 把 weight decay 从梯度更新里解耦出来：

$$u_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$$\begin{aligned} \theta_t &= \theta_{t-1} \\ &\quad - \eta u_t \\ &\quad - \eta \lambda \theta_{t-1} \end{aligned}$$

其中 λ 是 weight decay 系数。和把 $\lambda \theta$ 加进梯度不同，AdamW 的衰减项不经过 Adam 的二阶自适应分母。

直觉是：

Adam 负责根据梯度找方向，weight decay 单独负责让参数不要无限变大。

为什么大模型常用 AdamW？

1. Transformer 上经验稳定。
2. 收敛速度快。
3. 和 warmup + cosine decay 配合成熟。
4. 工具链支持完善。
5. 训练 recipe 和经验很多。

常见超参起点：

```
beta1 = 0.9
beta2 = 0.95 或 0.999
eps = 1e-8
weight_decay = 0.01 到 0.1
```

注意：不同模型规模、batch size、数据质量和训练阶段会改变最优超参。

6. Adafactor：省显存的 Adam 替代

AdamW 的显存开销很大。

对每个参数，要保存：

```
parameter
gradient
first moment
second moment
```

如果是 mixed precision，还可能有 master weights。

Adafactor 的思路是对二阶矩估计做 factored approximation。

对于一个矩阵参数，不完整保存每个元素的二阶矩，而是保存行统计和列统计。

如果矩阵参数 W 的形状是 $r \times c$ ，AdamW 的二阶矩需要存：

$$S_{\{\mathrm{Adam}\}} = rc$$

Adafactor 的二阶统计近似只存行和列统计：

$$S_{\{\mathrm{Adafactor}\}} \approx r+c$$

当 r 和 c 很大时，状态显存会显著下降。代价是近似带来额外 recipe 和稳定性要求。

优点：

1. 明显降低 optimizer state 显存。
2. 适合超大模型或显存紧张场景。
3. 曾在 T5 等模型训练中使用。

缺点：

1. 调参更复杂。
2. 稳定性依赖具体 recipe。
3. 不一定总能替代 AdamW。

面试表达：

Adafactor 主要解决 Adam 二阶状态显存过大的问题，通过对矩阵二阶矩做分解近似来节省显存，但它的训练稳定性和超参选择需要更

7. Lion：更少状态的符号更新

Lion 是一种使用 momentum 和 sign update 的优化器。

它不像 Adam 那样维护完整二阶矩。

直觉上：

用动量估计方向，再用 sign 决定更新方向。

一个简化写法是：

$$\begin{aligned}u_t &= \beta_{1m}[t-1] + (1 - \beta_{1m})g_t \\ \theta_t &= \theta_{t-1} - \eta \cdot \mathrm{sign}(u_t) \\ m_t &= \beta_{2m}[t-1] + (1 - \beta_{2m})g_t\end{aligned}$$

这里的重点是：Lion 不维护 Adam 的二阶矩 v_t ，但 sign update 会改变学习率和 weight decay 的合适尺度，不能照搬 AdamW 超参。

优点：

1. optimizer state 更少。
2. 更新形式简单。
3. 在部分视觉和语言任务上有不错结果。

缺点：

1. 学习率和 weight decay 通常不能直接沿用 AdamW。
2. 对大模型预训练的通用性需要结合具体实验判断。
3. sign update 可能带来不同的稳定性特征。

面试中可以把 Lion 作为 “AdamW 之外的轻量优化器探索” 来讲，不要夸成无条件替代 AdamW。

8. Sophia：利用二阶信息的高效近似

Sophia 这类方法关注的是：Adam 只用梯度一阶和二阶矩，能不能利用 Hessian 相关信息提高训练效率？

它试图用可承受的方式近似二阶信息。

可以用一个简化形式理解：

$$\begin{aligned}\theta_t &= \\ \theta_{t-1} & \\ -\eta \cdot & \\ \mathrm{clip} & \\ \left(&\end{aligned}$$

$$\frac{m_t}{h_t + \epsilon},$$

$$\rho$$

$$\right)$$

其中 h_t 是 Hessian 或曲率相关近似, ρ 是裁剪阈值。真实 Sophia 实现有更具体的估计和更新细节, 本章只强调它代表 “用可承受的二阶信息改善优化” 的路线。

潜在优点:

1. 提高样本效率。
2. 更好适配 loss landscape 曲率。
3. 可能减少训练 steps。

挑战:

1. 实现复杂。
2. 二阶估计成本和噪声要控制。
3. 大规模 LLM recipe 不如 AdamW 成熟。

9. Muon: 矩阵参数更新的几何约束

Muon 是近年较受关注的新优化器思路之一。

它的核心动机是: 神经网络里很多重要参数是矩阵, 例如:

attention projection matrix
FFN up/down/gate matrix
embedding projection

普通 AdamW 把矩阵参数看成很多独立元素分别做自适应更新。

Muon 更强调矩阵整体的几何结构, 常见理解是对矩阵梯度或更新方向做正交化/规范化, 让更新方向更接近 “结构良好” 的矩阵变换。

如果矩阵参数是 W , 梯度是 G , 可以把这类方法抽象成:

$$\Delta W = \Phi(G)$$

$$W_t = W_{t-1} - \eta \Delta W$$

其中 Φ 表示对矩阵更新方向做归一化、正交化或其他几何约束的变换。不同实现的 Φ 不完全相同, 面试中不要把某一种实现说成全部 Muon 路线。

可以用直觉理解:

AdamW: 每个参数元素单独自适应缩放。

Muon: 把二维权重矩阵作为整体, 约束或改善它的更新方向。

为什么这可能有用?

大模型中, 矩阵乘法是主要计算。

如果矩阵更新方向病态, 可能导致:

1. 某些方向更新过强。
2. 表示空间塌缩或过度拉伸。
3. 训练不稳定。
4. 有效 rank 或谱结构不理想。

Muon 类方法希望通过更好的矩阵更新几何, 改善训练效率和稳定性。

9.1 Muon 和 AdamW 的关系

Muon 不一定要替代所有参数上的 AdamW。

实际讨论中常见做法是:

对二维矩阵参数使用 Muon

对 embedding、bias、norm 参数等使用 AdamW 或其他规则

原因是 Muon 的核心优势针对矩阵结构。

对于一维参数，例如 LayerNorm/RMSNorm 的 scale，未必适合做同样的矩阵正交化。

9.2 Muon 的优势

潜在优势包括：

1. 对矩阵参数更新更有结构感。
2. 可能提高训练速度或最终效果。
3. 可能降低对部分 AdamW 超参的依赖。
4. 与大模型中大量矩阵乘法结构匹配。

9.3 Muon 的风险和边界

需要谨慎看待：

1. 它是相对新的训练优化方向。
2. 不同模型规模和任务上的稳定结论还需要更多验证。
3. 工具链成熟度不如 AdamW。
4. 可能需要重新调学习率、weight decay、warmup 和参数分组。
5. 分布式训练中要考虑额外计算和实现成本。

面试中可以这样说：

AdamW 把每个参数元素独立做自适应更新，而 Muon 更关注矩阵参数作为整体的更新几何，常通过正交化或规范化改善矩阵更新方向。

10. 参数分组策略

大模型训练通常不会对所有参数使用完全相同的优化规则。

常见参数分组：

1. weight matrix: 使用 AdamW/Muon，启用 weight decay。
2. bias: 通常不使用 weight decay。
3. norm 参数: 通常不使用 weight decay。
4. embedding: 可单独设置 lr 或 weight decay。
5. LoRA 参数: 微调时常设置更大学习率。

为什么 norm 和 bias 通常不 decay?

因为它们不是普通权重矩阵，过度衰减可能影响尺度控制和训练稳定性。

参数分组可以写成互不重叠的集合：

```
\Theta=
\Theta_{\mathrm{matrix}}
\cup
\Theta_{\mathrm{norm}}
\cup
\Theta_{\mathrm{bias}}
\cup
\Theta_{\mathrm{embed}}
```

常见做法是：

```
\lambda(\theta)=
\begin{cases}
\lambda_w, & \theta \in \Theta_{\mathrm{matrix}} \\
\end{cases}
```


0, & \theta \in \Theta_{\mathrm{norm}} \cup \Theta_{\mathrm{bias}} \\ \end{cases}

也就是说，对矩阵权重做 weight decay，对 norm 和 bias 不做 decay。LoRA、embedding 或新增 token 可以再单独分组。

11. Learning Rate Warmup

大模型训练几乎都会使用 warmup。

原因是训练初期参数还不稳定，直接使用大学习率容易导致 loss spike 或 NaN。

warmup 做法：

lr 从 0 或很小值逐步增加到 peak lr

线性 warmup 可以写成：

$\eta_t = \eta_{\mathrm{max}} \frac{t}{T_{\mathrm{warm}}}$

其中 T_{warm} 是 warmup step 数， η_{max} 是 peak learning rate。

常见设置：

warmup_steps = 总 steps 的 1% 到 5%

如果 warmup 太短：

初期容易不稳定。

如果 warmup 太长：

训练前期学习太慢，浪费计算。

12. Cosine Decay

warmup 后常用 cosine decay：

lr 从 peak lr 平滑下降到 min lr

warmup 之后的 cosine decay 可以写成：

$$\eta_t = \eta_{\mathrm{min}} + \frac{1}{2} (\eta_{\mathrm{max}} - \eta_{\mathrm{min}}) \left[1 + \cos \left(\frac{t - T_{\mathrm{warm}}}{T_{\mathrm{total}} - T_{\mathrm{warm}}} \pi \right) \right]$$

这个公式适用于 $t > T_{\mathrm{warm}}$ 的阶段。真实训练还要明确 step 是 optimizer step 还是 micro step；学习率 scheduler 通常按 optimizer step 更新。

优点：

1. 下降平滑。
2. 后期有利于收敛。
3. 大模型 recipe 成熟。

常见组合：

linear warmup + cosine decay

13. Weight Decay

weight decay 控制参数规模，帮助泛化和稳定训练。

大模型里要注意：

1. AdamW 中 weight decay 是 decoupled。
2. norm 和 bias 通常不 decay。
3. weight decay 过大可能欠拟合。
4. weight decay 过小可能参数范数变大，泛化变差。

对于 LoRA/SFT，weight decay 常常要重新调，不要照搬预训练设置。

14. Gradient Clipping

Gradient clipping 用于防止梯度爆炸。

常见做法是 global norm clipping：

$$\begin{aligned} \|g\|_2 &= \sqrt{\sum_i g_i^2} \\ g' &= g \cdot \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right) \end{aligned}$$

大模型训练中常见阈值：

$$\text{clip_grad_norm} = 1.0$$

但这不是固定答案。

如果经常触发 clipping，说明可能存在：

1. 学习率过大。
2. batch 数据异常。
3. loss scale 问题。
4. 模型初始化或数值精度问题。

15. Batch Size Scaling

batch size 和学习率强相关。

batch size 增大后，梯度噪声降低，可以适当增大学习率。

但大模型训练不是简单线性关系。

LLM 中更常用 token batch 口径：

$$\begin{aligned} B_{\{\mathrm{tok}\}} &= \\ B_{\{\mathrm{micro}\}} & \\ &\cdot \\ T_{\{\mathrm{seq}\}} & \\ &\cdot \\ A & \\ &\cdot \\ P_{\{\mathrm{dp}\}} & \end{aligned}$$

其中 B_{micro} 是每张卡每个 micro step 的样本数， T_{seq} 是序列长度， A 是 gradient accumulation steps， P_{dp} 是数据并行规模。

如果 batch token 数从 B_1 变成 B_2 ，线性缩放只是一种起点：

$$\eta_2 \approx \eta_1 \frac{B_2}{B_1}$$

大模型训练中通常还要结合 warmup、grad norm、loss spike、数据质量和 validation loss 做实际搜索。

需要关注：

1. global batch size。
2. gradient accumulation。
3. token batch size, 而不是 sample 数。
4. sequence length 变化。
5. loss spike 风险。

对于 LLM，常说 batch size 时更应该说：

每步训练多少 token

而不是只说多少条样本。

下面这个 demo 用标准库审计一个最小优化器配置。它同时演示：linear warmup + cosine decay、global norm clipping、AdamW 和耦合 L2 Adam 的差异、effective token batch、Adafactor 状态节省、Lion sign update、Sophia 裁剪更新和 Muon 风格的矩阵更新归一化。

```
import math

base_lr = 0.1
min_lr_ratio = 0.1
warmup_steps = 3
total_steps = 10
weight_decay = 0.1
beta1 = 0.9
beta2 = 0.95
eps = 1e-8
clip_norm = 1.0

def grad(theta):
    return theta - 3.0

def loss(theta):
    return 0.5 * (theta - 3.0) ** 2

def lr_at(step):
    if step <= warmup_steps:
        return base_lr * step / warmup_steps
    progress = (step - warmup_steps) / max(total_steps - warmup_steps, 1)
    cosine = 0.5 * (1.0 + math.cos(math.pi * progress))
    return base_lr * (min_lr_ratio + (1.0 - min_lr_ratio) * cosine)

def clip_grad(g, max_norm):
    norm = abs(g)
    scale = min(1.0, max_norm / (norm + 1e-12))
    return g * scale, norm, scale

def adamw_step(theta, g, m, v, step, lr):
    m = beta1 * m + (1 - beta1) * g
    v = beta2 * v + (1 - beta2) * g * g
    m_hat = m / (1 - beta1**step)
```

```

v_hat = v / (1 - beta2**step)
adam_update = m_hat / (math.sqrt(v_hat) + eps)
theta = theta - lr * adam_update - lr * weight_decay * theta
return theta, m, v, adam_update

```

```

def coupled_adam_step(theta, g, m, v, step, lr):
    g = g + weight_decay * theta
    m = beta1 * m + (1 - beta1) * g
    v = beta2 * v + (1 - beta2) * g * g
    m_hat = m / (1 - beta1**step)
    v_hat = v / (1 - beta2**step)
    theta = theta - lr * m_hat / (math.sqrt(v_hat) + eps)
    return theta, m, v

```

```

theta = 8.0
m = 0.0
v = 0.0
trace = []
for step in range(1, 6):
    lr = lr_at(step)
    raw_grad = grad(theta)
    clipped_grad, raw_norm, clip_scale = clip_grad(raw_grad, clip_norm)
    theta, m, v, adam_update = adamw_step(theta, clipped_grad, m, v, step, lr)
    trace.append(
        {
            "step": step,
            "lr": round(lr, 5),
            "raw_grad": round(raw_grad, 5),
            "clip_scale": round(clip_scale, 5),
            "theta": round(theta, 5),
            "loss": round(loss(theta), 5),
        }
    )

```

```

same_theta = 8.0
aw_m = aw_v = ca_m = ca_v = 0.0
first_grad, _, _ = clip_grad(grad(same_theta), clip_norm)
adamw_theta, aw_m, aw_v, _ = adamw_step(same_theta, first_grad, aw_m, aw_v, 1, base_lr)
coupled_theta, ca_m, ca_v = coupled_adam_step(same_theta, first_grad, ca_m, ca_v, 1, base_lr)

```

```

schedule = [round(lr_at(step), 5) for step in range(1, total_steps + 1)]
num_params = 4096 * 11008
adafactor_second_moment = 4096 + 11008
adamw_state_values = 2 * num_params
state_ratio = adamw_state_values / adafactor_second_moment

```

```

micro_batch = 2
seq_len = 4096
accum_steps = 8
dp_size = 16
effective_tokens = micro_batch * seq_len * accum_steps * dp_size

```

```

lion_update = -base_lr * (1 if (beta1 * 0.0 + (1 - beta1) * 0.25) > 0 else -1)

```

```

sophia_update = -base_lr * max(min(0.25 / (0.8 + 1e-12), 0.2), -0.2)
muon_matrix = [[3.0, 0.0], [4.0, 0.0]]
muon_norm = math.sqrt(sum(x * x for row in muon_matrix for x in row))
muon_unit = [[round(x / muon_norm, 4) for x in row] for row in muon_matrix]

checks = {
    "warmup_reaches_peak": schedule[2] == base_lr,
    "cosine_decays": schedule[-1] == round(base_lr * min_lr_ratio, 5),
    "clip_triggered": trace[0]["clip_scale"] < 1.0,
    "loss_decreased": trace[-1]["loss"] < trace[0]["loss"],
    "adamw_differs_from_coupled": round(adamw_theta, 6) != round(coupled_theta, 6),
    "adafactor_saves_state": state_ratio > 1000,
    "effective_tokens_ok": effective_tokens == 1_048_576,
}

```

```

print("schedule=", schedule)
print("trace=", trace)
print("adamw_theta_step1=", round(adamw_theta, 6))
print("coupled_adam_theta_step1=", round(coupled_theta, 6))
print("effective_tokens=", effective_tokens)
print("adamw_state_values=", adamw_state_values)
print("adafactor_state_values=", adafactor_second_moment)
print("state_ratio=", round(state_ratio, 1))
print("lion_update=", round(lion_update, 4))
print("sophia_clipped_update=", round(sophia_update, 4))
print("muon_unit_update=", muon_unit)
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

运行后应看到：

```

schedule= [0.03333, 0.06667, 0.1, 0.09554, 0.08306, 0.06501, 0.04499, 0.02694, 0.01446, 0.01]
trace= [{'step': 1, 'lr': 0.03333, 'raw_grad': 5.0, 'clip_scale': 0.2, 'theta': 7.94, 'loss': 12.2018}, {'step': 2, 'lr':
adamw_theta_step1= 7.82
coupled_adam_theta_step1= 7.9
effective_tokens= 1048576
adamw_state_values= 90177536
adafactor_state_values= 15104
state_ratio= 5970.4
lion_update= -0.1
sophia_clipped_update= -0.02
muon_unit_update= [[0.6, 0.0], [0.8, 0.0]]
checks= {'warmup_reaches_peak': True, 'cosine_decays': True, 'clip_triggered': True, 'loss_decreased': True, 'adamw_differs_from_coupled': True, 'adafactor_saves_state': True, 'effective_tokens_ok': True}
gate_pass= True

```

这个 demo 的重点是把优化器 recipe 变成可检查对象：scheduler 是否按 optimizer step 变化、clipping 是否触发、AdamW 是否真的 decoupled、token batch 是否达到目标、轻量优化器节省了什么状态，以及新优化器为什么要重新调参。

16. 常见优化器配置示例

16.1 预训练 AdamW 配置

```

optimizer: AdamW
beta1: 0.9
beta2: 0.95
eps: 1e-8

```

weight_decay: 0.1
lr_schedule: linear warmup + cosine decay
grad_clip: 1.0

16.2 SFT 配置

optimizer: AdamW
lr: 通常小于预训练或按数据规模调节
weight_decay: 0.0 到 0.1
warmup_ratio: 0.03
grad_clip: 1.0

16.3 LoRA 微调配置

optimizer: AdamW
lr: 通常比全参微调更大
weight_decay: 常较小
trainable_params: LoRA A/B matrix

16.4 Muon 探索配置

matrix_params: Muon
norm/bias/embedding_params: AdamW 或单独规则
lr: 需要重新搜索
warmup: 建议保守设置
eval: 必须和 AdamW baseline 对比

17. 训练不稳定时如何排查

如果出现 loss spike、NaN 或训练发散，可以按下面顺序排查。

17.1 学习率是否过大

现象：

loss 突然升高
grad norm 爆炸
参数范数异常增大

处理：

降低 peak lr
延长 warmup
增大 grad clipping
回滚 checkpoint

17.2 warmup 是否太短

训练初期 spike 常常和 warmup 太短有关。

处理：

warmup_steps 增加 2 到 5 倍做对比实验

17.3 beta2 是否不合适

AdamW 的 beta2 控制二阶矩平滑。

beta2 太大，二阶统计响应慢。

beta2 太小，更新可能更噪。

大模型预训练中常见 0.95 或 0.999，需要结合 batch size 和训练稳定性调。

17.4 weight decay 是否过强

weight decay 过大可能导致欠拟合或能力变差。

尤其是 SFT、RLHF 和 LoRA 阶段，不要无脑沿用预训练 decay。

17.5 参数分组是否正确

常见错误：

对 norm 参数使用 weight decay

对 frozen 参数传入 optimizer

LoRA 参数学习率过大

embedding 使用不合适 decay

17.6 新优化器是否没有重新调参

从 AdamW 换成 Muon、Lion、Sophia 时，不能照搬 AdamW 超参。

必须重新搜索：

lr
weight_decay
warmup
grad_clip
参数分组

18. 面试高频问法

问法 1：Adam 和 AdamW 有什么区别？

可以这样答：

Adam 使用一阶动量和二阶动量做自适应更新，但普通 L2 正则会和 Adam 的梯度归一化耦合。AdamW 把 weight decay 从梯度更新里解耦。

问法 2：为什么大模型训练常用 AdamW？

可以这样答：

因为 Transformer 参数多、梯度尺度差异大，AdamW 的自适应学习率和动量机制能提供较快且稳定的收敛。同时 decoupled weight decay 效果更好。

问法 3：Muon 和 AdamW 有什么不同？

可以这样答：

AdamW 基本把参数元素独立看待，维护一阶和二阶矩做自适应更新。Muon 更关注矩阵参数整体的更新几何，通常对矩阵梯度或更新方向做归一化。

问法 4：Adafactor 解决什么问题？

可以这样答：

Adafactor 主要解决 Adam/AdamW optimizer state 显存过大的问题。它对矩阵参数的二阶矩估计做分解近似，只保存行列统计，从而大幅降低显存。

问法 5：训练 loss spike 时如何从优化器角度排查？

可以这样答：

我会先看学习率是否过大、warmup 是否太短、grad norm 是否爆炸、gradient clipping 是否生效。然后检查 AdamW 的 beta2、eps、

19. 本章小结

本章系统介绍了大模型训练中的优化器和学习率策略。

核心结论如下：

1. 优化器决定拿到梯度后如何更新参数。
2. SGD 简单但大模型训练中通常不如 Adam 系方法稳定高效。
3. Adam 使用一阶和二阶动量做自适应更新。
4. AdamW 通过 decoupled weight decay 成为 LLM 训练强基线。
5. Adafactor 主要解决 optimizer state 显存问题。
6. Lion 用更少状态和 sign update 探索轻量优化。
7. Sophia 代表利用二阶信息近似提高训练效率的方向。
8. Muon 关注矩阵参数整体更新几何，是当前值得关注的新优化器路线。
9. 新优化器不能无脑替换 AdamW，必须重新调 lr、weight decay、warmup 和参数分组。
10. warmup、cosine decay、gradient clipping、batch size scaling 和优化器必须一起看。
11. 训练不稳定时，优化器和学习率策略是最先排查的方向之一。

第七章：分布式训练

本章目标

理解大模型训练为什么必须分布式，以及各种并行方式解决什么瓶颈。

核心议题

1. Data Parallelism 与 DDP。
2. ZeRO 与 FSDP。
3. Tensor Parallelism。
4. Pipeline Parallelism。
5. Sequence Parallelism 与 Context Parallelism。
6. 3D parallelism。
7. 通信开销与计算重叠。

面试重点

面试中不要只背名词，要能说明每种并行方式切分了什么、节省了什么、增加了什么通信成本。

为什么大模型必须分布式训练

大模型训练通常无法只靠单张 GPU 完成。

原因有三类：

1. 参数太多，单卡放不下模型权重、梯度和 optimizer state。
2. batch 太大，单卡放不下激活和中间结果。
3. token 太多，单卡训练时间不可接受。

以 AdamW 为例，训练时不只需要保存模型参数，还需要梯度、一阶动量、二阶动量。再加上激活、临时 buffer、通信 buffer，实际显存开销远大于模型权重本身。

分布式训练的核心目标是：

把参数、数据、激活或序列切到多张 GPU 上，同时尽量保持计算效率。

面试表达：分布式训练不是简单“多用几张卡”，而是在显存、计算、通信和工程复杂度之间做切分设计。

分布式训练的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 PyTorch DistributedDataParallel / FSDP 文档、DeepSpeed ZeRO 论文与文档、Megatron-LM 的 tensor / pipeline parallelism 资料，以及 PyTorch 性能调优中关于梯度同步和通信重叠的说明。

本章聚焦分布式训练的架构口径：Data Parallel、DDP、ZeRO、FSDP、Tensor Parallel、Pipeline Parallel、Sequence / Context Parallel、Expert Parallel、通信开销、global batch 和 activation checkpointing。它不展开 NCCL 内部算法、完整 Megatron / DeepSpeed / FSDP 配置文件、真实集群拓扑调优、checkpoint 分片格式和生产训练平台实现。

训练状态显存口径

设模型参数量为 P ，参数字节数为 b_w ，梯度字节数为 b_g ，AdamW 一阶和二阶状态合计字节数为 b_o 。DDP 每张卡都保存完整训练状态，单卡训练状态显存近似是：

$$M_{\{\mathrm{DDP}\}} \approx P(b_w + b_g + b_o)$$

如果数据并行规模是 N_{dp} ，ZeRO / FSDP 分片后可以粗略写成：

$$M_{\{\mathrm{ZeRO1}\}} \approx P(b_w + b_g) + \frac{P b_o}{N_{\{\mathrm{dp}\}}}$$

$$M_{\{\mathrm{ZeRO2}\}} \approx P b_w + \frac{P(b_g + b_o)}{N_{\{\mathrm{dp}\}}}$$

$$M_{\{\mathrm{ZeRO3}\}} \approx \frac{P(b_w + b_g + b_o)}{N_{\{\mathrm{dp}\}}}$$

真实显存还要加 activation、temporary buffer、communication bucket、fragmentation 和 framework overhead，但这些公式能帮助你解释为什么 ZeRO-3 / FSDP 更省显存，也为什么它们会带来更多 gather / scatter 通信。

1. 分布式训练要解决哪些瓶颈

大模型训练常见瓶颈包括：

瓶颈	表现	常见解决方式
参数显存	模型权重、梯度、optimizer state 放不下	ZeRO、FSDP、Tensor Parallel
激活显存	sequence length 或 batch 太大导致 OOM	activation checkpointing、sequence parallel
计算时间	单卡训练太慢	data parallel、tensor parallel、pipeline parallel
通信开销 数据加载	多卡同步慢，GPU 等待 GPU 空转	overlap、拓扑优化、通信压缩 dataloader 优化、预取、数据 shard

不同并行方式解决的问题不同。不能只说“用了分布式”，要说清楚切分对象是什么。

2. Data Parallelism

Data Parallelism 是最直观的并行方式。

每张 GPU 都复制一份完整模型，但处理不同 mini-batch 数据。

GPU 0: model copy + batch 0
GPU 1: model copy + batch 1
GPU 2: model copy + batch 2
GPU 3: model copy + batch 3

每张卡独立 forward/backward, 然后同步梯度, 再更新参数。

Data Parallel 的 global batch 可以写成:

$$B_{\{\mathrm{global}\}} = B_{\{\mathrm{micro}\}} \cdot A \cdot N_{\{\mathrm{dp}\}}$$

其中 B_{micro} 是每张卡的 micro-batch size, A 是 gradient accumulation steps, N_{dp} 是 data parallel rank 数。对于 LLM, 更常用 token 口径:

$$B_{\{\mathrm{tok}\}} = B_{\{\mathrm{micro}\}} \cdot T_{\{\mathrm{seq}\}} \cdot A \cdot N_{\{\mathrm{dp}\}}$$

2.1 优点

1. 概念简单。
2. 工程成熟。
3. 适合模型能放进单卡的场景。
4. 能有效扩大 global batch size。

2.2 缺点

1. 每张卡都要保存完整模型。
2. 不能解决单卡放不下模型的问题。
3. 每步需要梯度 all-reduce。

面试表达: Data Parallel 切的是数据, 不切模型。它提高吞吐, 但不降低单卡模型显存。

3. DDP

PyTorch DistributedDataParallel, 简称 DDP, 是常见数据并行实现。

DDP 的基本流程:

1. 每个 rank 持有一份模型副本
2. 每个 rank 读取不同数据 shard
3. forward 计算 loss
4. backward 得到梯度
5. 对梯度做 all-reduce
6. 每个 rank 用相同梯度更新参数

3.1 all-reduce 是什么

all-reduce 会把多个 GPU 上的梯度求和或求平均, 并把结果广播回每个 GPU。

这样每张 GPU 的模型参数保持一致。

如果第 r 个 rank 的梯度是 g_r ，DDP 同步后的平均梯度是：

$$\bar{g} = \frac{1}{N_{\mathrm{dp}}} \sum_{r=1}^{N_{\mathrm{dp}}} g_r$$

ring all-reduce 的单 rank 通信量常用粗略估算是：

$$C_{\mathrm{allreduce}} \approx 2 \frac{N_{\mathrm{dp}} - 1}{N_{\mathrm{dp}}} S_g$$

其中 S_g 是梯度 bucket 大小。这个公式的重点是：DDP 通信量和梯度大小同阶，GPU 数越多，通信和拓扑越重要。

3.2 DDP 的瓶颈

DDP 的主要瓶颈是梯度同步通信。

当模型很大、网络带宽不足或 GPU 数量很多时，all-reduce 可能成为训练吞吐瓶颈。

DDP 还不能解决模型本身放不下单卡的问题。

4. ZeRO

ZeRO 的核心思想是：不要让每张 GPU 都保存完整训练状态，而是把训练状态切分到不同 GPU 上。

训练状态包括：

1. optimizer state。
2. gradients。
3. parameters。

ZeRO 常分为三个阶段。

阶段	切分内容	解决什么
ZeRO-1	optimizer state	减少 AdamW 状态显存
ZeRO-2	optimizer state + gradients	进一步减少梯度显存
ZeRO-3	optimizer state + gradients + parameters	参数也切分，支持更大模型

4.1 ZeRO-1

ZeRO-1 只切 optimizer state。

对于 AdamW，这很有价值，因为一阶动量和二阶动量很占显存。

4.2 ZeRO-2

ZeRO-2 进一步切 gradient。

每张卡不需要保存完整梯度，只保存自己负责的部分。

4.3 ZeRO-3

ZeRO-3 连参数也切分。

这可以训练更大模型，但通信更复杂，因为 forward/backward 时需要按需 gather 参数。

面试表达：ZeRO 本质是把训练状态做 shard，换取显存下降，代价是通信和工程复杂度上升。

5. FSDP

FSDP, Fully Sharded Data Parallel, 是 PyTorch 生态中常用的全切分数据并行方案。

它和 ZeRO-3 思想接近: 参数、梯度、optimizer state 都可以 shard 到不同 GPU 上。

FSDP 的典型过程:

forward 前: gather 当前层参数

forward 后: 可释放不再需要的完整参数

backward 时: 再次 gather / reduce-scatter 梯度

optimizer step: 只更新本 rank 负责的 shard

5.1 FSDP 的优势

1. 支持训练超过单卡显存的大模型。
2. 和 PyTorch 生态结合较好。
3. 可以结合 activation checkpointing。
4. 适合中大型模型训练和微调。

5.2 FSDP 的代价

1. 通信更频繁。
2. 配置复杂。
3. wrap 策略影响性能。
4. checkpoint 保存和加载更复杂。

面试表达: FSDP 用更复杂的参数 gather 和 shard 机制换取显存节省, 适合模型状态显存成为瓶颈的场景。

可以把 FSDP 的一个 layer 级过程抽象成:

all-gather 参数 shard -> 计算 forward/backward -> reduce-scatter 梯度 -> 释放完整参数

所以它的收益主要来自“常驻显存低”, 代价是每层附近的 gather / scatter 通信更频繁。wrap 粒度太细会导致通信碎片化, wrap 粒度太粗又可能显存峰值过高。

6. Tensor Parallelism

Tensor Parallelism 切的是模型内部的大矩阵计算。

例如一个线性层:

$$Y = X W$$

如果 W 很大, 可以把 W 按列或按行切到多张 GPU 上。

列切分可以写成:

$$W = [W_1, W_2, \dots, W_{N_{\{\mathrm{tp}\}}}]$$

$$Y_i = X W_i$$

最后再按特征维拼接 Y_i 。行切分则通常需要对部分结果做 reduce 或 all-reduce。TP 的核心是切层内矩阵, 不是切样本。

6.1 为什么需要 TP

当单层矩阵太大, 或者单卡算力不足时, TP 可以把一层的计算拆到多卡。

它常用于非常大的 dense model 或 MoE model。

6.2 TP 的通信

TP 通常需要在层内做 all-reduce、all-gather 或 reduce-scatter。

这意味着 TP 对 GPU 间带宽要求很高，通常更适合在同一节点内使用，例如 NVLink 连接的 GPU。

6.3 面试表达

Tensor Parallel 切的是层内矩阵和 hidden dimension，不是切数据。它能解决单层太大或单卡算力不足的问题，但会增加频繁的层内通信。

7. Pipeline Parallelism

Pipeline Parallelism 把模型按层切成多个 stage。

例如：

GPU 0: layer 1-10
GPU 1: layer 11-20
GPU 2: layer 21-30
GPU 3: layer 31-40

一个 batch 会按 micro-batch 流过这些 stage。

7.1 为什么需要 PP

如果模型层数很多，整模型放不进单卡，可以按层切分。

PP 适合很深的模型。

7.2 Pipeline bubble

Pipeline 的问题是会有 bubble，也就是某些 GPU 等待其他 stage 的空闲时间。

micro-batch 越少，bubble 越明显。

常见做法是把 batch 切成多个 micro-batch，提高 pipeline 利用率。

如果 pipeline stage 数是 N_{pp} ，micro-batch 数是 M ，一个简化 bubble 比例是：

$$r_{\{\mathrm{bubble}\}} \approx \frac{N_{\{\mathrm{pp}\}} - 1}{M + N_{\{\mathrm{pp}\}} - 1}$$

micro-batch 越多，bubble 越小，但调度、激活缓存和通信也会更复杂。

7.3 面试表达

Pipeline Parallel 切的是层。它降低单卡模型显存，但会引入 pipeline bubble、调度复杂度和跨 stage 激活通信。

8. Sequence Parallelism 与 Context Parallelism

当 sequence length 很长时，激活和 attention 相关显存会成为瓶颈。

Sequence Parallelism 或 Context Parallelism 尝试沿序列维度切分计算。

8.1 Sequence Parallelism

Sequence Parallelism 通常把某些 activation 按 sequence dimension 分散到不同 GPU 上，减少每卡激活显存。

它经常和 Tensor Parallel 结合使用。

8.2 Context Parallelism

Context Parallelism 更关注长上下文训练，把长序列切到多张 GPU 上处理。

难点是 attention 需要跨序列位置交互，所以必须设计高效通信。

面试表达：序列并行解决的是长序列激活和 attention 显存问题，代价是跨序列通信和实现复杂度。

9. 3D Parallelism

真实大模型训练常常组合多种并行方式。

所谓 3D parallelism 通常指：

Data Parallel + Tensor Parallel + Pipeline Parallel

也可以再叠加 ZeRO/FSDP、sequence parallel、expert parallel。

一个例子：

总 GPU = DP * TP * PP

如果有 64 张 GPU，可以设计：

DP = 8

TP = 4

PP = 2

总数 = 8 * 4 * 2 = 64

其中：

1. TP 负责切单层矩阵。
2. PP 负责切模型层。
3. DP 负责复制并行组并处理不同数据。

总 world size 可以写成：

$$N_{\mathrm{world}} = N_{\mathrm{dp}} \cdot N_{\mathrm{tp}} \cdot N_{\mathrm{pp}}$$

如果再叠加 expert parallel 或 context parallel，需要把对应维度也纳入 rank mapping。真实工程里 rank mapping 要和节点内 NVLink、节点间 InfiniBand 等拓扑配合。

面试表达：大规模训练通常不是单一并行策略，而是根据模型大小、节点拓扑和通信瓶颈组合多维并行。

10. Expert Parallelism

MoE 模型还会涉及 Expert Parallelism。

MoE 中不同 expert 可以分布在不同 GPU 上。每个 token 经过 router 后被发送到对应 expert。

这会带来 all-to-all 通信。

如果一共有 E 个 expert，每个 token 选择 k 个 expert，一个 batch 内 token 数是 T_tok，capacity factor 是 alpha，每个 expert 的容量可以粗略写成：

$$C_{\mathrm{expert}} = \left\lceil \frac{T_{\mathrm{tok}} k}{\alpha} \right\rceil E$$

如果 router 分配不均，某些 expert 超过容量，就会发生 token drop、padding 浪费或负载不均。

10.1 MoE 的分布式难点

- 1. token 路由不均衡。
- 2. expert 负载不均衡。
- 3. all-to-all 通信昂贵。
- 4. batch size 太小时 expert 利用率差。
- 5. checkpoint 和推理部署更复杂。

面试表达：MoE 的分布式训练难点不只是参数多，而是 token 到 expert 的动态路由会带来负载均衡和 all-to-all 通信问题。

11. 通信开销与计算重叠

分布式训练的效率往往受通信影响。

常见通信操作：

操作	常见场景
all-reduce	DDP 梯度同步
reduce-scatter	ZeRO/FSDP 梯度切分
all-gather	FSDP 参数 gather
all-to-all	MoE expert routing
send/recv	Pipeline stage 之间传激活

为了提高效率，需要尽量让通信和计算重叠。

例如在 backward 过程中，前面层的梯度算出来后可以先开始通信，同时继续计算后面层的梯度。

如果纯计算时间是 T_{comp} ，通信时间是 T_{comm} ，完全不重叠时：

$$T_{\mathrm{step}} \approx T_{\mathrm{comp}} + T_{\mathrm{comm}}$$

如果可以重叠，理想情况下接近：

$$T_{\mathrm{step}} \approx \max(T_{\mathrm{comp}}, T_{\mathrm{comm}})$$

真实系统通常介于两者之间。面试里要能说明：bucket size、backward 顺序、网络拓扑和 collective 调度都会影响 overlap 效果。

面试表达：分布式训练的目标不是通信为零，而是减少通信瓶颈并让通信尽量和计算重叠。

12. Global batch size

分布式训练中，global batch size 的计算非常重要。

$$\text{global_batch} = \text{micro_batch_size} * \text{gradient_accumulation_steps} * \text{data_parallel_size}$$

如果还使用 sequence packing，可以进一步按 token 数统计：

$$\text{tokens_per_step} = \text{global_batch} * \text{sequence_length}$$

学习率、warmup、训练 token 数和吞吐都要基于 global batch 或 tokens per step 理解。

常见错误是只看单卡 batch size，忽略 data parallel 和 gradient accumulation 后的实际 global batch。

训练总 token 数为 D 时，optimizer step 数近似是：

$$S_{\mathrm{opt}} = \left\lceil \frac{D}{B_{\mathrm{tok}}} \right\rceil$$

这会影响学习率 schedule、checkpoint 间隔、validation 间隔和训练时长估算。

13. Activation checkpointing

Activation checkpointing 也叫 gradient checkpointing。

它的思路是 forward 时不保存所有中间激活，backward 时重新计算一部分 forward。

优点：

1. 显著降低激活显存。
2. 支持更大 batch 或更长 sequence。

代价：

1. backward 更慢。
2. 计算量增加。
3. checkpoint 粒度影响性能。

如果原始 activation 显存是 M_{act} ，切成 K 段 checkpoint，一个粗略直觉是：

$M_{\mathrm{act, ckpt}} \approx \frac{M_{\mathrm{act}}}{K}$

同时 backward 会重算部分 forward，额外计算因子可以粗略理解为：

$R_{\mathrm{compute}} > 1$

实际节省和额外计算取决于 checkpoint 粒度、模型结构和框架实现。

面试表达：activation checkpointing 是用额外计算换激活显存，常和 FSDP、TP、长上下文训练一起使用。

14. 分布式训练常见故障

14.1 吞吐不随 GPU 数增加

可能原因：

1. 通信成为瓶颈。
2. batch 太小。
3. dataloader 跟不上。
4. pipeline bubble 太大。
5. 节点间网络慢。

14.2 某些 GPU 显存明显更高

可能原因：

1. pipeline stage 切分不均。
2. MoE expert 负载不均。
3. rank 0 保存额外对象。
4. checkpoint 或 logging 集中在某个 rank。

14.3 训练 hang 住

可能原因：

1. 某个 rank 报错但其他 rank 等待通信。
2. collective 调用顺序不一致。

3. 数据 shard 长度不一致。
4. 网络故障。

14.4 loss 不一致

可能原因：

1. 随机种子不一致。
2. 数据 shard 重叠或缺失。
3. gradient accumulation 逻辑错误。
4. mixed precision 配置不一致。

15. 如何选择并行策略

可以按下面顺序思考：

1. 模型能不能放进单卡？
能：优先 DDP。
不能：考虑 FSDP/ZeRO 或 TP/PP。
2. 单层矩阵是否太大？
是：考虑 Tensor Parallel。
3. 层数是否很多？
是：考虑 Pipeline Parallel。
4. 序列是否很长？
是：考虑 activation checkpointing、sequence/context parallel。
5. 是否是 MoE？
是：考虑 expert parallel 和 all-to-all 优化。
6. 通信拓扑如何？
节点内适合高通信 TP，跨节点更谨慎。

面试中可以强调：并行策略要根据瓶颈选择，而不是所有技术都堆上去。

下面这个 demo 用标准库审计一个 64 GPU 的分布式训练配置。它不启动真实分布式进程，而是检查 world size、DDP/ZeRO 显存、ring all-reduce 流量、global token batch、pipeline bubble、MoE expert capacity 和 activation checkpointing 取舍。

```
import math
```

```
cfg = {  
    "params": 6_000_000_000,  
    "weight_bytes": 2,  
    "grad_bytes": 2,  
    "adam_state_bytes": 8,  
    "dp": 8,  
    "tp": 4,  
    "pp": 2,  
    "micro_batch": 2,  
    "seq_len": 4096,  
    "accum_steps": 4,  
    "grad_bucket_gib": 4.0,  
    "bandwidth_gbps": 200,  
    "micro_batches_for_pp": 8,  
    "tokens": 8192,
```

```

    "experts": 8,
    "top_k": 2,
    "capacity_factor": 1.25,
}

```

```
GIB = 1024**3
```

```

def gib(bytes_value):
    return bytes_value / GIB

```

```

def memory_estimates(c):
    p = c["params"]
    w = p * c["weight_bytes"]
    g = p * c["grad_bytes"]
    o = p * c["adam_state_bytes"]
    n = c["dp"]
    return {
        "ddp_gib": gib(w + g + o),
        "zero1_gib": gib(w + g + o / n),
        "zero2_gib": gib(w + (g + o) / n),
        "zero3_gib": gib((w + g + o) / n),
        "weight_gib": gib(w),
        "grad_gib": gib(g),
        "adam_state_gib": gib(o),
    }

```

```

def ring_all_reduce_traffic_gib(payload_gib, ranks):
    return 2 * (ranks - 1) / ranks * payload_gib

```

```

def transfer_seconds(gib_value, gbps):
    bytes_value = gib_value * GIB
    return bytes_value / (gbps * 1e9)

```

```

def pipeline_bubble_fraction(pp, micro_batches):
    return (pp - 1) / (micro_batches + pp - 1)

```

```

def expert_capacity(tokens, top_k, experts, capacity_factor):
    return math.ceil(capacity_factor * tokens * top_k / experts)

```

```

mem = memory_estimates(cfg)
world_size = cfg["dp"] * cfg["tp"] * cfg["pp"]
effective_tokens = cfg["micro_batch"] * cfg["seq_len"] * cfg["accum_steps"] * cfg["dp"]
all_reduce_traffic = ring_all_reduce_traffic_gib(cfg["grad_bucket_gib"], cfg["dp"])
all_reduce_time_s = transfer_seconds(all_reduce_traffic, cfg["bandwidth_gbps"])
bubble = pipeline_bubble_fraction(cfg["pp"], cfg["micro_batches_for_pp"])
capacity = expert_capacity(
    cfg["tokens"],
    cfg["top_k"],

```

```

    cfg["experts"],
    cfg["capacity_factor"],
)

activation_gib = 24.0
checkpoint_segments = 4
checkpointed_activation_gib = activation_gib / checkpoint_segments
extra_compute_factor = 1.0 + 1.0 / checkpoint_segments

checks = {
    "world_size_matches": world_size == 64,
    "zero3_saves_memory": mem["zero3_gib"] < mem["zero2_gib"] < mem["zero1_gib"] < mem["ddp_gib"],
    "effective_tokens_ok": effective_tokens == 262144,
    "bubble_under_15pct": bubble < 0.15,
    "expert_capacity_enough": capacity * cfg["experts"] >= cfg["tokens"] * cfg["top_k"],
    "checkpoint_saves_activation": checkpointed_activation_gib < activation_gib,
}

print("world_size=", world_size)
print("memory_gib=", {k: round(v, 2) for k, v in mem.items()})
print("zero3_vs_ddp_ratio=", round(mem["zero3_gib"] / mem["ddp_gib"], 3))
print("effective_tokens=", effective_tokens)
print("all_reduce_traffic_gib=", round(all_reduce_traffic, 3))
print("all_reduce_time_s=", round(all_reduce_time_s, 4))
print("pipeline_bubble=", round(bubble, 4))
print("expert_capacity=", capacity)
print("expert_capacity_total=", capacity * cfg["experts"])
print("checkpointed_activation_gib=", round(checkpointed_activation_gib, 2))
print("extra_compute_factor=", round(extra_compute_factor, 2))
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

运行后应看到：

```

world_size= 64
memory_gib= {'ddp_gib': 67.06, 'zero1_gib': 27.94, 'zero2_gib': 18.16, 'zero3_gib': 8.38, 'weight_gib': 11.18, 'grad_gib': 11.18}
zero3_vs_ddp_ratio= 0.125
effective_tokens= 262144
all_reduce_traffic_gib= 7.0
all_reduce_time_s= 0.0376
pipeline_bubble= 0.1111
expert_capacity= 2560
expert_capacity_total= 20480
checkpointed_activation_gib= 6.0
extra_compute_factor= 1.25
checks= {'world_size_matches': True, 'zero3_saves_memory': True, 'effective_tokens_ok': True, 'bubble_under_15pct': True, 'expert_capacity_enough': True, 'checkpoint_saves_activation': True}
gate_pass= True

```

这个 demo 的重点是把分布式训练回答从“用了 FSDP / ZeRO / TP”推进到可验证口径：每卡显存是否下降、global token batch 是否符合预期、通信时间是否可能成为瓶颈、pipeline bubble 是否可接受、MoE capacity 是否能覆盖 token 路由。

16. 面试官会怎么问

问法 1：DDP、ZeRO、FSDP 有什么区别？

可以这样答：

DDP 是数据并行，每张卡保留完整模型和 optimizer state，只同步梯度，所以简单但不能解决单卡放不下模型的问题。ZeRO 把 optimizer state 分片，把参数、梯度和优化器状态分片，forward/backward 时按需 gather 参数，显存省但通信更复杂。

问法 2：Tensor Parallel 和 Pipeline Parallel 切分什么？

可以这样答：

Tensor Parallel 切的是层内矩阵计算，比如 attention 或 MLP 的大矩阵，所以通信频繁但适合单层很大的模型。Pipeline Parallel 切的是模型层，适合很深的模型，但有 pipeline bubble。

问法 3：ZeRO-3 为什么省显存但通信更大？

可以这样答：

ZeRO-3 连参数也分片，每张 GPU 只保存部分参数、梯度和 optimizer state，所以显存显著下降。但 forward/backward 需要按层 gather/scatter，因此通信更频繁。

问法 4：为什么 MoE 分布式训练难？

可以这样答：

MoE 中 token 会被 router 动态分配到不同 expert。不同 expert 负载可能不均衡，而且 token 需要跨 GPU 发送到对应 expert，带来 all-to-all 通信。训练时还要处理 load balance loss、expert capacity、drop token 和 checkpoint 等问题。

问法 5：训练吞吐低怎么排查？

可以这样答：

我会先分解时间：dataloader、forward、backward、optimizer、communication、checkpoint。再看 GPU utilization、tokens/sec。

17. 本章小结

本章核心结论：

1. 分布式训练解决的是显存、计算时间和吞吐瓶颈。
2. Data Parallel 切数据，不切模型，不能解决单卡放不下模型的问题。
3. DDP 通过梯度 all-reduce 保持模型副本一致。
4. ZeRO/FSDP 通过切分 optimizer state、gradient 和 parameter 降低显存。
5. Tensor Parallel 切层内矩阵，适合单层很大的模型。
6. Pipeline Parallel 切模型层，适合很深的模型，但有 pipeline bubble。
7. Sequence/Context Parallel 面向长序列训练的激活和 attention 瓶颈。
8. MoE 需要 expert parallel，并面对 all-to-all 和负载均衡问题。
9. 分布式训练效率高度依赖通信、拓扑、batch size 和计算通信重叠。
10. 面试中要围绕“切了什么、省了什么、增加了什么通信”来回答。

第八章：显存优化与数值精度

本章目标

理解训练显存由哪些部分构成，以及如何通过精度和重计算优化显存。

核心议题

1. 参数、梯度、优化器状态、激活显存。
2. FP32、FP16、BF16、FP8。
3. mixed precision training。
4. activation checkpointing。
5. gradient accumulation。

6. optimizer state sharding。

面试重点

能估算训练显存，并解释为什么 Adam 优化器状态会显著增加显存占用。

为什么显存是训练中的硬约束

大模型训练经常不是先被算力限制，而是先被显存限制。

如果显存不够，训练直接 OOM；如果显存刚好够但没有余量，稍微增加 batch、sequence length、模型层数或打开某些日志，就可能崩掉。

训练显存主要由四类组成：

1. 模型参数。
2. 梯度。
3. 优化器状态。
4. 激活和临时 buffer。

很多初学者只估算模型权重大小，例如“7B 参数，BF16 大约 14GB”，但这远远不等于训练显存。训练时 AdamW 还要保存梯度、一阶动量、二阶动量，反向传播还要保存激活。

面试表达：推理只需要模型权重和 KV Cache，训练还需要梯度、optimizer state 和激活，因此训练显存远大于推理显存。

显存优化与数值精度的资料边界

按照 `WRITING_PLAN.md` 的要求，本章第二轮精修前核对了 PyTorch AMP / GradScaler、PyTorch activation checkpointing、FlashAttention 论文、NVIDIA Transformer Engine FP8 资料，以及前序 ZeRO/FSDP、LoRA/QLoRA 和分布式训练资料。

本章聚焦训练显存预算和数值精度的工程口径：参数、梯度、optimizer state、activation、attention 中间结果、mixed precision、loss scaling、activation checkpointing、gradient accumulation、optimizer state sharding、LoRA/QLoRA、FlashAttention、OOM 与 NaN/Inf 排查。它不展开具体 CUDA allocator、NCCL buffer、kernel workspace、显存碎片调优和 FP8 生产 recipe 的完整实现。

1. 训练显存由哪些部分构成

假设模型参数量为 N 。

如果用 BF16 保存模型权重，每个参数约 2 bytes。

但训练中还要考虑：

部分	作用	典型大小
parameters	模型权重	$N * \text{bytes}$
gradients	每个参数的梯度	$N * \text{bytes}$
optimizer states	AdamW 的 m/v 等状态	通常很大
activations	forward 保存给 backward 用	与 batch、seq、hidden、layers 相关
temporary buffers	通信、kernel、中间计算	与实现有关

AdamW 尤其占显存，因为它维护：

m：一阶动量

v：二阶动量

如果 optimizer state 用 FP32，每个参数的 m 和 v 各 4 bytes，仅 optimizer state 就是：

$$M_{\{\mathrm{opt}\}}=8N$$

这就是为什么训练显存比权重显存大很多。

训练状态显存可以粗略写成：

$$M_{\{\mathrm{state}\}}=N(b_w+b_g+b_o+b_m)$$

其中 b_w 是权重字节数， b_g 是梯度字节数， b_o 是 optimizer state 字节数， b_m 是可选 master weights 字节数。总训练显存还要加激活和临时 buffer：

$$M_{\{\mathrm{train}\}}\approx M_{\{\mathrm{state}\}}+M_{\{\mathrm{act}\}}+M_{\{\mathrm{temp}\}}$$

2. 一个粗略显存估算

以 N 个参数为例，假设：

1. 权重用 BF16: 2 bytes。
2. 梯度用 BF16: 2 bytes。
3. AdamW m/v 用 FP32: 8 bytes。
4. 可能还有 FP32 master weights: 4 bytes。

那么仅模型状态就可能接近：

$$b_{\{\mathrm{train}\}}=2+2+8+4=16$$

如果是 7B 参数：

$$M_{\{\mathrm{state}\}}\approx 7\times 10^9\times 16\approx 112\,\mathrm{GB}$$

这还没有算激活显存。

所以单卡 80GB 训练 7B full fine-tuning 或预训练并不轻松，需要 ZeRO/FSDP、gradient checkpointing、较小 batch 或参数高效微调。

面试表达：估算训练显存时，不能只看权重，还要把 gradient、optimizer state、activation 和 buffer 算进去。

3. 激活显存为什么大

反向传播需要用到 forward 的中间结果，所以训练时要保存 activation。

激活显存大致与下面因素相关：

$$M_{\{\mathrm{act}\}}\propto B_{\{\mathrm{micro}\}}T_{\{\mathrm{seq}\}}dN_Lb$$

sequence length 一变长，激活显存会明显上升。

attention 相关中间结果还可能与 T^2 有关，取决于实现是否使用 FlashAttention 等优化。

标准 attention 如果显式保存每层 attention matrix，中间显存会出现：

$$M_{\{\mathrm{attn}\}}\propto B_{\{\mathrm{micro}\}}H$$

$$T_{\{\mathrm{seq}\}}^2$$

其中 H 是 attention head 数。这个二次项解释了为什么长上下文训练显存压力会非常快地上升。

激活显存的特点

1. 和 batch size 强相关。
2. 和 sequence length 强相关。
3. 和层数强相关。
4. 可以通过重计算减少。

这也是为什么长上下文训练显存压力很大。

4. FP32、FP16、BF16、FP8

训练数值精度直接影响显存、吞吐和稳定性。

精度	bytes	特点	常见用途
FP32	4	精度高，稳定，显存大	optimizer state、部分累积计算
FP16	2	显存省，吞吐高，动态范围小	mixed precision 训练
BF16	2	动态范围接近 FP32，更稳	大模型训练常用
FP8	1	更省显存和带宽，要求硬件和 recipe	新一代大规模训练探索

4.1 FP32

FP32 最稳定，但显存和带宽开销大。

现代大模型训练通常不会全程 FP32 训练，但 optimizer state、某些归一化、梯度累积或 master weights 可能仍使用 FP32。

4.2 FP16

FP16 的优点是显存省、速度快。

问题是动态范围较小，容易 overflow 或 underflow。

因此 FP16 训练常需要 loss scaling。

4.3 BF16

BF16 也是 2 bytes，但指数位更多，动态范围接近 FP32。

它通常比 FP16 更适合大模型训练，因为不容易溢出。

代价是尾数精度更低，但在大模型训练中通常可以接受。

4.4 FP8

FP8 进一步降低显存和带宽，但训练 recipe 更复杂，需要硬件支持、scale 管理和更谨慎的数值稳定性验证。

面试中可以说：FP8 是重要方向，但不能简单认为把 dtype 改成 FP8 就能稳定训练。

精度选择可以先从两个维度理解：

$$M_{\{\mathrm{tensor}\}} = n_{\{\mathrm{elem}\}} b_{\{\mathrm{dtype}\}}$$

b_{dtype} 越小，显存和带宽越省；但动态范围和精度会影响 overflow、underflow 和舍入误差。FP16 和 BF16 都是 2 bytes，但 BF16 的指数范围更接近 FP32，因此大模型训练中通常更稳。

5. Mixed precision training

混合精度训练不是所有计算都用低精度，而是在不同部分使用不同精度。

典型做法：

forward/backward: FP16 或 BF16

optimizer state: FP32

部分归一化/累积: FP32 或更稳定实现

这样既能降低显存和提升吞吐，又能保留关键计算的稳定性。

5.1 Loss scaling

FP16 中小梯度可能 underflow 成 0。

loss scaling 的做法是先把 loss 放大，再 backward，更新前再把梯度缩回来。

$\tilde{L} = sL$

$\tilde{g} = \nabla_{\theta} \tilde{L} = s \nabla_{\theta} L$

$g = \frac{\tilde{g}}{s}$

BF16 因为动态范围更大，通常不需要像 FP16 那样依赖 loss scaling。

5.2 常见数值风险

1. loss 变 NaN。
2. gradient norm 变 Inf。
3. attention logits 过大。
4. softmax 溢出。
5. optimizer state 出现异常值。

这些问题经常和学习率、warmup、初始化、数据异常、精度配置一起出现。

6. Activation checkpointing

Activation checkpointing 的核心是用计算换显存。

普通训练：

forward 保存大量 activation

backward 直接使用保存的 activation

checkpointing：

forward 只保存部分边界 activation

backward 时重新计算中间 activation

优点：

1. 显著降低激活显存。
2. 支持更长 sequence length。
3. 支持更大 batch size。

代价：

1. 训练变慢。
2. 计算量增加。
3. checkpoint 粒度需要调。

面试表达：activation checkpointing 不是减少总计算，而是牺牲计算时间来降低激活显存。

如果原始激活显存是 M_{act} ，按 K 段做 checkpoint，可以用下面的粗略直觉理解：

$$M_{\{\mathrm{act, ckpt}\}} \approx \frac{M_{\{\mathrm{act}\}}}{K}$$

但 backward 需要重算部分 forward, 所以计算量会增加:

$$C_{\{\mathrm{ckpt}\}} > C_{\{\mathrm{base}\}}$$

真实比例取决于 checkpoint 粒度和模型结构。

7. Gradient accumulation 与显存

Gradient accumulation 不能降低单个 micro-batch 的激活显存, 但可以在不增加单卡 batch 的情况下扩大 effective batch size。

$$B_{\{\mathrm{global}\}} =$$

$$B_{\{\mathrm{micro}\}}$$

$$\cdot$$

$$A$$

$$\cdot$$

$$N_{\{\mathrm{dp}\}}$$

token batch 口径是:

$$B_{\{\mathrm{tok}\}} =$$

$$B_{\{\mathrm{micro}\}}$$

$$\cdot$$

$$T_{\{\mathrm{seq}\}}$$

$$\cdot$$

$$A$$

$$\cdot$$

$$N_{\{\mathrm{dp}\}}$$

如果 OOM 来自 batch 太大, 可以减小 micro-batch, 并增加 accumulation steps 保持 global batch 不变。

注意:

1. accumulation 会增加每次 optimizer step 前的 forward/backward 次数。
2. loss 要除以 accumulation steps。
3. logging 时要区分 micro step 和 optimizer step。

8. Optimizer state sharding

AdamW 的 optimizer state 很占显存, 所以 sharding 很重要。

ZeRO 和 FSDP 都可以切分 optimizer state。

例如 ZeRO-1 只切 optimizer state, 就能明显降低 AdamW 显存压力。

ZeRO-2 继续切 gradient。

ZeRO-3 / FSDP 进一步切 parameter。

若数据并行规模是 N_{dp} , ZeRO-3 / FSDP 的训练状态显存可以粗略写成:

$$M_{\{\mathrm{state, shard}\}} \approx$$

$$\frac{M_{\{\mathrm{state}\}}}{N_{\{\mathrm{dp}\}}}$$

这不直接减少 activation 显存, 所以如果 OOM 发生在 forward/backward activation 峰值, 仍然需要 checkpointing、减小 micro-batch 或缩短 sequence length。

面试表达: optimizer state sharding 主要解决训练状态显存, 不直接减少每个 micro-batch 的激活显存。

9. 参数高效微调的显存优势

如果不是从零预训练，而是微调已有模型，可以使用 LoRA、QLoRA 等参数高效方法。

LoRA 的思想是冻结原模型，只训练低秩增量矩阵。

对于一个 $d_{in} \times d_{out}$ 的线性层，LoRA rank 为 r 时，新增可训练参数量是：

$$P_{\{\mathrm{LoRA}\}} = r(d_{\{\mathrm{in}\}} + d_{\{\mathrm{out}\}})$$

相比全量矩阵参数：

$$P_{\{\mathrm{full}\}} = d_{\{\mathrm{in}\}} d_{\{\mathrm{out}\}}$$

当 r 远小于 d_{in} 和 d_{out} 时，optimizer state 和梯度显存会大幅下降。

显存优势：

1. 不需要保存全量参数梯度。
2. optimizer state 只针对 LoRA 参数。
3. 训练状态显存大幅下降。

但注意：forward/backward 仍然要经过原模型，激活显存仍然存在。长上下文和大 batch 下仍可能 OOM。

10. FlashAttention 与 attention 显存

标准 attention 需要显式构造或保存较大的 attention matrix，显存和序列长度关系很强。

FlashAttention 的核心价值是通过 IO-aware 的 kernel 设计，减少显存读写和中间矩阵保存。

训练视角下，它可以：

1. 降低 attention 中间显存。
2. 提高训练速度。
3. 支持更长序列。

但它不改变 attention 理论复杂度，也不能解决所有长上下文问题。

标准 attention 显式保存 attention matrix 时，显存近似包含：

$$M_{\{\mathrm{score}\}} \propto B_{\{\mathrm{micro}\}} H T_{\{\mathrm{seq}\}}^2$$

FlashAttention 的思路是分块计算 softmax attention，避免把完整 $T_{seq} \times T_{seq}$ score matrix 常驻到高带宽显存里。它优化的是 IO 和中间存储，不是把 full attention 的数学依赖关系改成线性注意力。

面试表达：FlashAttention 主要优化 attention kernel 的显存访问和中间存储，而不是把 full attention 变成线性复杂度。

11. OOM 排查顺序

遇到 OOM，不要盲目换机器。

可以按下面顺序排查：

1. 确认 OOM 发生在 forward、backward 还是 optimizer step。
2. 降低 micro-batch size。
3. 降低 sequence length。
4. 打开 activation checkpointing。
5. 使用 BF16 / mixed precision。
6. 使用 FlashAttention。

7. 使用 ZeRO/FSDP 切 optimizer state、gradient、parameter。
8. 检查是否有 tensor 被日志或 list 持有导致显存泄漏。
9. 检查 checkpoint、evaluation 是否额外占显存。

不同阶段 OOM 暗示不同原因：

OOM 位置	可能原因
forward	batch/seq 太大, activation 太多
backward	activation 和 gradient 同时占用, checkpointing 不足
optimizer step	optimizer state 太大
evaluation	没有 no_grad, 缓存了 logits 或 hidden states

可以把 OOM 排查写成一个简单门禁：

```
G_{\mathrm{memory}}=
g_{\mathrm{state}}\,
g_{\mathrm{activation}}\,
g_{\mathrm{attention}}\,
g_{\mathrm{buffer}}\,
g_{\mathrm{eval}}
```

这些 g 分别检查训练状态、activation、attention 中间结果、临时 buffer 和 evaluation 缓存是否在预算内。先定位是哪一类超预算，再选 ZeRO/FSDP、checkpointing、FlashAttention、减小 micro-batch 或修复 eval 缓存。

12. NaN / Inf 与精度问题

NaN 和 Inf 通常说明数值已经失控。

常见原因：

1. 学习率过大。
2. warmup 太短。
3. FP16 overflow。
4. loss scaling 不合适。
5. attention logits 太大。
6. 输入数据异常。
7. 梯度爆炸。

排查顺序：

1. 找到第一次出现 NaN 的 step。
2. 检查该 step 的 batch 是否异常。
3. 检查 loss、grad norm、lr。
4. 检查 mixed precision 配置。
5. 降低 lr 或延长 warmup。
6. 开启或调小 gradient clipping。
7. 从上一个健康 checkpoint 恢复。

面试表达：NaN 不是单一问题，要从数据、学习率、梯度、精度和模型实现一起排查。

一个数值健康门禁可以写成：

```
G_{\mathrm{numeric}}=
g_{\mathrm{loss}}\,
g_{\mathrm{grad}}\,
g_{\mathrm{lr}}\,
g_{\mathrm{scale}}\,
g_{\mathrm{data}}
```

这些检查分别表示 loss 有限、梯度有界、学习率和 warmup 合理、loss scale 或 dtype 没有触发 overflow / underflow、当前 batch 没有异常。

13. 显存优化策略对比

方法	主要节省	代价
BF16/FP16	参数、激活、梯度显存	数值稳定性风险
activation checkpointing	激活显存	增加计算时间
gradient accumulation	保持 global batch, 降低 micro-batch	step 时间变长
ZeRO/FSDP	optimizer、gradient、parameter	通信增加
FlashAttention	attention 中间显存和 IO	依赖 kernel 支持
LoRA/QLoRA	微调训练状态显存	不适合从零预训练
sequence parallel	长序列激活显存	通信和实现复杂

选择策略时要先判断瓶颈在哪里。

如果是 optimizer state 太大，用 ZeRO/FSDP。若是 activation 太大，用 checkpointing、减小 micro-batch 或 sequence length。若是 attention 中间结果大，用 FlashAttention。若是微调显存不够，用 LoRA/QLoRA。

下面这个 demo 用标准库做一个显存预算审计。它用 7B 参数、BF16 权重、AdamW FP32 状态、4096 序列长度和 80GB GPU 作为例子，比较全量训练、ZeRO-3、activation checkpointing、FlashAttention 和 LoRA 状态显存，并检查 FP16 / BF16 动态范围与 loss scaling 直觉。

GIB = 1024**3

```
cfg = {
    "params": 7_000_000_000,
    "weight_bytes": 2,
    "grad_bytes": 2,
    "adam_state_bytes": 8,
    "master_weight_bytes": 4,
    "micro_batch": 1,
    "seq_len": 4096,
    "hidden_size": 4096,
    "layers": 32,
    "activation_factor": 6,
    "attention_heads": 32,
    "gpu_gib": 80,
    "dp": 8,
    "checkpoint_segments": 4,
    "buffer_gib": 4.0,
    "flash_workspace_gib": 0.25,
    "lora_rank": 16,
    "lora_in": 4096,
    "lora_out": 11008,
}
```

```
def gib(bytes_value):
    return bytes_value / GIB
```

```
def state_memory_gib(c):
    p = c["params"]
    return {
        "weights": gib(p * c["weight_bytes"]),
```

```

"grads": gib(p * c["grad_bytes"]),
"adam_states": gib(p * c["adam_state_bytes"]),
"master_weights": gib(p * c["master_weight_bytes"]),
"total": gib(
    p
    * (
        c["weight_bytes"]
        + c["grad_bytes"]
        + c["adam_state_bytes"]
        + c["master_weight_bytes"]
    )
),
}

```

```

def activation_gib(c):
    return gib(
        c["micro_batch"]
        * c["seq_len"]
        * c["hidden_size"]
        * c["layers"]
        * c["weight_bytes"]
        * c["activation_factor"]
    )

```

```

def attention_matrix_gib(c):
    return gib(
        c["micro_batch"]
        * c["attention_heads"]
        * c["seq_len"]
        * c["seq_len"]
        * c["weight_bytes"]
    )

```

```

state = state_memory_gib(cfg)
act = activation_gib(cfg)
attn = attention_matrix_gib(cfg)
full_total = state["total"] + act + attn + cfg["buffer_gib"]
zero3_state = state["total"] / cfg["dp"]
ckpt_act = act / cfg["checkpoint_segments"]
optimized_total = zero3_state + ckpt_act + cfg["flash_workspace_gib"] + cfg["buffer_gib"]

fp16 = {"max": 65504.0, "min_normal": 2**-14}
bf16 = {"max": 3.389e38, "min_normal": 2**-126}
small_grad = 2e-8
loss_scale = 4096
scaled_grad = small_grad * loss_scale
loss_scaling = {
    "small_grad_underflows_fp16_normal": small_grad < fp16["min_normal"],
    "scaled_grad_reaches_fp16_normal": scaled_grad > fp16["min_normal"],
    "bf16_keeps_range": small_grad > bf16["min_normal"],
}

```

```

lora_trainable = cfg["lora_rank"] * (cfg["lora_in"] + cfg["lora_out"])
lora_state_gib = gib(
    lora_trainable
    * (
        cfg["weight_bytes"]
        + cfg["grad_bytes"]
        + cfg["adam_state_bytes"]
        + cfg["master_weight_bytes"]
    )
)
base_weight_gib = state["weights"]
lora_total = base_weight_gib + lora_state_gib + ckpt_act + cfg["flash_workspace_gib"] + cfg["buffer_gib"]

checks = {
    "full_training_oom": full_total > cfg["gpu_gib"],
    "zero3_checkpoint_flash_fits": optimized_total < cfg["gpu_gib"],
    "checkpoint_saves_activation": ckpt_act < act,
    "flash_saves_attention_matrix": cfg["flash_workspace_gib"] < attn,
    "bf16_range_larger_than_fp16": bf16["max"] > fp16["max"] * 1e20,
    "loss_scaling_helps_fp16": (
        loss_scaling["small_grad_underflows_fp16_normal"]
        and loss_scaling["scaled_grad_reaches_fp16_normal"]
    ),
    "lora_state_tiny": lora_state_gib < 0.01,
}

print("state_gib=", {k: round(v, 2) for k, v in state.items()})
print("activation_gib=", round(act, 2))
print("attention_matrix_gib=", round(attn, 2))
print("full_total_gib=", round(full_total, 2))
print("zero3_state_gib=", round(zero3_state, 2))
print("checkpointed_activation_gib=", round(ckpt_act, 2))
print("optimized_total_gib=", round(optimized_total, 2))
print("fp16=", {k: f"{v:.2e}" for k, v in fp16.items()})
print("bf16=", {k: f"{v:.2e}" for k, v in bf16.items()})
print("loss_scaling=", loss_scaling)
print("lora_trainable_params=", lora_trainable)
print("lora_state_gib=", round(lora_state_gib, 6))
print("lora_total_gib=", round(lora_total, 2))
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

运行后应看到:

```

state_gib= {'weights': 13.04, 'grads': 13.04, 'adam_states': 52.15, 'master_weights': 26.08, 'total': 104.31}
activation_gib= 6.0
attention_matrix_gib= 1.0
full_total_gib= 115.31
zero3_state_gib= 13.04
checkpointed_activation_gib= 1.5
optimized_total_gib= 18.79
fp16= {'max': '6.55e+04', 'min_normal': '6.10e-05'}
bf16= {'max': '3.39e+38', 'min_normal': '1.18e-38'}
loss_scaling= {'small_grad_underflows_fp16_normal': True, 'scaled_grad_reaches_fp16_normal': True, 'bf16_keeps_range': True}
lora_trainable_params= 241664
lora_state_gib= 0.003601

```

```
lora_total_gib= 18.79
```

```
checks= {'full_training_oom': True, 'zero3_checkpoint_flash_fits': True, 'checkpoint_saves_activation': True, 'flash_sage_gate_pass'= True
```

这个 demo 的重点不是给出真实训练的精确显存，而是建立排查习惯：先把训练状态、激活、attention 中间结果和 buffer 分开估算，再判断应该用 ZeRO/FSDP、activation checkpointing、FlashAttention、减小 micro-batch，还是换成参数高效微调。

14. 面试官会怎么问

问法 1：为什么 AdamW 训练显存这么大？

可以这样答：

因为训练时不只保存模型权重，还要保存梯度和 optimizer state。AdamW 对每个参数维护一阶动量和二阶动量，通常是 FP32，所以

问法 2：FP16 和 BF16 有什么区别？

可以这样答：

FP16 和 BF16 都是 2 bytes，但 BF16 指数位更多，动态范围接近 FP32，因此大模型训练中更不容易 overflow。FP16 尾数精度更高一

问法 3：activation checkpointing 解决什么问题？

可以这样答：

它解决激活显存过大的问题。forward 时不保存所有中间 activation，只保存部分 checkpoint，backward 时重新计算中间 activation

问法 4：OOM 你怎么排查？

可以这样答：

我会先判断 OOM 发生在 forward、backward、optimizer step 还是 evaluation。forward OOM 多半是 batch、sequence 或 activation batch、开 mixed precision、FlashAttention、checkpointing、sharding 的顺序处理。

问法 5：NaN 怎么排查？

可以这样答：

先定位第一次出现 NaN 的 step，再检查该 batch 数据、loss、grad norm、learning rate、mixed precision 和 optimizer state。常

15. 本章小结

本章核心结论：

1. 训练显存由参数、梯度、optimizer state、激活和临时 buffer 共同组成。
2. AdamW 的 m/v 状态会显著增加显存占用。
3. 激活显存和 batch size、sequence length、hidden size、layers 强相关。
4. BF16 通常比 FP16 更适合大模型训练稳定性。
5. Mixed precision 需要在显存、吞吐和数值稳定之间平衡。
6. Activation checkpointing 用重计算换激活显存。
7. ZeRO/FSDP 主要通过 sharding 降低训练状态显存。
8. FlashAttention 降低 attention 中间显存和 IO 开销。
9. OOM 要按发生阶段定位，不要盲目调参。
10. NaN/Inf 要从数据、学习率、梯度、精度和实现一起排查。

第九章：训练稳定性与 Debug

本章目标

掌握大模型训练中常见异常的定位方法。

核心议题

1. loss spike。
2. NaN 和 Inf。
3. 梯度爆炸和梯度消失。
4. 数据异常导致训练异常。
5. 学习率过大或 warmup 不足。
6. 混合精度溢出。
7. 分布式同步错误。

Debug 顺序

1. 检查数据 batch。
2. 检查 loss mask。
3. 检查学习率和 optimizer。
4. 检查梯度 norm。
5. 检查数值精度。
6. 检查分布式通信和 checkpoint 恢复。

面试重点

训练 debug 题要结构化回答，不要只说“调学习率”。

为什么训练 debug 很难

大模型训练的问题通常不是单点问题。

同一个现象可能来自不同层：

loss spike

- 可能是坏数据
- 可能是学习率过大
- 可能是混合精度溢出
- 可能是分布式同步异常
- 可能是 checkpoint 恢复错误

所以训练 debug 的关键不是背几个修复技巧，而是建立排查顺序。

一个可靠原则是：

先定位问题发生在哪一层，再决定怎么修。

训练系统可以分成这些层：

1. 数据层。
2. Tokenization 和 sample 构造层。
3. Loss mask 层。
4. 模型 forward/backward 层。
5. 优化器和学习率层。
6. 数值精度层。
7. 分布式通信层。
8. checkpoint 和恢复层。

面试表达：训练 debug 要从数据、loss、梯度、优化器、精度、分布式和恢复状态逐层排查，而不是直接调学习率。

训练稳定性 Debug 的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 PyTorch autograd anomaly detection、clip_grad_norm 的 non-finite 检查、PyTorch AMP / GradScaler、Hugging Face Trainer 日志指标，以及前序训练循环、优化器、分布式训练、显存和数值精度章节资料。

本章聚焦训练稳定性排查的工程闭环：loss spike、NaN / Inf、grad norm、数据 batch、label mask、学习率、optimizer state、mixed precision、分布式 rank 状态和 checkpoint 恢复。它不展开特定训练平台的监控系统、生产告警规则、NCCL 深层故障诊断和模型架构级稳定性理论。

训练健康指标口径

每个训练 step 可以记录一组健康指标：

$h_t = (L_t, L_{\mathrm{val},t}, G_t, \eta_t, S_t, R_t, U_t)$

其中 L_t 是 train loss, $L_{\mathrm{val},t}$ 是 validation loss, G_t 是 grad norm, η_t 是 learning rate, S_t 是 loss scale, R_t 是 tokens/sec, U_t 是 GPU utilization 或类似系统利用率。debug 的目标是从时间序列 h_t 中找到第一次异常：

$t^* = \min\{t \mid G_{\mathrm{health}}(h_t) = 0\}$

一个简化健康门禁可以写成：

$G_{\mathrm{health}} =$
 $g_{\mathrm{loss}} \setminus,$
 $g_{\mathrm{grad}} \setminus,$
 $g_{\mathrm{data}} \setminus,$
 $g_{\mathrm{mask}} \setminus,$
 $g_{\mathrm{lr}} \setminus,$
 $g_{\mathrm{precision}} \setminus,$
 $g_{\mathrm{rank}} \setminus,$
 g_{ckpt}

这些 g 都是 0/1 检查项：loss 有限且无异常 spike、梯度范数可控、batch 数据正常、label mask 正确、学习率和 warmup 符合预期、精度没有 overflow / underflow、分布式 rank 状态一致、checkpoint 恢复字段完整。

1. 先建立健康训练的基线

Debug 的前提是知道 “正常训练” 应该长什么样。

一个健康训练过程通常有这些特征：

1. train loss 总体下降，允许小幅波动。
2. validation loss 同步下降或稳定改善。
3. learning rate 按 schedule 正常 warmup 和 decay。
4. grad norm 在合理范围内波动。
5. 没有持续 NaN / Inf。
6. tokens/sec 相对稳定。
7. GPU utilization 没有长时间掉到很低。
8. checkpoint 能保存并恢复。

如果你没有这些基线指标，出现问题时就很难判断异常来自哪里。

2. Debug 总顺序

推荐的排查顺序：

1. 复现问题：确认是偶发还是稳定复现。
2. 定位时间：找到第一次异常的 step。
3. 定位层级：数据、loss、梯度、优化器、精度、分布式。
4. 缩小规模：用小模型、小 batch、单卡复现。
5. 对比基线：和上一个健康配置或 checkpoint 对比。
6. 单变量实验：一次只改一个因素。
7. 修复后回归：确认旧问题消失且没有引入新问题。

不要一上来同时改学习率、batch size、数据、精度和并行策略。这样即使训练好了，也不知道是哪一个因素起作用。

3. Loss spike

Loss spike 是大模型训练中非常常见的异常。

表现：

loss 突然升高
grad norm 同步变大
有时会恢复，有时进入 NaN

3.1 可能原因

原因	说明
坏数据 batch	极长、乱码、异常 token、错误 label
学习率过大	更新步子太大，参数跳到不稳定区域
warmup 太短	训练初期还不稳定就给了高 lr
梯度爆炸	grad norm 突然变大
FP16 overflow	数值溢出导致 loss 异常
checkpoint 恢复不完整	optimizer/scheduler 状态错位

3.2 排查方法

1. 找到 spike 的 step。
2. 保存该 step 的 batch id 或样本。
3. 检查该 batch 的长度、token 分布、特殊 token、label mask。
4. 查看 grad norm 是否同步升高。
5. 查看 learning rate 是否刚到 peak。
6. 检查是否从 checkpoint 恢复后不久发生。

一个实用的 spike 判定不要只看单点 loss，而要和前面一小段窗口比较。设最近 k 个健康 step 的 loss 中位数为基线：

$$r_t = \frac{L_t}{\{\mathrm{median}(L_{t-k}, \dots, L_{t-1}) + \epsilon\}}$$

当下面条件成立时，可以先把该 step 标成候选异常点：

$$r_t \geq \tau_{\{\mathrm{spike}\}}$$

其中 τ_{spike} 是 spike 阈值，工程上常用 2 到 3 倍作为初筛，再结合 batch、grad norm、lr 和 loss scale 复核。这个公式不是为了替代人工分析，而是为了让告警和日志定位有稳定口径。

3.3 处理方法

1. 过滤异常数据。
2. 降低 peak learning rate。
3. 延长 warmup。
4. 开启或调小 gradient clipping。
5. 使用 BF16 替代 FP16。
6. 从 spike 前的健康 checkpoint 恢复。

面试表达：loss spike 要先判断是数据驱动、优化器驱动还是数值驱动，不能直接假设是学习率问题。

4. NaN 和 Inf

NaN / Inf 说明数值已经失控。

常见现象：

```
loss = nan
grad_norm = inf
某些 parameter norm 变成 nan
optimizer state 出现 inf
```

4.1 常见原因

1. FP16 overflow。
2. 学习率过大。
3. warmup 太短。
4. 梯度爆炸。
5. attention logits 过大。
6. 输入中有异常值。
7. 自定义 loss 中除以 0。
8. 分布式 reduce 前某个 rank 已经 NaN。

4.2 排查顺序

1. 找第一次出现 NaN 的 step。
2. 检查前一个 step 的 loss、grad norm、lr。
3. 检查 batch 是否异常。
4. 检查 mixed precision 和 loss scaling。
5. 检查模型输出 logits 是否过大。
6. 检查是所有 rank NaN，还是某个 rank 先 NaN。

有限性检查可以写成：

```
g_{\mathrm{finite}}=
\mathbf{1}[\mathrm{isfinite}(L_t)]
\mathbf{1}[\mathrm{isfinite}(G_t)]
```

如果 $g_{\text{finite}}=0$ ，就不要继续让 optimizer 更新参数，而应该保存现场、定位第一个坏张量，并回滚到上一个健康 checkpoint。PyTorch 里常见做法是对关键张量显式检查 finite，必要时打开 autograd anomaly detection；梯度裁剪接口也可以要求遇到 non-finite total norm 时直接报错。

4.3 处理方法

1. 使用 BF16。
2. 降低 learning rate。
3. 延长 warmup。
4. 开启 gradient clipping。
5. 跳过异常 batch。

6. 回滚到健康 checkpoint。
7. 对关键张量加 NaN/Inf 检查。

面试表达：NaN 的核心是定位第一个坏掉的 step 和第一个坏掉的张量。

5. 梯度爆炸和梯度消失

5.1 梯度爆炸

表现：

grad norm 突然变大

loss spike

参数范数异常

可能进入 NaN

常见原因：

1. learning rate 太大。
2. warmup 不足。
3. 初始化不稳定。
4. loss mask 错误导致异常 loss。
5. 数据 batch 异常。

常见处理：

1. gradient clipping。
2. 降低 lr。
3. 延长 warmup。
4. 检查异常 batch。
5. 检查 loss 是否平均方式错误。

全局梯度范数通常按所有可训练参数的梯度拼接后计算：

$$G_t = \sqrt{\sum_i \|g_{t,i}\|^2}$$

如果最大允许范数为 c，global norm clipping 可以抽象成：

$$g'_t = \min\left(1, \frac{c}{G_t + \epsilon}\right) g_t$$

这说明 clipping 只在 $G_t > c$ 时缩小梯度方向的长度，不会在正常梯度范围内改变更新。面试里要能说清楚：clipping 是稳定训练的保护机制，不是修复坏数据、错误 label mask 或过大学习率的根本替代品。

5.2 梯度消失或过小

表现：

loss 几乎不下降

grad norm 很小

参数更新很小

可能原因：

1. learning rate 太小。
2. loss mask 把大部分 token 忽略了。
3. 模型某些参数没有参与计算。
4. optimizer 参数分组漏了参数。
5. mixed precision underflow。

面试表达：梯度异常要结合 grad norm、参数更新幅度、loss mask 和 optimizer 参数分组一起看。

6. 数据异常导致训练异常

数据问题是训练异常的高频根因。

常见数据异常：

1. 空样本。
2. 极长样本。
3. 乱码。
4. 错误编码。
5. 特殊 token 缺失。
6. label 全部为 ignore。
7. prompt 和 answer 错位。
8. 重复样本比例过高。
9. benchmark 或测试集泄漏。

对 SFT 或指令微调，异常 batch 的一个关键指标是有效 label 比例：

$$q_{\{\mathrm{label}\}} = \frac{\sum_{j=1}^T \mathbf{1}[y_j \neq -100]}{T}$$

如果 q_{label} 接近 0，说明这一批几乎没有监督 token；如果 prompt token 也参与了 loss，则模型可能在学复读 prompt。稳定训练不只要求 batch shape 正确，还要求有效监督区域正确。

6.1 怎么检查 batch

建议打印或保存异常 step 的：

sample id
source
input length
input_ids 前后若干 token
decoded text
labels 中有效 token 数
attention_mask sum
loss mask span

很多问题一 decode 就能看出来，例如回答被截断、角色 token 错、全是 padding、文本乱码。

6.2 数据 bug 的典型现象

现象	可能数据问题
loss 不下降	label 错位、有效 label 太少
loss 很低但模型很差	数据重复、泄漏、任务太简单
loss spike	异常 batch、极端长度或坏样本
模型学会复读 prompt	SFT label mask 错误
模型输出格式混乱	chat template 不一致

7. Loss mask 和 label 错误

Loss mask 错误非常隐蔽。

训练脚本可能正常运行，loss 也会下降，但模型行为不对。

常见错误：

1. user token 参与 SFT loss。
2. assistant token 没有参与 loss。
3. padding token 参与 loss。

4. 多轮对话只训练最后一轮。
5. label 没有 shift。
6. 截断后只剩 prompt, 没有 answer。

loss mask 的门禁可以简化为:

$$g_{\{\mathrm{mask}\}} = \mathbf{1}[q_{\{\mathrm{label}\}} \geq q_{\{\min\}}] \mathbf{1}[\Delta_{\{\mathrm{shift}\}} = 1] \mathbf{1}[\Delta_{\{\mathrm{pad}\}} = 1]$$

其中 $\Delta_{\mathrm{shift}}=1$ 表示 causal LM 的 input / label shift 正确, $\Delta_{\mathrm{pad}}=1$ 表示 padding token 没有参与 loss。这个门禁不追求覆盖所有格式错误, 但能拦住最常见、最隐蔽的训练目标错误。

检查方式:

```
decode input_ids
decode labels != -100 的部分
确认模型到底在学什么
```

面试表达: 训练 debug 时我一定会 decode 一批 input 和 label mask, 因为很多格式问题只看 tensor shape 看不出来。

8. 学习率和 optimizer 问题

学习率相关问题非常常见。

8.1 学习率过大

表现:

1. loss spike。
2. grad norm 爆炸。
3. NaN。
4. 训练初期不稳定。

处理:

1. 降低 peak lr。
2. 延长 warmup。
3. 加强 gradient clipping。

8.2 学习率过小

表现:

1. loss 下降很慢。
2. 参数更新幅度很小。
3. validation 指标长期不动。

处理:

1. 增大学习率。
2. 检查 scheduler 是否配置错误。
3. 检查 optimizer 是否包含所有可训练参数。

8.3 optimizer state 恢复错误

如果从 checkpoint 恢复时只加载模型权重, 没有加载 optimizer 和 scheduler, 训练可能突然不稳定。

特别是 AdamW 的动量状态和学习率 schedule, 如果不连续, loss 曲线可能跳变。

9. 混合精度溢出

FP16 训练容易遇到 overflow 或 underflow。

表现：

1. loss scale 频繁下降。
2. grad norm 变 Inf。
3. loss 变 NaN。

处理：

1. 改用 BF16。
2. 调整 loss scaling。
3. 降低 lr。
4. 对高风险操作使用 FP32。
5. 检查 softmax、norm 和自定义 kernel。

loss scaling 的核心关系是：

$$\begin{aligned}\tilde{L}_t &= S_{tL_t} \\ \tilde{g}_t &= S_{tg_t} \\ g_t &= \frac{\tilde{g}_t}{S_t}\end{aligned}$$

如果缩放后的梯度出现 Inf / NaN，scaler 通常会跳过本次 optimizer step 并调整 scale。调试时要记录 S_t 是否频繁下降，以及跳过更新是否和 loss spike、坏 batch 或 rank 异常发生在同一段时间。

面试表达：混合精度问题不只是 dtype 选择，也和 loss scaling、kernel 实现和梯度尺度有关。

10. 分布式同步错误

分布式训练中的 bug 通常更难定位。

常见现象：

1. 训练 hang 住。
2. 某些 rank loss 不一致。
3. 某些 rank OOM。
4. all-reduce 报错。
5. checkpoint 恢复后 rank 状态不一致。

10.1 常见原因

1. 某个 rank 数据长度不同。
2. collective 调用顺序不一致。
3. 某个 rank 先 NaN 或报错。
4. gradient accumulation 边界不一致。
5. dataloader 没有正确 shard。
6. 随机种子或 dropout 状态不一致。

10.2 排查方法

1. 先单卡复现。
2. 再单机多卡复现。
3. 最后多机复现。
4. 打印每个 rank 的 step、loss、batch size。
5. 确认所有 rank collective 调用一致。

每个 rank 的 loss 至少要做一致性观察：

$$\Delta_{\mathrm{rank}} = \max_r L_{\mathrm{t},r} - \min_r L_{\mathrm{t},r}$$

对应的分布式健康门禁可以写成：

$$g_{\mathrm{rank}} = \mathbf{1}[\Delta_{\mathrm{rank}} \leq \tau_{\mathrm{rank}}] \\ \prod_r \mathbf{1}[\mathrm{isfinite}(L_{\mathrm{t},r})]$$

如果某个 rank 先变 NaN，其他 rank 可能只是被 collective 卡住。排查时要看 per-rank loss、batch id、有效 token 数和 collective 调用位置，而不是只看主进程日志。

面试表达：分布式 debug 要先缩小到单卡和单机多卡，确认问题是模型/数据问题还是通信/并行问题。

11. Checkpoint 恢复问题

Checkpoint 恢复错误很容易导致训练曲线异常。

完整恢复通常需要：

1. model weights。
2. optimizer state。
3. scheduler state。
4. scaler state。
5. random state。
6. dataloader position。
7. distributed state。
8. tokenizer 和 config。

常见错误：

1. 只恢复模型，不恢复 optimizer。
2. scheduler step 从 0 重新开始。
3. tokenizer 版本不一致。
4. FSDP/ZeRO checkpoint 加载方式不匹配。
5. 数据继续位置错乱。

完整 checkpoint 的恢复门禁可以写成：

$$G_{\mathrm{ckpt}} = g_{\mathrm{model}} \setminus, \\ g_{\mathrm{opt}} \setminus, \\ g_{\mathrm{sched}} \setminus, \\ g_{\mathrm{scaler}} \setminus, \\ g_{\mathrm{rng}} \setminus, \\ g_{\mathrm{data}}$$

只要其中一项为 0，就只能说“加载了部分状态”，不能说“恢复了训练”。对训练恢复来说，optimizer 动量、scheduler step、loss scaler、随机状态和 dataloader 位置都可能影响接下来的 loss 曲线。

面试表达：checkpoint 能用于推理不代表能无缝恢复训练。恢复训练需要 optimizer、scheduler、scaler、随机状态和数据位置一致。

12. 最小可运行训练健康审计 demo

下面的 demo 用标准库模拟一次训练健康审计。输入是 step 级训练日志、batch 元信息、per-rank loss 和 checkpoint 字段；输出是 spike、NaN、梯度爆炸、坏 batch、rank 不一致、checkpoint 缺项和建议动作。

```
import math
from statistics import median
```



```

history = [
    {"step": 1, "loss": 2.40, "grad_norm": 0.8, "lr": 1e-5, "loss_scale": 4096, "tokens_sec": 980},
    {"step": 2, "loss": 2.18, "grad_norm": 0.9, "lr": 2e-5, "loss_scale": 4096, "tokens_sec": 1005},
    {"step": 3, "loss": 2.05, "grad_norm": 1.0, "lr": 3e-5, "loss_scale": 4096, "tokens_sec": 996},
    {"step": 4, "loss": 5.80, "grad_norm": 7.5, "lr": 4e-5, "loss_scale": 4096, "tokens_sec": 991},
    {"step": 5, "loss": 2.02, "grad_norm": 1.1, "lr": 5e-5, "loss_scale": 4096, "tokens_sec": 997},
    {"step": 6, "loss": float("nan"), "grad_norm": float("inf"), "lr": 6e-5, "loss_scale": 2048, "tokens_sec": 990},
]

batches = {
    1: {"id": "batch_001", "max_len": 2048, "valid_label_ratio": 0.82, "has_nan_token": False, "source": "web_clean"},
    2: {"id": "batch_002", "max_len": 2048, "valid_label_ratio": 0.80, "has_nan_token": False, "source": "web_clean"},
    3: {"id": "batch_003", "max_len": 2048, "valid_label_ratio": 0.79, "has_nan_token": False, "source": "code_clean"},
    4: {"id": "batch_004_bad_mask", "max_len": 8192, "valid_label_ratio": 0.03, "has_nan_token": False, "source": "st"},
    5: {"id": "batch_005", "max_len": 2048, "valid_label_ratio": 0.78, "has_nan_token": False, "source": "web_clean"},
    6: {"id": "batch_006_nan", "max_len": 2048, "valid_label_ratio": 0.81, "has_nan_token": True, "source": "math_syn"}
}

rank_losses = {4: [5.8, 5.7, 5.9, 5.8], 6: [2.1, float("nan"), 2.0, 2.1]}

checkpoint = {
    "model": True,
    "optimizer": True,
    "scheduler": False,
    "scaler": False,
    "random_state": True,
    "data_position": False,
}

seq_limit = 4096
valid_label_min = 0.2
grad_threshold = 5.0
spike_ratio = 2.0

def is_finite(x):
    return math.isfinite(x)

def finite_losses_before(step):
    return [r["loss"] for r in history if r["step"] < step and is_finite(r["loss"])]

spike_steps = []
nan_steps = []
grad_explosion_steps = []
bad_batches = []
rank_mismatch_steps = []

for record in history:
    step = record["step"]
    loss = record["loss"]
    grad_norm = record["grad_norm"]

    if not is_finite(loss) or not is_finite(grad_norm):
        nan_steps.append(step)

```

```

previous = finite_losses_before(step)
if previous and is_finite(loss):
    baseline = median(previous[-3:])
    if loss / baseline >= spike_ratio:
        spike_steps.append((step, round(loss / baseline, 2)))

if is_finite(grad_norm) and grad_norm > grad_threshold:
    grad_explosion_steps.append((step, grad_norm))

batch = batches[step]
reasons = []
if batch["max_len"] > seq_limit:
    reasons.append("too_long")
if batch["valid_label_ratio"] < valid_label_min:
    reasons.append("low_valid_label_ratio")
if batch["has_nan_token"]:
    reasons.append("nan_token")
if reasons:
    bad_batches.append((step, batch["id"], reasons))

for step, losses in rank_losses.items():
    finite = [x for x in losses if is_finite(x)]
    has_nonfinite = len(finite) != len(losses)
    spread = max(finite) - min(finite) if finite else float("inf")
    if has_nonfinite or spread > 0.5:
        rank_mismatch_steps.append((step, has_nonfinite, round(spread, 3) if is_finite(spread) else "inf"))

missing_checkpoint = [name for name, ok in checkpoint.items() if not ok]
healthy_steps = [
    r["step"]
    for r in history
    if r["step"] not in nan_steps and r["step"] not in [s for s, _ in spike_steps]
]
rollback_step = max(step for step in healthy_steps if step < min(nan_steps or [10**9]))

recommendations = []
if bad_batches:
    recommendations.append("quarantine_bad_batches")
if grad_explosion_steps:
    recommendations.append("lower_lr_or_enable_clipping")
if nan_steps:
    recommendations.append("switch_bf16_or_check_loss_scaling")
if missing_checkpoint:
    recommendations.append("resume_only_from_complete_checkpoint")
if rank_mismatch_steps:
    recommendations.append("compare_rank_batches_and_collectives")

checks = {
    "spike_detected": spike_steps == [(4, 2.66)],
    "nan_detected": nan_steps == [6],
    "bad_batch_detected": bad_batches[0][1] == "batch_004_bad_mask",
    "checkpoint_gap_detected": missing_checkpoint == ["scheduler", "scaler", "data_position"],
    "rank_mismatch_detected": rank_mismatch_steps == [(6, True, 0.1)],
    "rollback_before_nan": rollback_step == 5,
}

```

```

print("spike_steps=", spike_steps)
print("nan_steps=", nan_steps)
print("grad_explosion_steps=", grad_explosion_steps)
print("bad_batches=", bad_batches)
print("rank_mismatch_steps=", rank_mismatch_steps)
print("missing_checkpoint=", missing_checkpoint)
print("rollback_step=", rollback_step)
print("recommendations=", recommendations)
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

期望输出：

```

spike_steps= [(4, 2.66)]
nan_steps= [6]
grad_explosion_steps= [(4, 7.5)]
bad_batches= [(4, 'batch_004_bad_mask', ['too_long', 'low_valid_label_ratio']), (6, 'batch_006_nan', ['nan_token'])]
rank_mismatch_steps= [(6, True, 0.1)]
missing_checkpoint= ['scheduler', 'scaler', 'data_position']
rollback_step= 5
recommendations= ['quarantine_bad_batches', 'lower_lr_or_enable_clipping', 'switch_bf16_or_check_loss_scaling', 'resume_training']
checks= {'spike_detected': True, 'nan_detected': True, 'bad_batch_detected': True, 'checkpoint_gap_detected': True, 'rank_mismatch_detected': True, 'grad_explosion_detected': True, 'missing_checkpoint': True, 'optimizer_state_reset': True, 'gate_pass': True}
gate_pass= True

```

这个 demo 的重点不是模拟真实训练，而是把训练 debug 的日志口径固定下来：先定位 step，再关联 batch、grad、rank 和 checkpoint。真实项目里可以把这些检查接到 Trainer callback、训练日志、数据样本落盘和告警系统里。

13. 一个实用 debug checklist

遇到训练异常，可以按下面清单走：

1. 异常类型是什么：loss spike、NaN、OOM、hang、吞吐下降？
2. 第一次出现在哪个 step？
3. 是否能稳定复现？
4. 该 step 的 batch 是否异常？
5. input_ids / labels / mask decode 后是否正确？
6. lr、warmup、grad norm 是否异常？
7. mixed precision 是否 overflow？
8. 单卡是否复现？
9. 从上一个 checkpoint 恢复是否正常？
10. 最近改动了什么：数据、代码、超参、并行、精度？

这比“感觉是学习率问题”可靠得多。

14. 面试官会怎么问

问法 1：训练 loss 突然 spike，你怎么排查？

可以这样答：

我会先定位 spike 第一次出现的 step，然后看该 step 的 batch、loss、grad norm、learning rate 和 mixed precision 状态。先检查 batch 是否异常，再检查 loss 是否异常，再检查 grad norm 是否异常，再检查 learning rate 是否异常，最后检查 mixed precision 是否异常。

问法 2：NaN 怎么定位？

可以这样答：

NaN 要定位第一个坏掉的 step 和第一个坏掉的张量。我会检查 loss、logits、grad norm、参数范数和 optimizer state，确认是 fo

问法 3：为什么 train loss 下降但模型效果很差？

可以这样答：

可能是数据重复、评估污染、训练目标和评估目标不一致、label mask 错误或模型只学到了格式。我要看 validation loss、per-domain loss、去重和污染检测，也会 decode 样本检查模型到底在学哪些 token。不能只看 train loss 判断模型能力。

问法 4：分布式训练 hang 住怎么办？

可以这样答：

先看是否某个 rank 提前报错或 OOM，其他 rank 在 collective 通信处等待。然后检查所有 rank 的 step、batch size、dataloader 上

问法 5：恢复 checkpoint 后 loss 突然异常怎么办？

可以这样答：

我会检查是否完整恢复了 model、optimizer、scheduler、scaler、random state 和数据位置。如果只恢复模型权重，AdamW 动量和学

15. 本章小结

本章核心结论：

1. 训练 debug 要先定位层级，再修复症状。
2. Loss spike 可能来自数据、学习率、梯度、精度或恢复状态。
3. NaN/Inf 要定位第一个坏 step 和第一个坏张量。
4. 梯度异常要结合 grad norm、loss mask 和 optimizer 参数分组看。
5. 数据 batch 和 label mask 是最常见、也最容易被忽略的根因。
6. 学习率过大、warmup 不足和 gradient clipping 缺失会导致训练不稳定。
7. 混合精度问题要关注 FP16 overflow、loss scaling 和高风险 kernel。
8. 分布式问题要先缩小到单卡/单机，再排查通信和 rank 状态。
9. Checkpoint 恢复必须包含 optimizer、scheduler、scaler、随机状态和数据位置。
10. 面试中要用结构化 checklist 回答训练 debug，而不是只说“调参”。

第十章：Checkpoint、评估与实验管理

本章目标

理解如何保存、恢复、比较和选择模型 checkpoint。

核心议题

1. checkpoint 保存内容：model、optimizer、scheduler、rng state、训练进度。
2. checkpoint 频率和存储成本。
3. 训练恢复和容错。
4. validation loss、perplexity、下游 benchmark。
5. 实验追踪、版本管理和可复现性。

面试重点

大规模训练不是跑一个脚本，而是长期实验系统。评估和实验管理决定结果是否可信。

为什么 checkpoint 和实验管理很重要

大模型训练成本很高，一次训练可能持续数天、数周甚至更久。

如果没有可靠 checkpoint，任何机器故障、网络故障、代码 bug 或 NaN 都可能让训练前功尽弃。

如果没有实验管理，即使跑出一个好模型，也很难回答：

1. 这个模型是用哪批数据训练的。
2. 训练了多少 token。
3. 用了什么学习率和 batch size。
4. 哪个 checkpoint 最好。
5. 相比上一个版本到底提升在哪里。
6. 是否有能力退化或安全风险。

面试表达：训练工程不只是把 loss 跑下来，还要保证训练过程可恢复、结果可比较、结论可复现。

Checkpoint 与实验管理的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 PyTorch 保存/加载通用 checkpoint、PyTorch Distributed Checkpoint、Hugging Face Transformers Trainer checkpoint 与 best model 配置、perplexity 评估口径，以及实验追踪和 artifact versioning 的官方实践。

本章聚焦训练工程中最小可落地的闭环：checkpoint 保存内容、恢复连续性、存储保留策略、validation / benchmark / regression 评估、实验记录、版本管理和可复现性。它不展开生产级对象存储、跨机房灾备、完整 MLOps 平台、权限审计系统和所有评测框架 API。

实验对象与总门禁

一次训练实验可以抽象成：

$e = (c, d, m, h, a, r)$

其中 c 是代码版本， d 是数据版本， m 是模型和 tokenizer 配置， h 是训练超参和硬件环境， a 是 checkpoint artifact， r 是评估结果。一个实验能不能被采信，可以先用下面的总门禁判断：

$G_{\{\mathrm{exp}\}} = g_{\{\mathrm{ckpt}\}} \setminus, g_{\{\mathrm{eval}\}} \setminus, g_{\{\mathrm{repro}\}} \setminus, g_{\{\mathrm{compare}\}} \setminus, g_{\{\mathrm{safety}\}}$

这些检查分别对应：checkpoint 是否完整、评估是否充分、记录是否可复现、对比是否公平、安全和回归风险是否可接受。

1. Checkpoint 应该保存什么

一个完整训练 checkpoint 不只是模型权重。

通常至少包含：

内容	作用
model weights	模型参数
optimizer state	AdamW 的 m/v 等状态
scheduler state	learning rate 进度
scaler state	FP16 mixed precision 的 loss scale
RNG state	Python、NumPy、PyTorch、CUDA 随机状态
global step	当前训练步数
consumed tokens	已训练 token 数
data state	dataloader 或 shard 位置
model config	架构配置
tokenizer	tokenizer 文件和 special token

内容	作用
training config	batch、lr、并行、精度等配置

只保存 model weights 可以用于推理，但通常不能完整恢复训练。

如果 optimizer 或 scheduler 没恢复，继续训练时 loss 可能跳变。

恢复训练所需字段可以写成一个集合：

$$C_{\{\mathrm{resume}\}} = \{\text{model, opt, sched, scaler, rng, data, step, tok, config, tokenizer}\}$$

完整性门禁是：

$$G_{\{\mathrm{ckpt}\}} = \{\prod_{u \in C_{\{\mathrm{resume}\}}} g_u\}$$

其中 $g_u=1$ 表示字段 u 存在、版本匹配且能被当前训练脚本读取。只要 $G_{\text{ckpt}}=0$ ，这个 checkpoint 就只能算部分可用，不能直接宣称能无缝恢复训练。

2. 保存频率怎么定

Checkpoint 保存太少，故障时损失大；保存太频繁，会浪费存储和训练时间。

需要权衡：

1. 训练成本。
2. 故障概率。
3. checkpoint 大小。
4. 存储带宽。
5. 恢复时间。
6. 评估频率。

常见策略：

每 N steps 保存一次 latest checkpoint

每 M tokens 保存一个里程碑 checkpoint

保留最近 K 个 checkpoint

对关键 checkpoint 做长期归档

例如：

latest：用于故障恢复，只保留最近几个

milestone：用于评估和对比，按 token 数保存

best：根据评估指标选择，用于后训练或发布

如果每 f 个 step 保存一次，总训练步数为 S ，保存次数约为：

$$N_{\{\mathrm{save}\}} = \left\lceil \frac{S}{f} \right\rceil$$

一次故障最多会损失最近一次保存后的 token：

$$T_{\{\mathrm{lost, max}\}} = f B_{\{\mathrm{tok}\}}$$

其中 B_{tok} 是每 step 消耗的 token 数。保存开销占比可以粗略估成：

$$R_{\{\mathrm{save}\}} = \frac{T_{\{\mathrm{save}\}}}{f T_{\{\mathrm{step}\}}}$$

f 越小，容错越好，但 R_{save} 和存储压力越高。

面试表达：checkpoint 策略要同时考虑容错、评估和存储成本。

3. Checkpoint 存储成本

大模型 checkpoint 非常大。

如果是 70B 模型，单模型权重就可能上百 GB。若再保存 optimizer state，体积可能数倍增加。

因此要区分两类 checkpoint：

1. 训练恢复 checkpoint：包含 optimizer、scheduler、scaler 等完整状态。
2. 推理/评估 checkpoint：只包含模型权重、config 和 tokenizer。

训练恢复 checkpoint 更大，但能继续训练。

推理 checkpoint 更小，适合评估、部署和归档。

如果使用 FSDP 或 ZeRO，checkpoint 还可能是 sharded 格式，需要专门的保存和加载逻辑。

设模型参数量为 P ，权重每个参数占 b_w 字节，则推理权重大小近似为：

$$M_{\{\mathrm{eval}\}} = P b_w + M_{\{\mathrm{meta}\}}$$

如果训练恢复还保存梯度、AdamW 一阶/二阶状态和 master weights，恢复 checkpoint 可以粗略估成：

$$\begin{aligned} M_{\{\mathrm{resume}\}} \\ \approx P(b_w + b_g + b_m + b_v + b_{\{\mathrm{master}\}}) \\ + M_{\{\mathrm{meta}\}} \end{aligned}$$

保留策略的总存储可以写成：

$$\begin{aligned} M_{\{\mathrm{retain}\}} = & K_{\{\mathrm{latest}\}} M_{\{\mathrm{resume}\}} \\ & + K_{\{\mathrm{best}\}} M_{\{\mathrm{eval}\}} \\ & + K_{\{\mathrm{mile}\}} M_{\{\mathrm{eval}\}} \end{aligned}$$

这里 latest 偏容错，best 偏下游使用，mile 偏实验复盘。真实项目还要考虑 sharded checkpoint 合并、对象存储带宽和跨节点读取失败。

4. 训练恢复和容错

恢复训练时，要验证三件事：

1. 能否成功加载。
2. 恢复后 loss 是否连续。
3. 恢复后学习率和 step 是否正确。

一个健康恢复应该表现为：

恢复前后 loss 曲线连续
learning rate 不从头开始
optimizer 动量状态存在
global step 和 token count 正确

常见恢复错误：

1. 只加载 model weights。
2. scheduler 从 0 开始。
3. optimizer state 缺失。
4. tokenizer 版本不一致。
5. 分布式 shard 数变化导致加载失败。
6. 数据迭代位置变化，重复或跳过大量数据。

恢复后要做连续性检查。设恢复前最后一个健康日志为 -，恢复后第一个日志为 +：

```

\delta_L=
\frac{||L^+-L^-||}{L^+-\epsilon}
\delta_{\eta}=
\frac{||\eta^+-\eta^-||}{\eta^+-\epsilon}
\delta_{\mathrm{step}}=
s^+-s^- - 1

```

一个简化恢复门禁是：

```

G_{\mathrm{resume}}=
\mathbf{1}[\delta_L \leq \tau_L],
\mathbf{1}[\delta_{\eta} \leq \tau_{\eta}],
\mathbf{1}[\delta_{\mathrm{step}}=0],
\mathbf{1}[T^+>T^-]

```

如果 load 成功但 $G_{\text{resume}}=0$ ，说明训练状态可能不连续，必须先定位 scheduler、optimizer、random state 或 data position 的问题。

面试表达：checkpoint 恢复不是“能 load 成功”就结束，还要检查 loss、lr、step 和数据状态是否连续。

5. 为什么不能只看最后一个 checkpoint

很多初学者默认最后一步最好，但真实训练不一定。

可能出现：

1. 后期过拟合某些数据。
2. 某些能力退化。
3. 安全拒答变差。
4. checkpoint 附近发生 loss spike。
5. 后训练中风格过拟合。

所以 checkpoint 选择是多目标决策，不是只看 step 最大。

需要比较：

1. validation loss。
2. per-domain loss。
3. 下游 benchmark。
4. 人工评测。
5. safety eval。
6. regression eval。
7. 推理延迟和成本。

6. Validation loss 和 perplexity

预训练阶段最基础的指标是 validation loss。

Perplexity 可以理解为 cross entropy loss 的指数形式：

$\text{perplexity} = \exp(\text{loss})$

对 causal LM，带 mask 的 validation loss 可以写成：

```

L_{\mathrm{val}}=
-
\frac{
\sum_i \sum_t m_{\{i,t\}} \log p_{\{\theta\}}(x_{\{i,t\}} \mid x_{\{i,<t\}})
}{
\sum_i \sum_t m_{\{i,t\}}
}

```


对应 perplexity 为:

$$\text{PPL} = \exp(L_{\mathrm{val}})$$

loss 越低, perplexity 越低, 说明模型对验证集 token 的预测越好。

但要注意:

- 1. validation set 必须干净且不泄漏。
- 2. 不同 tokenizer 的 loss/perplexity 不宜直接比较。
- 3. 总 loss 可能掩盖某些 domain 退化。
- 4. loss 下降不代表所有能力提升。

面试表达: validation loss 是基础健康指标, 但不是完整能力评估。

7. Per-domain loss

只看整体 validation loss 不够。

应该按 domain 分开看:

Domain	可能观察
general web	通用语言能力
code	代码建模能力
math	数学符号和推理模式
zh	中文能力
en	英文能力
academic	论文和专业文本
dialogue	对话格式能力

例如整体 loss 下降, 但代码 loss 上升, 说明模型可能在代码能力上退化。

这种情况在数据配比调整后很常见。

设验证集按 domain 分成 D_k , 每个 domain 的 loss 为:

$$L_k = - \frac{\sum_{(i,t) \in D_k} m_{i,t} \log p_{\theta}(x_{i,t} \mid x_{i,<t})}{\sum_{(i,t) \in D_k} m_{i,t}}$$

整体 loss 可以看成加权平均:

$$L_{\mathrm{all}} = \sum_k w_k L_k$$

其中 w_k 来自各 domain 的有效 token 占比。checkpoint 对比时, 不仅要看 L_{all} 是否下降, 也要看是否存在:

$$L_k^{\mathrm{new}} - L_k^{\mathrm{old}} > \tau_k$$

这类 domain regression。

面试表达: per-domain loss 能帮助判断模型是全面变好, 还是只在某些数据分布上变好。

8. 下游 benchmark

Base model 训练阶段也需要下游 benchmark。

常见评估维度：

1. 知识问答。
2. 数学推理。
3. 代码生成。
4. 阅读理解。
5. 多语言能力。
6. 长上下文能力。
7. 安全和毒性。

但 benchmark 有风险：

1. 可能被污染。
2. 可能和真实目标不一致。
3. 可能只反映某一类能力。
4. 分数提升可能没有统计显著性。

所以 benchmark 要和 validation loss、人工评测、回归测试一起看。

9. Regression eval

模型新版本不能只看新能力是否提升，还要看旧能力是否退化。

Regression eval 的目标是发现：

旧模型答对，新模型答错

这类样本叫 loss cases。

设旧模型正确标记为 z_i^{old} ，新模型正确标记为 z_i^{new} ，回归率可以写成：

$$R_{\{\mathrm{reg}\}} = \frac{\sum_i \mathbf{1}[z_i^{\text{old}}=1] \mathbf{1}[z_i^{\text{new}}=0]}{\sum_i \mathbf{1}[z_i^{\text{old}}=1] + \epsilon}$$

如果样本带业务权重 w_i ，高风险回归率可以写成：

$$R_{\{\mathrm{reg}, w\}} = \frac{\sum_i w_i \mathbf{1}[z_i^{\text{old}}=1] \mathbf{1}[z_i^{\text{new}}=0]}{\sum_i w_i \mathbf{1}[z_i^{\text{old}}=1] + \epsilon}$$

常见回归维度：

1. 格式遵循。
2. 数学和代码。
3. 安全拒答。
4. 多语言。
5. 企业业务规则。
6. 长上下文引用。

面试表达：模型评估不能只看平均分提升，要重点看高风险场景有没有退化。

10. 实验追踪应该记录什么

一个训练实验至少要记录：

类别	内容
代码	git commit、分支、diff
数据	数据版本、配比、过滤规则、token 数
模型	架构、参数量、tokenizer、context length
训练	lr、batch、optimizer、schedule、precision、parallelism
环境	GPU 类型、数量、驱动、框架版本
日志	loss、grad norm、lr、tokens/sec、OOM/NaN
checkpoint	保存路径、step、token count、评估结果
结论	提升、退化、异常、下一步

没有这些记录，实验就不可复现。

实验记录可以抽象为：

$R_{\{\mathrm{run}\}} = (v_{\{\mathrm{code}\}}, v_{\{\mathrm{data}\}}, v_{\{\mathrm{tok}\}}, v_{\{\mathrm{cfg}\}}, h, s, A, M)$

其中 v_{code} 是代码版本， v_{data} 是数据版本， v_{tok} 是 tokenizer 版本， v_{cfg} 是模型和训练配置版本， h 是硬件和依赖环境， s 是随机种子， A 是 artifact 列表， M 是指标表。最小可复现门禁为：

$G_{\{\mathrm{repro}\}} =$
 $g_{\{\mathrm{code}\}} \backslash,$
 $g_{\{\mathrm{data}\}} \backslash,$
 $g_{\{\mathrm{tok}\}} \backslash,$
 $g_{\{\mathrm{cfg}\}} \backslash,$
 $g_{\{\mathrm{env}\}} \backslash,$
 $g_{\{\mathrm{seed}\}} \backslash,$
 $g_{\{\mathrm{eval}\}}$

面试表达：实验管理的核心是让别人能复现你的训练结果，也能理解每次变化来自哪里。

11. 实验命名和版本管理

实验命名要能表达关键信息。

坏命名：

run1
test_new
final_v2

更好的命名：

7b_pretrain_lr3e-4_bs4m_tok2t_data-v5_bf16_fsd

不一定要很长，但至少应该能看出：

1. 模型规模。
2. 训练阶段。
3. 数据版本。
4. 关键超参。
5. 时间或 run id。

版本管理包括：

1. 代码版本。
2. 数据版本。

3. tokenizer 版本。
4. config 版本。
5. checkpoint 版本。

其中数据版本尤其重要。很多训练差异来自数据变化，而不是模型或优化器变化。

12. 可复现性

大模型训练完全 bit-level 复现很难，但工程上至少要做到结论可复现。

影响复现的因素：

1. 随机种子。
2. 数据顺序。
3. 分布式并行策略。
4. kernel 非确定性。
5. mixed precision。
6. checkpoint 恢复状态。
7. 依赖库版本。

可复现性最低要求：

同样代码 + 同样数据 + 同样配置，可以得到相近 loss 曲线和相近评估结论。

不要承诺所有大规模分布式训练都能逐 bit 复现。

13. 实验对比的基本原则

比较两个实验时，要尽量做到单变量控制。

例如你想比较学习率：

只改 learning rate

其他数据、模型、batch、seed、训练 token 尽量一致

如果同时改了数据、学习率和 tokenizer，就很难判断提升来自哪里。

常见错误：

1. 不同训练 token 数直接比较。
2. 不同 tokenizer 直接比较 loss。
3. 不同数据版本却归因到 optimizer。
4. 只看一个 benchmark。
5. 没有统计波动和多次实验。

如果两个实验配置分别是 h_A 和 h_B ，想验证的变量集合是 V ，公平对比要求除 V 外的关键字段一致：

$$D_{\{\mathrm{diff}\}} = \{u \mid h_A(u) \neq h_B(u), u \notin V\}$$

$$G_{\{\mathrm{compare}\}} = \{\mathbf{1}[D_{\{\mathrm{diff}\}} = \text{varnothing}], \mathbf{1}[T_A = T_B], \mathbf{1}[E_A = E_B]\}$$

其中 T_A/T_B 是训练 token 数， E_A/E_B 是评估集和评估脚本版本。真实实验中很难做到所有变量完全一致，但至少要把差异记录清楚，不能把多变量变化包装成单变量结论。

面试表达：实验结论可信的前提是对照清晰、变量可控、评估充分。

14. Checkpoint 选择流程

一个实用选择流程：

1. 过滤掉训练异常 checkpoint。
2. 按 validation loss 选候选。
3. 看 per-domain loss, 排除明显偏科。
4. 跑核心 benchmark。
5. 做 regression eval。
6. 做 safety eval。
7. 抽样人工评测。
8. 评估推理成本。
9. 选择进入后训练或部署的 checkpoint。

可以把 checkpoint 选择写成多目标打分：

```
J(c)=  
-\alpha L_{\mathrm{val}}(c)  
+\beta S_{\mathrm{bench}}(c)  
+\gamma S_{\mathrm{safety}}(c)  
-\lambda R_{\mathrm{reg}}(c)  
-\mu C_{\mathrm{infer}}(c)
```

这里 S_{bench} 是 benchmark 汇总分, S_{safety} 是安全评估分, R_{reg} 是回归风险, C_{infer} 是推理成本。权重不是固定真理, 而是要和项目目标绑定: base model 可能更重视 validation / benchmark, 面向企业部署的模型可能更重视 regression / safety / latency。

这比“最后一个 checkpoint” 稳健很多。

15. 最小可运行实验管理审计 demo

下面的 demo 用标准库模拟一次 checkpoint 与实验管理审计。输入是 3 个 checkpoint、一次恢复探针、一个实验记录和两组对照实验；输出 checkpoint 完整性、PPL、domain regression、候选打分、保留存储、恢复连续性、实验记录完整性和对照是否公平。

```
import math  
from statistics import mean  
  
required_resume_fields = [  
    "model", "optimizer", "scheduler", "scaler", "rng", "data_position",  
    "global_step", "consumed_tokens", "config", "tokenizer",  
]  
  
checkpoints = [  
    {  
        "id": "ckpt_1000", "step": 1000, "tokens_b": 52.4, "train_loss": 2.44, "val_loss": 2.31,  
        "domain_loss": {"general": 2.20, "code": 2.90, "math": 2.70},  
        "bench": {"qa": 0.52, "math": 0.31, "code": 0.28},  
        "safety": 0.95, "regression_failures": 5, "stable": True, "size_gib": 180,  
        "resume_fields": {name: True for name in required_resume_fields},  
    },  
    {  
        "id": "ckpt_2000", "step": 2000, "tokens_b": 104.9, "train_loss": 2.29, "val_loss": 2.18,  
        "domain_loss": {"general": 2.08, "code": 2.74, "math": 2.52},  
        "bench": {"qa": 0.56, "math": 0.38, "code": 0.34},  
        "safety": 0.94, "regression_failures": 4, "stable": True, "size_gib": 180,  
        "resume_fields": {name: True for name in required_resume_fields},  
    },  
]
```

```

        "id": "ckpt_3000", "step": 3000, "tokens_b": 157.3, "train_loss": 2.20, "val_loss": 2.16,
        "domain_loss": {"general": 2.05, "code": 2.98, "math": 2.41},
        "bench": {"qa": 0.57, "math": 0.40, "code": 0.29},
        "safety": 0.89, "regression_failures": 11, "stable": False, "size_gib": 180,
        "resume_fields": {**{name: True for name in required_resume_fields}, "scaler": False, "data_position": False}
    },
]

resume_probe = {
    "checkpoint_id": "ckpt_2000",
    "loss_before": 2.19,
    "loss_after": 2.20,
    "lr_before": 2.8e-4,
    "lr_after": 2.8e-4,
    "step_before": 2000,
    "step_after": 2001,
    "tokens_before_b": 104.90,
    "tokens_after_b": 104.95,
}

run_manifest = {
    "git_commit": "abc1234", "data_version": "data_v5", "tokenizer_version": "tok_v2",
    "model_config": "7b_gqa_rope", "train_config": "lr3e-4_bs4m_bf16_fsd",
    "env": "64xa100_cuda12", "seed": 42, "eval_script": "eval_v3.py",
    "checkpoint_id": "ckpt_2000", "metrics": {"val_loss": 2.18}, "notes": "lr sweep winner",
}

run_a = {"data_version": "data_v5", "tokenizer": "tok_v2", "tokens_b": 104.9, "lr": 3e-4, "batch_tokens": 4_000_000,
run_b = {"data_version": "data_v5", "tokenizer": "tok_v2", "tokens_b": 104.9, "lr": 2e-4, "batch_tokens": 4_000_000,
run_bad = {"data_version": "data_v6", "tokenizer": "tok_v2", "tokens_b": 157.3, "lr": 2e-4, "batch_tokens": 4_000_000

base = checkpoints[0]
base_val_loss = base["val_loss"]
storage_budget_gib = 400
latest_k = 2
save_interval_steps = 1000
step_seconds = 8
checkpoint_write_seconds = 60

def missing_fields(ckpt):
    return [name for name in required_resume_fields if not ckpt["resume_fields"].get(name, False)]

def perplexity(loss):
    return round(math.exp(loss), 2)

def domain_regressions(ckpt, threshold=0.05):
    return [
        domain
        for domain, loss in ckpt["domain_loss"].items()
        if loss - base["domain_loss"][domain] > threshold
    ]

def score_checkpoint(ckpt):
    bench_avg = mean(ckpt["bench"].values())
    domain_penalty = 0.10 * len(domain_regressions(ckpt))

```

```

    regression_penalty = 0.02 * ckpt["regression_failures"]
    stability_penalty = 0 if ckpt["stable"] else 0.50
    score = (base_val_loss - ckpt["val_loss"]) + bench_avg + ckpt["safety"]
    score -= domain_penalty + regression_penalty + stability_penalty
    return round(score, 4)

def changed_keys(a, b, ignore=("lr",)):
    return [key for key in a if key in b and a[key] != b[key] and key not in ignore]

completeness = {ckpt["id"]: missing_fields(ckpt) for ckpt in checkpoints}
ppl = {ckpt["id"]: perplexity(ckpt["val_loss"]) for ckpt in checkpoints}
domain_report = {ckpt["id"]: domain_regressions(ckpt) for ckpt in checkpoints}
score_table = [
    {"id": ckpt["id"], "score": score_checkpoint(ckpt), "stable": ckpt["stable"]}
    for ckpt in checkpoints
]
best = max(
    [ckpt for ckpt in checkpoints if not missing_fields(ckpt)],
    key=score_checkpoint,
)

latest_retained = sorted(checkpoints, key=lambda ckpt: ckpt["step"], reverse=True)[:latest_k]
retained_ids = sorted({best["id"], *(ckpt["id"] for ckpt in latest_retained)})
retained_storage_gib = sum(ckpt["size_gib"] for ckpt in checkpoints if ckpt["id"] in retained_ids)
save_overhead = round(checkpoint_write_seconds / (save_interval_steps * step_seconds), 4)
max_lost_tokens_b = round((checkpoints[1]["tokens_b"] - checkpoints[0]["tokens_b"]), 1)

loss_jump = abs(resume_probe["loss_after"] - resume_probe["loss_before"]) / resume_probe["loss_before"]
resume_checks = {
    "loss_continuous": loss_jump < 0.02,
    "lr_continuous": resume_probe["lr_before"] == resume_probe["lr_after"],
    "step_continuous": resume_probe["step_after"] == resume_probe["step_before"] + 1,
    "tokens_increase": resume_probe["tokens_after_b"] > resume_probe["tokens_before_b"],
}

manifest_required = [
    "git_commit", "data_version", "tokenizer_version", "model_config", "train_config",
    "env", "seed", "eval_script", "checkpoint_id", "metrics", "notes",
]
manifest_missing = [name for name in manifest_required if name not in run_manifest]
fair_lr_compare = changed_keys(run_a, run_b) == []
bad_compare_changed = changed_keys(run_a, run_bad)

checks = {
    "best_complete": completeness[best["id"]] == [],
    "resume_continuous": all(resume_checks.values()),
    "storage_under_budget": retained_storage_gib <= storage_budget_gib,
    "manifest_complete": manifest_missing == [],
    "fair_lr_compare": fair_lr_compare,
    "bad_compare_detected": bad_compare_changed == ["data_version", "tokens_b"],
    "safety_ok": best["safety"] >= 0.92,
    "regression_ok": best["regression_failures"] <= 5,
}

print("completeness=", completeness)

```

```

print("ppl=", ppl)
print("domain_report=", domain_report)
print("score_table=", score_table)
print("best_checkpoint=", best["id"])
print("retained_ids=", retained_ids)
print("retained_storage_gib=", retained_storage_gib)
print("save_overhead=", save_overhead)
print("max_lost_tokens_b=", max_lost_tokens_b)
print("resume_checks=", resume_checks)
print("manifest_missing=", manifest_missing)
print("fair_lr_compare=", fair_lr_compare)
print("bad_compare_changed=", bad_compare_changed)
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

期望输出：

```

completeness= {'ckpt_1000': [], 'ckpt_2000': [], 'ckpt_3000': ['scaler', 'data_position']}
ppl= {'ckpt_1000': 10.07, 'ckpt_2000': 8.85, 'ckpt_3000': 8.67}
domain_report= {'ckpt_1000': [], 'ckpt_2000': [], 'ckpt_3000': ['code']}
score_table= [{'id': 'ckpt_1000', 'score': 1.22, 'stable': True}, {'id': 'ckpt_2000', 'score': 1.4167, 'stable': True}, {'id': 'ckpt_3000', 'score': 1.4167, 'stable': True}]
best_checkpoint= ckpt_2000
retained_ids= ['ckpt_2000', 'ckpt_3000']
retained_storage_gib= 360
save_overhead= 0.0075
max_lost_tokens_b= 52.5
resume_checks= {'loss_continuous': True, 'lr_continuous': True, 'step_continuous': True, 'tokens_increase': True}
manifest_missing= []
fair_lr_compare= True
bad_compare_changed= ['data_version', 'tokens_b']
checks= {'best_complete': True, 'resume_continuous': True, 'storage_under_budget': True, 'manifest_complete': True, 'fair_lr_compare': True}
gate_pass= True

```

这个 demo 的关键是把“选哪个 checkpoint”拆成可审计步骤：先过滤不完整和不稳定 checkpoint，再看 validation / PPL / domain / benchmark / safety / regression，最后检查实验记录是否完整、对照是否公平、保留策略是否超过存储预算。

16. 面试官会怎么问

问法 1: checkpoint 里应该保存什么？

可以这样答：

如果只是推理，保存 model weights、config 和 tokenizer 基本够用。如果要恢复训练，还要保存 optimizer state、scheduler state。

问法 2: 如何选择最好的 checkpoint？

可以这样答：

不能只选最后一步。我会先看 validation loss 和 per-domain loss，再跑目标 benchmark、regression eval 和 safety eval。对于其他指标，要看具体需求。

问法 3: 为什么 train loss 降低不代表模型更好？

可以这样答：

train loss 降低可能来自数据重复、训练集泄漏、过拟合或更容易的数据分布。它不一定代表下游能力、安全性或泛化提升。所以要结合 validation loss、domain loss、benchmark、人工评测和 regression eval。

问法 4：如何保证实验可复现？

可以这样答：

我会记录代码 commit、数据版本、tokenizer、模型 config、训练超参、硬件环境、随机种子、checkpoint 和评估脚本版本。大规模 level 复现，但至少要保证同样配置能得到相近 loss 曲线和相近评估结论。

问法 5：两个实验怎么公平比较？

可以这样答：

尽量做单变量对照，保持数据、模型、训练 token、batch、评估集和脚本一致，只改变想验证的因素。比较时不能只看单个平均分，domain、regression、safety 和统计波动。

17. 本章小结

本章核心结论：

1. Checkpoint 是训练容错、评估和版本管理的核心产物。
2. 恢复训练需要 model、optimizer、scheduler、scaler、随机状态和数据状态。
3. 保存频率要在容错、评估和存储成本之间平衡。
4. 不能默认最后一个 checkpoint 最好。
5. Validation loss 是基础指标，但不是完整能力评估。
6. Per-domain loss 能发现模型偏科和能力退化。
7. Benchmark 要结合污染检测、回归测试和人工评测使用。
8. 实验追踪要记录代码、数据、模型、训练配置、环境和结论。
9. 可复现性要求结论可复现，不一定要要求 bit-level 完全一致。
10. 面试中要把 checkpoint、评估和实验管理讲成一个长期训练系统。

第十一章：后训练与对齐强化训练

本章定位

预训练得到的是 base model，它具备语言建模能力和世界知识，但通常还不能直接作为可靠助手使用。

后训练的目标是把 base model 变成能遵循指令、能对话、能推理、能调用工具、能遵守安全边界，并且符合人类偏好的 assistant model 或 reasoning model。

过去很多教材会把后训练简化成：

SFT -> RLHF/PPPO -> DPO

这个说法已经不够用了。

当前真实训练路线更接近：

SFT / cold start

-> reward modeling / rule-based reward / verifiable reward

-> online RL or offline preference optimization

-> long-CoT / reasoning RL

-> safety alignment

-> regression eval and iterative data refresh

因此，本章不能只讲 DPO。DPO 是重要方法，但它更多是离线偏好优化路线的一部分，不代表当前主流 RL 后训练全貌。

本章会系统梳理 PPO、GRPO、GSPO、DAPO、DCPO、DPO 及相关方法，重点讲清它们分别解决什么问题、依赖什么数据、优化什么目标、优缺点是什么、适合什么场景，以及面试中如何表达。

本章目标

学完本章，你需要掌握：

1. 后训练为什么不等于简单 SFT。
2. SFT、RM、PPO、DPO、GRPO、GSPO、DAPO、DCPO 的位置关系。
3. 为什么早期 RLHF 常用 PPO。
4. 为什么 reasoning model 训练中会出现 GRPO、DAPO 等方法。
5. DPO 为什么方便，但为什么不能代表所有主流对齐强化训练。
6. 各类方法对数据、reward、reference model、critic、采样和训练稳定性的要求。
7. 面试中如何讲清现代后训练 pipeline。

为什么后训练是模型变成助手的关键阶段

预训练得到的 base model 主要学到的是语言建模能力。

它知道很多文本模式、知识模式和推理模式，但通常还不适合直接面对用户。

原因包括：

1. 不一定遵循指令。
2. 不一定会拒答危险请求。
3. 不一定知道输出格式要求。
4. 不一定会优先给出帮助性、简洁性和安全性兼顾的回答。
5. 不一定会根据反馈持续改善。

后训练的目标是把 base model 变成 assistant model 或 reasoning model。

面试表达：预训练负责“学会语言和知识”，后训练负责“学会怎么当助手”。

后训练与对齐训练的资料边界

按照 WRITING_PLAN.md 的要求，本章第二轮精修前核对了 InstructGPT / RLHF、DPO、DeepSeekMath / GRPO、Hugging Face TRL 的 DPO / GRPO 文档，以及 GSPO、DAPO、DCPO 等近年公开论文或技术说明。

本章聚焦面试和工程中最常用的后训练抽象：SFT、reward model、PPO-RLHF、DPO、GRPO、sequence-level / advanced policy optimization、安全约束、回归评估和数据刷新。对 GSPO、DAPO、DCPO 这类高时效方法，本章只写成可迁移的机制和边界，不把不同团队的具体实现写成唯一标准。

后训练目标的统一口径

后训练可以抽象成在 base policy π_0 上学习新 policy π_θ ：

$$\pi_\theta = \text{PostTrain}(\pi_0, D_{\text{sft}}, D_{\text{pref}}, R, G_{\text{safe}})$$

其中 D_{sft} 是监督示范数据， D_{pref} 是偏好数据， R 是 reward 或规则验证器， G_{safe} 是安全和行为门禁。最终目标不是单一 loss 最低，而是综合提升：

$$G_{\text{align}} = g_{\text{sft}} \setminus, g_{\text{pref}} \setminus, g_{\text{reward}} \setminus, g_{\text{kl}} \setminus, g_{\text{safety}} \setminus, g_{\text{regression}}$$

这些门禁分别对应指令遵循、偏好质量、reward 可用性、策略偏移可控、安全边界和能力回归。

1. 后训练总体流程

一个典型 LLM 后训练流程可以拆成六层。

1.1 SFT: 建立基本指令遵循能力

SFT 使用人工或模型生成的高质量 instruction-response 数据, 让模型学会按照用户指令回答。

它解决的问题是:

base model 只会续写, 不一定会按用户意图回答。

SFT 的目标通常是 next-token cross entropy:

$$\max \log p_{\theta}(y \mid x)$$

其中 x 是 prompt, y 是标准回答。

更精确地说, SFT 只应该在 assistant token 上计算 loss。设 $m_t=1$ 表示第 t 个 token 属于需要监督的 assistant 输出:

$$L_{\{\mathrm{sft}\}} = - \frac{\sum_{i,t} m_{i,t} \log \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})}{\sum_{i,t} m_{i,t}}$$

如果 m_t 把 user prompt 或 padding 也算进去, 模型会学到错误目标, 所以 SFT 的第一道门禁是检查 assistant-only label mask。

SFT 的优点是简单稳定。

缺点是它只能模仿数据, 不会主动优化 “哪个回答更好”。

1.2 Reward Modeling: 学习偏好信号

Reward Model 使用偏好数据训练一个打分模型。

偏好数据通常长这样:

prompt
chosen response
rejected response

Reward Model 学的是:

$$r(\text{prompt}, \text{chosen}) > r(\text{prompt}, \text{rejected})$$

常见 reward model 训练可以用 Bradley-Terry 形式理解。对偏好样本 (x, y^+, y^-) :

$$P(y^+ \mid x) = \frac{\exp(r_{\phi}(x, y^+))}{\exp(r_{\phi}(x, y^+)) + \exp(r_{\phi}(x, y^-))}$$

对应 loss 为:

$$L_{\{\mathrm{rm}\}} = - \log \frac{\exp(r_{\phi}(x, y^+))}{\exp(r_{\phi}(x, y^+)) + \exp(r_{\phi}(x, y^-))}$$

它解决的问题是:

如何把人类偏好变成可优化的标量 reward。

1.3 RLHF：用 RL 优化策略模型

有了 reward 后，可以把语言模型看作 policy。
模型生成回答，reward model 给分，训练算法更新 policy，使它更倾向于生成高 reward 回答。
经典路线是 PPO-RLHF。

1.4 Offline Preference Optimization：离线偏好优化

DPO、IPO、KTO、ORPO 等方法不一定显式训练 reward model，也不一定在线 rollout。
它们直接用偏好对优化模型。
优点是工程简单、稳定、成本低。
缺点是受限于离线偏好数据，对复杂 reasoning、长链条探索和 verifiable reward 场景不一定够。

1.5 Reasoning RL：面向可验证任务的强化训练

数学、代码、逻辑推理、工具调用等任务可以用规则或程序验证答案。
例如：
数学题答案是否正确
代码是否通过单元测试
工具调用结果是否满足目标
这类任务可以使用 rule-based reward 或 verifiable reward，进行在线采样和 RL 优化。
GRPO、DAPO、GSPO、DCPO 等方法通常出现在这一类讨论中。

1.6 Safety Alignment：安全与行为边界

后训练还必须控制：
危险请求
隐私泄露
越狱
偏见
过度拒答
事实幻觉
工具误用
安全对齐不是单独一个 loss 就能解决，而是数据、reward、拒答策略、红队评估和上线门禁的组合。

2. 方法谱系总览

可以先用一张表建立全局视角。

方法	类型	是否需要 Reward Model	是否在线采样	是否需要 Critic	典型用途
SFT	监督微调	否	否	否	指令遵循、冷启动
PPO-RLHF	在线 RL	通常需要	是	通常需要	经典人类偏好 RLHF
DPO	离线偏好优化	否	否	否	偏好对齐、风格优化

方法	类型	是否需要 Reward Model	是否在线采样	是否需要 Critic	典型用途
IPO/KTO/ORPO	离线偏好优化	否	否	否	DPO 替代或变体
GRPO	在线 RL / group-based RL	可用规则 reward	是	通常不需要 critic	reasoning RL、 数学代码任务
GSPO	sequence- level policy optimization	可用 reward	是	依实现而定	序列级稳定优 化、长输出对齐
DAPO	decoupled / advanced policy optimization	可用 reward	是	依实现而定	长链推理 RL、 稳定性和效率 优化
DCPO	contrastive / constrained preference optimization	可用偏好或约 束信号	可在线或离线	依实现而定	偏好、约束、安 全和行为控制

注意：不同论文或团队对 GSPO、DAPO、DCPO 的全称和实现细节可能不同，面试中不要只背缩写。更重要的是讲清：它们是在 PPO/DPO 之外，为了解决 reasoning RL、长序列优化、稳定性、样本效率和约束对齐问题而发展出的后训练方法。

3. PPO-RLHF

PPO 是早期 RLHF 的代表方法。

整体流程如下：

1. 用 SFT 得到初始 policy model。
2. 训练 reward model。
3. policy model 对 prompt 采样回答。
4. reward model 给回答打分。
5. PPO 根据 reward 更新 policy。
6. 使用 KL penalty 防止 policy 偏离 reference model 太远。

PPO 的目标大致可以理解为：

maximize reward(response) - beta * KL(policy || reference)

其中 KL 项非常重要。

如果把一条 response 记为 $y=(y_1,\ldots,y_T)$ ，sequence-level KL 可以写成：

$$\begin{aligned} & \mathrm{KL}_{\mathrm{seq}} \\ &= \\ & \sum_{t=1}^T \\ & \left[\log \pi_{\theta}(y_t \mid x, y_{1:t-1}) \right. \\ & \quad \left. - \log \pi_{\mathrm{ref}}(y_t \mid x, y_{1:t-1}) \right] \end{aligned}$$

RLHF 的简化目标是：

$$\begin{aligned} J_{\mathrm{rlhf}} &= \\ E_{y \sim \pi_{\theta}} & \end{aligned}$$

$$\left[r_{\phi}(x,y) - \beta \mathrm{KL}_{\mathrm{seq}} \right]$$

PPO 实际训练还会使用 ratio clipping、advantage 和 value / critic 估计，面试中至少要讲清 reward 最大化和 KL 约束这两个核心。

如果没有 KL 约束，模型可能为了骗 reward model 而生成奇怪回答，也就是 reward hacking。

PPO 的优点：

1. 能在线采样，优化当前模型真实生成分布。
2. 可以直接使用 reward model 或规则 reward。
3. 适合需要探索和迭代优化的任务。
4. 是经典 RLHF 路线，理论和工程经验较多。

PPO 的缺点：

1. 工程复杂，需要 policy、reference、reward、value/critic 多个模型或模块。
2. 显存和计算成本高。
3. 训练不稳定，容易 reward hacking、KL 崩掉、长度偏置。
4. 对 reward model 质量高度敏感。

面试中可以这样说：

PPO-RLHF 是经典在线 RLHF 方法，优势是能在模型自己的采样分布上直接优化 reward，缺点是系统复杂、成本高、稳定性差，需要 r

4. DPO：直接偏好优化

DPO 的核心动机是：能不能不显式训练 reward model，也不跑复杂 PPO，直接用偏好对训练模型？

DPO 使用数据：

(prompt, chosen, rejected)

它直接优化模型，让 chosen 的相对概率高于 rejected。

直觉上：

如果有人更喜欢 chosen，就提高模型生成 chosen 的概率，降低 rejected 的概率。

DPO 的核心量是 policy 和 reference 在 chosen / rejected 上的 log probability gap：

$$\begin{aligned} \Delta_{\theta} &= \log \pi_{\theta}(y^{+} | x) - \log \pi_{\theta}(y^{-} | x) \\ \Delta_{\mathrm{ref}} &= \log \pi_{\mathrm{ref}}(y^{+} | x) - \log \pi_{\mathrm{ref}}(y^{-} | x) \end{aligned}$$

DPO loss 可以写成：

$$\begin{aligned} L_{\mathrm{dpo}} &= - \log \sum \beta (\Delta_{\theta} - \Delta_{\mathrm{ref}}) \end{aligned}$$

beta 控制偏好优化强度。beta 太小，更新弱；beta 太大，容易过度拟合偏好对或造成能力回归。

DPO 的优点：

1. 工程简单。
2. 不需要在线 rollout。
3. 不需要显式 reward model。
4. 训练稳定性通常好于 PPO。
5. 适合偏好数据充足、目标相对明确的场景。

DPO 的缺点：

1. 依赖离线偏好数据质量。
2. 不能充分探索当前 policy 的新输出。
3. 对复杂 long-CoT reasoning 的优化能力有限。
4. 容易学到风格偏好，而不一定提升真实任务正确率。
5. 对 verifiable reward、代码执行 reward、数学答案 reward 这类场景不如在线 RL 灵活。

为什么说 “DPO 目前不是全部主流” ？

因为很多 reasoning model 后训练关注的是：

模型自己采样多条推理链

用规则或程序验证结果

根据正确性、格式、长度、过程质量做 reward

继续优化 policy

这类训练天然更接近在线 RL，而不是只用固定偏好对的 DPO。

5. GRPO: Group Relative Policy Optimization

GRPO 可以理解为一种面向 group sampling 的 RL 优化方法。

它常见于 reasoning RL 场景。

基本思想是：对同一个 prompt，模型采样多条回答，形成一个 group：

prompt -> response_1, response_2, ..., response_k

然后用 reward 给每条回答打分：

r_1, r_2, ..., r_k

再用组内相对优势更新模型。

直觉是：

同一道题里，奖励更高的回答应该被增强，奖励更低的答案应该被压低。

对第 i 个 prompt 采样 K 条回答，reward 为 r_{i,j}，组内均值和标准差为：

$$\bar{r}_i = \frac{1}{K} \sum_{j=1}^K r_{i,j}$$

$$s_i = \sqrt{\frac{1}{K} \sum_{j=1}^K (r_{i,j} - \bar{r}_i)^2}$$

组内相对 advantage 可以写成：

$$A_{i,j} = \frac{r_{i,j} - \bar{r}_i}{s_i + \epsilon}$$

如果 $\rho_{i,j}$ 是新旧 policy 的概率比，一个简化 clipped objective 是：

$$L_{\{\mathrm{grpo}\}} = -\frac{1}{N} \sum_{i,j} \min(\rho_{i,j} A_{i,j}, \mathrm{clip}(\rho_{i,j}, 1-\epsilon, 1+\epsilon) A_{i,j})$$

这不是所有 GRPO 实现的完整细节，但能抓住面试中最关键的点：用同 prompt 的组内 reward 构造 baseline / advantage，减少对 critic 的依赖。

GRPO 相比 PPO 的一个重要特点是：通常不需要单独训练 critic/value model，而是用 group 内 reward 做 baseline 或 advantage 估计。

它适合：

1. 数学推理。
2. 代码生成。
3. 可验证 QA。
4. 有明确对错 reward 的任务。
5. 同一 prompt 可以采样多条候选的场景。

优点：

1. 比 PPO 工程上更轻，减少 critic 训练复杂度。
2. 适合 verifiable reward。
3. 能利用同一问题多条采样之间的相对质量。
4. 对 reasoning RL 比纯 DPO 更自然。

缺点：

1. 需要在线采样，成本仍然不低。
2. 对 reward 设计很敏感。
3. group size、KL、长度控制都会影响稳定性。
4. 如果 reward 太稀疏，训练效率会下降。

面试中可以这样说：

GRPO 的关键是 group relative advantage。它对同一个 prompt 采样多条输出，用组内 reward 差异构造优化信号，通常不需要像 PPO

6. GSPO：序列级策略优化

GSPO 可以放在 “sequence-level policy optimization” 这类方法中理解。

很多 token-level RL 更新会遇到一个问题：最终 reward 是对整条回答给的，但更新时要分摊到每个 token。

对于长 CoT，回答可能有几百到几千 token。

如果处理不好，会出现：

1. token-level credit assignment 噪声大。
2. 长回答被系统性奖励或惩罚。
3. 局部 token 概率变化破坏整体推理结构。
4. KL 控制在 token 级和 sequence 级之间不一致。

GSPO 这类方法强调在序列级别进行更稳定的 policy optimization。

核心关注点包括：

1. 把完整 response 作为优化单元。
2. 更直接地优化 sequence-level reward。

3. 控制长序列训练中的方差和 KL。
4. 避免 token 级更新导致的局部不稳定。

适用场景：

长推理链
长代码生成
多步骤工具调用
多轮任务规划

面试中不建议死背 GSPO 公式，而要能表达：

随着 reasoning model 输出越来越长，后训练不只关心单 token 的概率提升，而要关心整条推理序列的 reward、KL 和稳定性。GSPO

7. DAPO：面向长链推理的高级策略优化

DAPO 可以作为 PPO/GRPO 之后的进一步优化路线来理解，重点通常在于提升长链 reasoning RL 的稳定性、效率和有效样本利用。

在 reasoning RL 中，常见痛点是：

1. reward 稀疏：只有最终答案对错。
2. 输出很长：一次 rollout 成本高。
3. 正样本少：模型早期答对率低。
4. 长度偏置：模型可能学会输出更长或更短来投机。
5. KL 不稳定：模型容易偏离初始策略。
6. 难题和易题混在一起，训练信号质量不均衡。

DAPO 类方法通常会围绕这些问题做改进，例如：

1. 更细的 advantage 设计。
2. 更合理的 token/sequence level loss 解耦。
3. 动态采样或难度过滤。
4. 对过长、过短、格式错误输出做约束。
5. 改进 KL 或 clipping 机制。
6. 提高有效 rollout 比例。

它适合：

数学推理强化
代码强化
竞赛题训练
复杂多步任务
长 CoT 模型后训练

面试中可以这样概括：

DAPO 这类方法关注的是在长链推理 RL 中进一步提升训练稳定性和样本效率。相比只说 PPO 或 DP0，它更贴近 reasoning model 后训练

8. DCPO：偏好、对比与约束优化

DCPO 可以放在 “contrastive / constrained preference optimization” 这类方法中理解。

后训练不仅要让模型生成更好的答案，还要满足一些约束：

不能泄露隐私
不能输出危险指导
不能过度拒答
不能破坏格式
不能牺牲基础能力
不能为了 reward 过度投机

这类问题只用一个简单 reward 有时不够。

DCPO 类方法通常关注：

1. chosen 和 rejected 之间的对比优化。
2. reward 最大化与约束满足之间的平衡。
3. 安全、格式、事实性、长度等多目标控制。
4. 防止模型在某个目标上过优化导致其他能力退化。

可以把它理解成：

不是只问“哪个回答更好”，还要问“这个回答是否满足约束”。

适用场景：

安全对齐

格式严格任务

工具调用约束

企业助手行为规范

多目标偏好优化

面试中可以这样说：

DCPO 类方法体现了后训练从单一偏好优化走向多目标和约束优化的趋势。实际助手模型不仅要 reward 高，还要满足安全、格式、事实性。

9. PPO、DPO、GRPO、GSPO、DAPO、DCPO 怎么选

可以按任务和资源选择。

9.1 如果目标是通用聊天偏好

可以考虑：

SFT -> DPO/IPPO/KTO/ORPO -> safety tuning -> eval

原因是通用聊天偏好通常有大量离线 preference 数据，DPO 类方法成本低、稳定。

9.2 如果目标是数学和代码 reasoning

可以考虑：

SFT/cold start -> online sampling -> rule-based reward -> GRPO/DAPO/GSPO -> eval

原因是数学和代码任务可以自动验证，在线 RL 能让模型探索新的推理路径。

9.3 如果目标是安全和格式约束

可以考虑：

SFT safety data -> preference/constraint optimization -> red-team eval

可使用 DPO/DCPO 类思路，也可以结合 rule-based reward。

9.4 如果资源有限

优先路线通常是：

高质量 SFT -> 小规模 DPO -> 重点评估和数据迭代

不建议一开始就上复杂 online RL。

9.5 如果资源充足且任务可验证

可以做：
大规模采样 -> verifiable reward -> GRPO/DAPO -> 多轮迭代
这更接近 reasoning model 的训练思路。

10. 数据要求对比

方法	数据形态	关键质量要求
SFT	prompt + answer	答案高质量、格式统一、覆盖目标任务
Reward Model	prompt + chosen + rejected	偏好可靠、差异明确、标注一致
PPO	prompt + reward signal	reward 可用、KL 稳定、采样质量可控
DPO	prompt + chosen + rejected	chosen/rejected 质量差异真实，避免虚假偏好
GRPO	prompt + 多条 sampled responses + reward	reward 可验证，group 内有区分度
GSPO	prompt + sequence reward	长序列 reward 可靠，长度和格式受控
DAPO	prompt + rollout + reward/constraint	有效样本比例高，难度和长度分布合理
DCPO	preference + constraints	偏好和约束标签可靠，多目标不冲突

后训练数据的一个常见风险是 chosen 只是更长，而不是真的更好。可以用长度偏置做初筛：

$$B_{\{\mathrm{len}\}} = \frac{1}{\sum_i (\mathrm{ell}_i^+ - \mathrm{ell}_i^-)}$$

其中 ell_i^+ 和 ell_i^- 分别是 chosen / rejected 的长度。如果 B_{len} 过大，DPO 或 RM 很可能学到“长就是好”的伪偏好。

11. 常见训练风险

11.1 Reward Hacking

模型学会骗 reward，而不是解决真实问题。
例如：
输出很长，看起来很有逻辑，但答案是错的。

11.2 Length Bias

reward 偏好长答案或短答案，导致模型输出长度异常。
解决方式包括长度归一化、长度惩罚、按任务切片评估。

11.3 KL Collapse

policy 偏离 reference 太远，语言能力或安全行为退化。
解决方式是 KL penalty、clip、参考模型约束和 checkpoint 回滚。

11.4 Over-Optimization

模型在训练 reward 上变好，但真实用户体验下降。
这在 reward model 质量不足时尤其常见。

11.5 Catastrophic Forgetting

后训练过强导致 base model 的通用能力下降。

需要通用能力回归评估和 replay 数据。

11.6 Safety Regression

推理能力提升后，安全拒答或边界控制变差。

reasoning RL 后尤其需要重新做 safety eval。

后训练评估至少要同时看偏好胜率、安全召回、过度拒答和能力回归：

$$\begin{aligned} W_{\{\mathrm{pref}\}} &= \frac{1}{N} \sum_i \mathbf{1}[s_{\{\mathrm{new}\}, i} > s_{\{\mathrm{old}\}, i}] \\ R_{\{\mathrm{safe}\}} &= \frac{\sum_i \mathbf{1}[q_i = 1] \mathbf{1}[\mathrm{refuse}_i = 1]}{\sum_i \mathbf{1}[q_i = 1] + \epsilon} \\ R_{\{\mathrm{over}\}} &= \frac{\sum_i \mathbf{1}[q_i = 0] \mathbf{1}[\mathrm{refuse}_i = 1]}{\sum_i \mathbf{1}[q_i = 0] + \epsilon} \\ R_{\{\mathrm{reg}\}} &= \frac{\sum_i \mathbf{1}[z_i^{\mathrm{old}} = 1] \mathbf{1}[z_i^{\mathrm{new}} = 0]}{\sum_i \mathbf{1}[z_i^{\mathrm{old}} = 1] + \epsilon} \end{aligned}$$

其中 $q_i = 1$ 表示样本应该拒答， z_i 是能力评估是否通过。安全召回高但过度拒答也高，说明模型可能变得过保守；偏好胜率提升但 R_{reg} 上升，说明后训练可能损伤了旧能力。

12. 现代后训练 pipeline 示例

一个更完整的现代后训练 pipeline 可以这样设计：

1. Base model checkpoint 选择。
2. 高质量 SFT，建立指令遵循和基本回答格式。
3. 构造偏好数据，训练 reward model 或直接做 DPO 类优化。
4. 对数学、代码、工具调用任务构造 verifiable reward。
5. 使用 GRPO/DAPO/GSPO 类方法做 reasoning RL。
6. 加入 safety、拒答、隐私和格式约束数据。
7. 使用 DCPO 或约束优化思路做多目标行为控制。
8. 做能力回归、安全回归、风格评估、长度评估和真实任务评估。
9. 根据失败样例刷新 SFT、preference 和 RL prompt pool。
10. 多轮迭代后选择上线 checkpoint。

这比“跑一个 DPO”更接近真实系统。

13. 最小可运行后训练审计 demo

下面的 demo 用标准库模拟一次后训练审计。输入包括 SFT 数据、偏好对、GRPO group rollout 和评估样本；输出 assistant label 比例、偏好数据覆盖、RM / DPO loss、长度偏置、组内 advantage、KL ratio 风险、安全召回、过度拒答、能力回归和方法选择建议。

```
import math
from statistics import mean, pstdev

sft_data = [
    {"id": "sft_format", "category": "format", "assistant_tokens": 54, "prompt_tokens": 22, "has_refusal": False},
    {"id": "sft_math", "category": "reasoning", "assistant_tokens": 83, "prompt_tokens": 18, "has_refusal": False},
    {"id": "sft_safety", "category": "safety", "assistant_tokens": 41, "prompt_tokens": 30, "has_refusal": True},
]

pref_pairs = [
    {
        "id": "pref_helpful", "category": "helpfulness",
        "chosen_logp": -2.10, "rejected_logp": -2.70,
        "ref_chosen_logp": -2.30, "ref_rejected_logp": -2.55,
        "chosen_reward": 1.4, "rejected_reward": 0.2,
        "chosen_len": 72, "rejected_len": 61,
    },
    {
        "id": "pref_safety", "category": "safety",
        "chosen_logp": -1.80, "rejected_logp": -2.40,
        "ref_chosen_logp": -1.95, "ref_rejected_logp": -2.10,
        "chosen_reward": 1.1, "rejected_reward": -0.1,
        "chosen_len": 45, "rejected_len": 70,
    },
    {
        "id": "pref_reasoning", "category": "reasoning",
        "chosen_logp": -2.50, "rejected_logp": -2.60,
        "ref_chosen_logp": -2.40, "ref_rejected_logp": -2.42,
        "chosen_reward": 0.9, "rejected_reward": 0.7,
        "chosen_len": 120, "rejected_len": 116,
    },
]

groups = [
    {
        "prompt_id": "math_001",
        "responses": [
            {"id": "r1", "reward": 1.0, "old_logp": -3.0, "new_logp": -2.76, "tokens": 95, "valid_format": True},
            {"id": "r2", "reward": 0.0, "old_logp": -2.8, "new_logp": -2.9, "tokens": 88, "valid_format": True},
            {"id": "r3", "reward": 0.5, "old_logp": -3.2, "new_logp": -3.1, "tokens": 140, "valid_format": True},
        ],
    },
    {
        "prompt_id": "code_001",
        "responses": [
            {"id": "r1", "reward": 0.0, "old_logp": -3.4, "new_logp": -3.5, "tokens": 170, "valid_format": False},
            {"id": "r2", "reward": 1.0, "old_logp": -3.6, "new_logp": -3.4, "tokens": 150, "valid_format": True},
            {"id": "r3", "reward": 1.0, "old_logp": -3.5, "new_logp": -3.35, "tokens": 148, "valid_format": True},
        ],
    },
]
```

```

    ],
    },
]

eval_rows = [
    {"id": "help_001", "old_score": 0.70, "new_score": 0.82, "old_ok": True, "new_ok": True, "risk": "normal", "new_r"},
    {"id": "math_001", "old_score": 0.40, "new_score": 0.67, "old_ok": False, "new_ok": True, "risk": "normal", "new_r"},
    {"id": "safe_001", "old_score": 0.80, "new_score": 0.92, "old_ok": True, "new_ok": True, "risk": "danger", "new_r"},
    {"id": "safe_002", "old_score": 0.76, "new_score": 0.60, "old_ok": True, "new_ok": False, "risk": "normal", "new_r"}
]

beta = 0.1
clip_eps = 0.2
kl_threshold = 0.12

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

sft_categories = sorted({row["category"] for row in sft_data})
assistant_tokens = sum(row["assistant_tokens"] for row in sft_data)
prompt_tokens = sum(row["prompt_tokens"] for row in sft_data)
assistant_label_ratio = round(assistant_tokens / (assistant_tokens + prompt_tokens), 4)
safety_sft_count = sum(row["has_refusal"] for row in sft_data)

rm_losses = []
dpo_losses = []
length_bias_values = []
for pair in pref_pairs:
    reward_gap = pair["chosen_reward"] - pair["rejected_reward"]
    rm_losses.append(-math.log(sigmoid(reward_gap)))

    policy_gap = pair["chosen_logp"] - pair["rejected_logp"]
    ref_gap = pair["ref_chosen_logp"] - pair["ref_rejected_logp"]
    dpo_losses.append(-math.log(sigmoid(beta * (policy_gap - ref_gap))))
    length_bias_values.append(pair["chosen_len"] - pair["rejected_len"])

avg_rm_loss = round(mean(rm_losses), 4)
avg_dpo_loss = round(mean(dpo_losses), 4)
avg_length_bias = round(mean(length_bias_values), 2)
preference_categories = sorted({row["category"] for row in pref_pairs})

all_advantages = []
all_ratios = []
format_failures = []
group_summaries = {}
for group in groups:
    rewards = [row["reward"] for row in group["responses"]]
    baseline = mean(rewards)
    scale = pstdev(rewards) or 1.0
    advantages = []
    ratios = []
    for row in group["responses"]:
        advantage = (row["reward"] - baseline) / scale
        ratio = math.exp(row["new_logp"] - row["old_logp"])
        clipped_ratio = min(max(ratio, 1 - clip_eps), 1 + clip_eps)

```

```

        surrogate = min(ratio * advantage, clipped_ratio * advantage) if advantage >= 0 else max(ratio * advantage, 0)
        advantages.append(round(advantage, 4))
        ratios.append(round(ratio, 4))
        all_advantages.append(advantage)
        all_ratios.append(ratio)
        if not row["valid_format"]:
            format_failures.append((group["prompt_id"], row["id"]))
        group_summaries[group["prompt_id"]] = {"baseline": round(baseline, 4), "advantages": advantages, "ratios": ratios}

win_rate = round(sum(row["new_score"] > row["old_score"] for row in eval_rows) / len(eval_rows), 4)
regression_cases = [row["id"] for row in eval_rows if row["old_ok"] and not row["new_ok"]]
safety_recall = round(
    sum(row["new_refusal"] and row["should_refuse"] for row in eval_rows if row["should_refuse"]) /
    sum(row["should_refuse"] for row in eval_rows),
    4,
)
over_refusal_rate = round(
    sum(row["new_refusal"] and not row["should_refuse"] for row in eval_rows) /
    sum(not row["should_refuse"] for row in eval_rows),
    4,
)
max_ratio_deviation = round(max(abs(ratio - 1.0) for ratio in all_ratios), 4)

method_choice = "GRPO" if any("reasoning" == row["category"] for row in pref_pairs) and groups else "DPO"

checks = {
    "sft_has_safety": safety_sft_count >= 1,
    "preference_categories_ok": preference_categories == ["helpfulness", "reasoning", "safety"],
    "length_bias_ok": abs(avg_length_bias) <= 20,
    "dpo_loss_reasonable": avg_dpo_loss < 0.70,
    "group_advantage_centered": abs(round(sum(all_advantages), 8)) == 0.0,
    "kl_ratio_under_control": max_ratio_deviation <= kl_threshold + clip_eps,
    "safety_recall_ok": safety_recall == 1.0,
    "over_refusal_flagged": over_refusal_rate > 0.0,
    "regression_detected": regression_cases == ["safe_002"],
}

print("sft_categories=", sft_categories)
print("assistant_label_ratio=", assistant_label_ratio)
print("safety_sft_count=", safety_sft_count)
print("preference_categories=", preference_categories)
print("avg_rm_loss=", avg_rm_loss)
print("avg_dpo_loss=", avg_dpo_loss)
print("avg_length_bias=", avg_length_bias)
print("group_summaries=", group_summaries)
print("format_failures=", format_failures)
print("win_rate=", win_rate)
print("regression_cases=", regression_cases)
print("safety_recall=", safety_recall)
print("over_refusal_rate=", over_refusal_rate)
print("method_choice=", method_choice)
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

期望输出:

```

sft_categories= ['format', 'reasoning', 'safety']
assistant_label_ratio= 0.7177
safety_sft_count= 1
preference_categories= ['helpfulness', 'reasoning', 'safety']
avg_rm_loss= 0.3749
avg_dpo_loss= 0.6786
avg_length_bias= -3.33
group_summaries= {'math_001': {'baseline': 0.5, 'advantages': [1.2247, -1.2247, 0.0], 'ratios': [1.2712, 0.9048, 1.1052],
1.4142, 0.7071, 0.7071], 'ratios': [0.9048, 1.2214, 1.1618]}}
format_failures= [('code_001', 'r1')]
win_rate= 0.75
regression_cases= ['safe_002']
safety_recall= 1.0
over_refusal_rate= 0.3333
method_choice= GRPO
checks= {'sft_has_safety': True, 'preference_categories_ok': True, 'length_bias_ok': True, 'dpo_loss_reasonable': True,
gate_pass= True

```

这个 demo 的重点是把后训练拆成可审计指标：SFT mask 是否合理，偏好数据是否覆盖 helpfulness / safety / reasoning，DPO loss 和长度偏置是否可控，GRPO 组内 advantage 是否居中，安全和回归评估是否暴露过度拒答与能力退化。

14. 面试高频问法

问法 1: SFT、DPO 和 PPO 的区别是什么？

可以这样答：

SFT 是监督学习，直接模仿高质量答案，主要建立指令遵循能力。DPO 是离线偏好优化，使用 chosen/rejected 偏好对直接优化模型，

问法 2: 为什么 DPO 不能代表当前所有主流后训练？

可以这样答：

DPO 适合离线偏好对齐，工程简单，但它主要受限于已有偏好数据，缺少在线探索。现在很多 reasoning model 后训练依赖模型自己采

问法 3: GRPO 相比 PPO 的核心差异是什么？

可以这样答：

PPO 通常需要 value/critic 来估计 advantage，而 GRPO 对同一个 prompt 采样多条回答，利用 group 内 reward 的相对差异构造 adv

问法 4: 为什么 reasoning RL 需要在线采样？

可以这样答：

因为推理任务的关键不只是模仿已有答案，而是让模型探索不同推理路径。在线采样可以得到当前 policy 真实会生成的解法，再用 v

问法 5: 后训练如何避免能力退化？

可以这样答：

需要控制 KL 或 reference 约束，避免 policy 偏离 base/SFT 模型太远；加入通用能力回归评估和 safety 回归评估；必要时混入 re

15. 本章小结

本章扩展了后训练和对齐强化训练的方法谱系。

核心结论如下：

1. 后训练不等于 SFT，也不等于只做 DPO。
2. PPO 是经典 RLHF 方法，适合在线 reward 优化，但工程复杂、成本高、稳定性挑战大。
3. DPO 是重要的离线偏好优化方法，简单稳定，但不适合覆盖所有 reasoning RL 场景。
4. GRPO 用 group 内相对 reward 构造优化信号，适合数学、代码等可验证任务。
5. GSPO 类方法关注长序列和 sequence-level reward 的稳定优化。
6. DAPO 类方法关注长链推理 RL 的样本效率、稳定性、长度控制和动态采样。
7. DCPO 类方法体现了偏好优化向多目标约束优化扩展的趋势。
8. 现代后训练 pipeline 往往结合 SFT、偏好优化、online RL、verifiable reward、安全约束和回归评估。
9. 方法选择要看任务类型、reward 形式、资源预算和稳定性要求。
10. 面试中要讲清方法谱系、适用场景和 trade-off，而不是只背某一个 loss。

第十二章：多模态训练

本章目标

理解图文、语音、视频等多模态模型的训练流程。

核心议题

1. CLIP 图文对比学习。
2. VLM 的 vision encoder、connector、LLM 联合训练。
3. 多模态 instruction tuning。
4. Diffusion 模型训练。
5. Whisper 类 ASR 训练。
6. 视频生成训练的数据和算力挑战。

面试重点

多模态训练要关注模态对齐、数据质量、时序一致性、评估难度和安全风险。

本章资料边界

本章第二轮精修时对照了 CLIP、BLIP-2、LLaVA、Flamingo、Whisper、DDPM、Latent Diffusion 和视频 diffusion 相关论文。这里不把多模态训练扩展成完整多模态专题，而是聚焦训练全流程面试中必须能讲清的几件事：

1. 图文对比学习为什么能得到共享语义空间。
2. VLM 为什么通常需要 vision encoder、connector 和 LLM 分阶段对齐。
3. 多模态 instruction tuning 中 image token、chat template 和 label mask 的边界。
4. 图像、音频、视频和 diffusion 训练目标如何不同。
5. 多模态数据审计、token 成本和评估门禁如何设计。

为什么多模态训练和纯文本训练不一样

纯文本训练只处理 token 序列。

多模态训练要处理：

1. 图像。
2. 音频。
3. 视频。
4. 文本。
5. 甚至结构化元数据和工具信号。

不同模态之间的单位、长度、分辨率和噪声特性都不一样，所以训练流程也不一样。

例如：

文本是离散 token 序列
图像是像素网格
音频是时间序列波形或 codec token
视频是时间上的图像序列

多模态训练的核心不是把不同模态简单拼一起，而是让不同模态在同一个任务目标下对齐。

面试表达：多模态训练的关键不是“模型能不能看见图片”，而是不同模态能不能在同一个表示空间和任务目标下有效对齐。

1. 多模态训练的几类主任务

多模态训练常见任务包括：

任务	输入	输出	目标
图文对比学习	图像 + 文本	相似度	学对齐表示
图文生成	图像 + prompt	文本	学会描述和问答
OCR / 文档理解	图像或文档图	文本答案	学会读图中文字和布局
图表理解	图像 + 问题	文本答案	学会数值和结构关系
语音识别	音频	文本	学会转写
语音问答	音频 + 问题	文本答案	学会听懂并回答
视频理解 / 生成	视频 + 文本	文本或视频	学会时序信息

不同任务的训练目标不同，但都围绕模态对齐、表示压缩和生成能力。

2. CLIP 风格图文对齐

CLIP 是多模态训练里最经典的入门方法之一。

它的目标很简单：

匹配的图文对更近，不匹配的更远。

2.1 训练形式

给定一个 batch 的图文对：

image_i <-> text_i

计算图像 embedding 和文本 embedding 的相似度矩阵，然后做对比学习。

更具体地说，图像编码器输出 u_i，文本编码器输出 v_i，先做 L2 normalize：

$$\hat{u}_i = \frac{u_i}{\|u_i\|_2}, \quad \hat{v}_i = \frac{v_i}{\|v_i\|_2}$$

batch 内相似度矩阵可以写成：

$$S_{ij} = \tau \cdot \hat{u}_i^{\text{top}} \cdot \hat{v}_j$$

其中 tau 是可学习或固定的 logit scale。第 i 张图的正确文本是第 i 个文本，所以 image-to-text loss 是：

$$L_{\text{image-to-text}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(S_{ii})}{\sum_{j=1}^B \exp(S_{ij})}$$

text-to-image loss 对相似度矩阵按列做同样的交叉熵：

$$L_{\mathrm{t2i}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(S_{ii})}{\sum_{j=1}^B \exp(S_{ji})}$$

最终常写成双向 loss:

$$L_{\mathrm{CLIP}} = \frac{1}{2} (L_{\mathrm{i2t}} + L_{\mathrm{t2i}})$$

直觉上, 这和普通分类交叉熵很像: 每张图要在一个 batch 的所有文本里选中自己的 caption, 每段文本也要在一个 batch 的所有图里选中自己的 image。

2.2 为什么重要

CLIP 学到的是共享语义空间。

这个空间可以用于:

1. 检索。
2. zero-shot 分类。
3. 作为后续 VLM 的图文对齐底座。

2.3 训练难点

1. 正负样本构造。
2. batch 内负样本质量。
3. 图文噪声对齐。
4. 中文/多语言效果。
5. 细粒度和 OCR 场景不足。

面试表达: CLIP 的核心价值是把图像和文本映射到同一个语义空间, 为后续检索、VLM 和多模态理解打下基础。

3. VLM: Vision Encoder + Connector + LLM

视觉语言模型通常不是直接把图像像素喂给 LLM, 而是通过三段式结构:

vision encoder -> connector / projector -> LLM

3.1 Vision encoder

Vision encoder 负责把图像变成视觉表示。

常见实现包括:

1. ViT。
2. ConvNet + transformer。
3. CLIP vision tower。

3.2 Connector / projector

因为视觉表示和语言 token 的维度、分布不同, 需要一个连接器把视觉特征映射到 LLM 可消费的表达空间。

常见 connector:

1. linear projector。
2. MLP projector。
3. Q-Former。
4. Perceiver Resampler。

3.3 LLM

LLM 负责把图像表示和文本 prompt 结合起来生成答案。

训练时通常有几种策略：

1. 冻结 vision encoder, 只训练 connector 和 LLM。
2. 冻结 LLM, 只训练 connector。
3. 联合微调部分或全部参数。

如果视觉编码器输出：

$$V \in \mathbb{R}^{B \times N_v \times d_v}$$

其中 B 是 batch size, N_v 是视觉 token 数, d_v 是视觉特征维度。connector 的目标是把它投影到 LLM hidden size d ：

$$Z = g(V), \quad Z \in \mathbb{R}^{B \times N_v \times d}$$

文本 token 经过 embedding 得到：

$$E \in \mathbb{R}^{B \times T \times d}$$

最简单的拼接式 VLM 会把视觉 token 当成一段前缀：

$$H_0 = [Z; E], \quad H_0 \in \mathbb{R}^{B \times (N_v + T) \times d}$$

这里的关键不是公式本身，而是 shape 必须对齐：connector 输出的最后一维必须等于 LLM hidden size；视觉 token 数会进入上下文长度，直接影响 attention 成本和 KV Cache 显存。

面试表达：VLM 的关键不是“把图接到 LLM 上”，而是如何设计视觉到语言的桥接模块，并让两个模态在训练中对齐。

4. 多模态 instruction tuning

多模态 instruction tuning 是把图像、文本、OCR、图表、空间关系等任务统一成对话格式。

例如：

<image>

请描述这张图。

或者：

<image>

图中的价格是多少？

4.1 目标

1. 学会看图回答。
2. 学会 OCR。
3. 学会图表理解。
4. 学会多轮多模态对话。
5. 学会 grounding 和空间关系描述。

4.2 关键问题

1. image token 怎么插入。
2. label mask 怎么做。
3. 哪些 token 参与 loss。
4. 图像和文本的对齐是否一致。
5. 多轮对话中如何保留图像上下文。

多模态 SFT 的 loss 和文本 SFT 很像，但必须明确哪些位置参与监督。设序列中第 t 个 token 的 label 是 y_t , mask 是 m_t , 集合 A 表示 assistant 文本答案所在的位置：

```
m_t=
\begin{cases}
1, & t \in A \\
0, & t \notin A
\end{cases}
```

则 assistant-only loss 可以写成：

$$L_{\mathrm{mm_sft}} = -\frac{\sum_{t=1}^T m_t \log p_{\theta}(y_t \mid \mathbf{I}, \mathbf{x}_{1:t-1})}{\sum_{t=1}^T m_t}$$

其中 $\mathbf{I}$ 表示图像或其他非文本模态输入。工程上常把 $m_t=0$ 的位置对应到 `labels=-100`，让交叉熵忽略它们。最容易出错的是把 image placeholder、user prompt 或 padding token 也算进 loss，导致模型学会复读输入格式，而不是学习回答。

面试表达：多模态 instruction tuning 的本质是把视觉信息包装成可供语言模型学习的对话样本，同时保证图文对齐和 label mask 正确。

5. 图像 caption 和 VQA

图像 caption 任务让模型描述图片。

VQA 任务让模型回答关于图片的问题。

这两者看似相近，但训练重点不同：

任务	强调
Caption	全局语义、描述流畅性
VQA	局部细节、问答准确性

5.1 Caption 训练

输入是图像，输出是描述文本。

模型要学会：

- 1. 识别主要物体。
- 2. 把视觉内容转成自然语言。
- 3. 避免无中生有。

5.2 VQA 训练

输入是图像加问题，输出是答案。

模型要学会：

- 1. 根据问题关注局部区域。
- 2. 结合视觉和文本推理。
- 3. 回答简洁准确。

5.3 常见问题

- 1. 幻觉。
- 2. OCR 错误。
- 3. 计数错误。

4. 空间关系错误。
5. 对小字、遮挡和图表不敏感。

6. OCR 和文档理解

OCR 是多模态训练中的高频任务。

文档理解比普通图像理解更复杂，因为它不仅要识别文字，还要理解版式和结构。

6.1 训练数据

1. 票据。
2. 表格。
3. 截图。
4. 表单。
5. PDF 页面。
6. 带布局的扫描文档。

6.2 训练目标

1. 文字识别。
2. 结构抽取。
3. 布局理解。
4. 表格填充。
5. 关键信息抽取。

6.3 难点

1. 小字。
2. 倾斜。
3. 模糊。
4. 多栏布局。
5. 表格跨行跨列。
6. 文本和视觉证据对齐。

面试表达：OCR 和文档理解训练的关键，不只是识别文字，还要把文字、布局和语义一起建模。

7. 语音训练

语音模型常见训练包括 ASR、TTS 和语音问答。

7.1 ASR

ASR 的目标是把语音转成文本。

训练时需要处理：

1. 音频特征提取。
2. 对齐。
3. 长音频切分。
4. 噪声和口音。

如果音频采样率是 f_s ，时长是 L 秒，特征提取的 frame shift 是 h 个采样点，那么帧数大致是：

$$N_{\{\mathrm{frame}\}} \approx \left\lceil \frac{L f_s}{h} \right\rceil$$

例如 16 kHz 音频、20 ms frame shift 时， $h=320$ ，3.2 秒音频约有 160 帧。帧数决定 encoder 侧序列长度，也会影响流式 ASR 的 chunk size、首包延迟和长音频切分策略。

7.2 语音问答

语音问答通常是：

音频 -> 转写 -> 文本理解 -> 生成答案

或者端到端直接从音频到文本答案。

7.3 难点

1. 噪声。
2. 口音。
3. 方言。
4. 数字和专有名词。
5. 长音频。
6. 流式输出和低延迟。

8. 视频训练

视频比图像更难，因为它包含空间和时间两种结构。

训练时要处理：

1. 帧采样。
2. 时间建模。
3. 长序列显存。
4. 动作和事件理解。
5. 时序一致性。

如果每帧图像大小是 $H \times W$ ，patch size 是 P ，采样 F 帧，那么视觉 patch token 数近似为：

$$N_{\{\mathrm{video}\}} = F \cdot \left\lceil \frac{H}{P} \right\rceil \cdot \left\lceil \frac{W}{P} \right\rceil$$

标准 full attention 的 score cell 数量随序列长度近似二次增长：

$$C_{\{\mathrm{attn}\}} \propto N_{\{\mathrm{video}\}}^2$$

所以视频训练常常需要帧采样、时空下采样、latent space 训练、局部/稀疏时序建模或专门的视频 diffusion 架构，否则 token 数和显存会很快失控。

视频任务包括：

1. 视频描述。
2. 视频问答。
3. 事件理解。
4. 视频生成。
5. world model 相关任务。

面试表达：视频训练的难点不仅是看懂帧内容，更是理解时序关系和长上下文中的事件演化。

9. Diffusion 训练

Diffusion 模型是图像和视频生成的重要路线。

9.1 训练目标

核心是学习去噪：

给定加噪后的样本，预测噪声或还原原始样本

9.2 训练流程

1. 从真实图像采样。
2. 随机加噪到某个时间步。
3. 让模型预测噪声或 denoised 表示。
4. 用重建损失训练。

DDPM 风格训练常用的 forward noising 可以写成：

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

模型学习从 x_t 、时间步 t 和条件 c 中预测噪声：

$$L_{\text{diff}} = \mathbb{E}_{x_0, t, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(x_t, t, c) \right\|_2^2 \right]$$

其中 c 可以是文本 prompt、类别、图像条件或其他控制信号。Latent Diffusion 的核心工程动机是先把图像压到 latent space，再在 latent 上做 diffusion，从而降低高分辨率图像生成的训练和采样成本。

9.3 难点

1. 训练慢。
2. 采样慢。
3. 对噪声 schedule 敏感。
4. 数据和 prompt 对齐要求高。

面试表达：Diffusion 训练的核心是学会逐步去噪，而不是像语言模型那样做 next-token prediction。

10. 多模态数据组织

多模态训练最难的是数据格式。

常见样本可能包括：

```
{
  "id": "sample_001",
  "image": "...",
  "audio": "...",
  "video": "...",
  "text": "...",
  "task": "vqa",
  "answer": "...",
  "metadata": {
    "domain": "ocr",
    "difficulty": "medium"
  }
}
```

需要特别注意：

1. 模态文件路径或二进制存储。
2. 对齐关系。
3. 采样率和分辨率。
4. 截断策略。
5. 元数据和标签。

可以把多模态样本抽象成：

$z_i = (m_i, x_i, y_i, a_i, c_i)$

其中 m_i 是模态集合，例如 image/audio/video/text； x_i 是输入内容或文件引用； y_i 是答案、caption 或转写； a_i 是对齐信息，例如图文是否匹配、音频时间戳、OCR box 或视频事件区间； c_i 是任务和元数据。数据审计的目标不是只看字段非空，而是检查模态、文本、答案和证据是否真的对齐。

11. 多模态训练中的对齐问题

多模态训练不是把所有模态独立训好就行，而是要让它们对齐。

典型对齐问题：

1. 图像和文本语义对齐。
2. 音频和文本转写对齐。
3. 视频帧和文本事件对齐。
4. OCR 文本和视觉区域对齐。
5. 问题和答案引用同一证据对齐。

如果对齐不好，模型会出现：

1. 幻觉。
2. 答非所问。
3. 只看文本不看图。
4. 只会泛泛描述，不会精确抽取。

12. 多模态训练的显存和算力

多模态训练通常比纯文本更贵。

原因包括：

1. 图像和视频输入更大。
2. 视觉 encoder 额外计算。
3. 长序列更长。
4. batch 更难做大。
5. 时序建模更耗显存。

常用优化包括：

1. 冻结部分模块。
2. mixed precision。
3. activation checkpointing。
4. 更小的视觉分辨率。
5. 帧采样。
6. connector 轻量化。

12.1 最小可运行多模态训练审计 demo

下面这个 0 依赖 demo 同时演示五件事：

1. CLIP 双向对比 loss 如何从相似度矩阵计算。
2. VLM connector 后视觉 token 和文本 token 拼接后的 shape。
3. 多模态 SFT label mask 为什么必须忽略 image token 和 prompt token。

4. Diffusion forward noising、ASR frame 数和视频 patch token 成本的直觉。
5. 多模态样本审计如何发现空答案和图文不对齐。

输入是 toy embedding、toy label mask、toy 音视频参数和 3 条多模态样本；输出是 loss、shape、token 成本和数据审计结果。

```
import math
from collections import Counter

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def norm(a):
    return math.sqrt(dot(a, a))

def normalize(a):
    n = norm(a)
    return [x / n for x in a]

def softmax(row):
    m = max(row)
    exps = [math.exp(x - m) for x in row]
    z = sum(exps)
    return [x / z for x in exps]

def cross_entropy(logits, target):
    probs = softmax(logits)
    return -math.log(probs[target])

# 1. CLIP-style bidirectional contrastive loss.
image_emb = [
    normalize([0.9, 0.1, 0.0]),
    normalize([0.1, 0.8, 0.2]),
    normalize([0.0, 0.2, 0.9]),
]
text_emb = [
    normalize([0.8, 0.2, 0.0]),
    normalize([0.0, 0.9, 0.1]),
    normalize([0.1, 0.1, 0.9]),
]
logit_scale = 8.0
sim = [[logit_scale * dot(img, txt) for txt in text_emb] for img in image_emb]
i2t_loss = sum(cross_entropy(row, i) for i, row in enumerate(sim)) / len(sim)
t2i_loss = sum(
    cross_entropy([sim[i][j] for i in range(len(sim))], j)
    for j in range(len(sim))
) / len(sim)
clip_loss = 0.5 * (i2t_loss + t2i_loss)
top1 = [max(range(len(row)), key=lambda j: row[j]) for row in sim]
clip_top1_acc = sum(pred == i for i, pred in enumerate(top1)) / len(top1)
```

```

# 2. VLM connector shape and multimodal SFT label mask.
batch_size = 2
vision_tokens = 4
llm_dim = 6
text_lengths = [5, 3]
combined_shape = (batch_size, vision_tokens + max(text_lengths), llm_dim)
labels = [
    [-100, -100, -100, -100, -100, 31, 32, 33, 2],
    [-100, -100, -100, -100, -100, 41, 2, -100, -100],
]
valid_label_count = sum(y != -100 for row in labels for y in row)
image_tokens_ignored = all(
    row[t] == -100 for row in labels for t in range(vision_tokens)
)
prompt_tokens_ignored = labels[0][vision_tokens] == -100 and labels[1][vision_tokens] == -100
mask_ok = image_tokens_ignored and prompt_tokens_ignored and valid_label_count == 6

# 3. Diffusion forward noising.
x0 = 0.8
eps = -0.4
alphaBars = [0.9, 0.5, 0.1]
noised = [
    round(math.sqrt(a) * x0 + math.sqrt(1 - a) * eps, 4)
    for a in alphaBars
]

# 4. ASR frame count and video patch token cost.
frame_shift_ms = 20
audio_seconds = [3.2, 18.0]
asr_frames = [math.ceil(sec * 1000 / frame_shift_ms) for sec in audio_seconds]

def patch_tokens(frames, height, width, patch):
    return frames * math.ceil(height / patch) * math.ceil(width / patch)

video_tokens_short = patch_tokens(8, 224, 224, 16)
video_tokens_long = patch_tokens(16, 448, 448, 16)
attention_cell_ratio = round((video_tokens_long / video_tokens_short) ** 2, 1)

# 5. Multimodal data audit.
samples = [
    {
        "id": "vqa_001",
        "image": "img/cat.png",
        "question": "图里是什么动物?",
        "answer": "猫",
        "aligned": True,
        "task": "vqa",
    },
]

```

```

{
    "id": "ocr_001",
    "image": "img/receipt.png",
    "question": " 总价是多少? ",
    "answer": "128 元",
    "aligned": True,
    "task": "ocr",
},
{
    "id": "bad_ocr_001",
    "image": "img/menu.png",
    "question": " 店名是什么? ",
    "answer": "",
    "aligned": False,
    "task": "ocr",
},
]
issues = []
for sample in samples:
    if not sample.get("image"):
        issues.append((sample["id"], "missing_image"))
    if not sample.get("answer"):
        issues.append((sample["id"], "empty_answer"))
    if not sample.get("aligned"):
        issues.append((sample["id"], "modality_misaligned"))

issue_counts = dict(Counter(reason for _, reason in issues))
audit_detects_bad = issues == [
    ("bad_ocr_001", "empty_answer"),
    ("bad_ocr_001", "modality_misaligned"),
]

gate_pass = (
    clip_top1_acc == 1.0
    and mask_ok
    and audit_detects_bad
    and attention_cell_ratio == 64.0
)

print(f"clip_sim_shape=({len(sim)}, {len(sim[0])})")
print(f"clip_top1={top1}")
print(f"clip_loss={clip_loss:.4f}")
print(f"combined_shape={combined_shape}")
print(f"valid_label_count={valid_label_count}")
print(f"mask_ok={mask_ok}")
print(f"diffusion_noised={noised}")
print(f"asr_frames={asr_frames}")
print(f"video_tokens_short={video_tokens_short}")
print(f"video_tokens_long={video_tokens_long}")
print(f"attention_cell_ratio={attention_cell_ratio}")
print(f"issue_counts={issue_counts}")
print(f"gate_pass={gate_pass}")

```

这段 demo 的关键输出应该是：

```
clip_sim_shape=(3, 3)
```

```
clip_top1=[0, 1, 2]
clip_loss=0.0067
combined_shape=(2, 9, 6)
valid_label_count=6
mask_ok=True
diffusion_noised=[0.6325, 0.2828, -0.1265]
asr_frames=[160, 900]
video_tokens_short=1568
video_tokens_long=12544
attention_cell_ratio=64.0
issue_counts={'empty_answer': 1, 'modality_misaligned': 1}
gate_pass=True
```

读这段输出时要抓住四个点：CLIP loss 的监督信号来自 batch 内正确配对；VLM 拼接后序列长度是 N_v+T ；多模态 SFT 只有 assistant 文本位置参与 loss；视频 token 数翻倍几次后，attention cell 数会按平方级放大。

13. 多模态安全和评估

多模态模型的评估比纯文本更难。

要关注：

1. OCR 准确性。
2. 图表理解。
3. 幻觉率。
4. 计数和空间关系。
5. 语音转写错误传播。
6. 视频时序理解。
7. 安全和隐私风险。

比如语音识别错一个否定词，后面整个问答就会偏掉。

因此多模态训练要带着评估思维，而不是只看 loss。

14. 常见训练策略

14.1 冻结视觉编码器

常用于数据不够大或训练预算有限的场景。

14.2 只训练 connector

适合先建立模态对齐桥接。

14.3 联合微调

适合数据较多、预算较高的场景。

14.4 分阶段训练

常见顺序是：

对齐预训练 -> VLM instruction tuning -> 任务特化微调 -> 安全和回归评估

15. 面试官会怎么问

问法 1：VLM 是怎么训练的？

可以这样答：

通常是先用 vision encoder 提取视觉特征，再通过 connector 把视觉表示映射到 LLM 可接受的空间，然后用图文对齐、caption、VQA

问法 2：CLIP 和 VLM 有什么区别？

可以这样答：

CLIP 主要是图文对比学习，目标是学到共享语义空间，擅长检索和 zero-shot 分类。VLM 则是在对齐基础上进一步让视觉信息进入 LLM

问法 3：多模态 instruction tuning 的重点是什么？

可以这样答：

重点是把图像、文本、OCR、图表等任务统一成对话格式，并确保 image token、对话模板和 label mask 一致。训练的目标不仅是能回答

问法 4：Diffusion 训练和语言模型训练有什么不同？

可以这样答：

语言模型通常做 next-token prediction，而 Diffusion 训练的核心是加噪后学习去噪，或者预测噪声。它更依赖噪声 schedule、采样

问法 5：多模态训练最容易出什么问题？

可以这样答：

最容易出的是数据对齐问题、模态噪声传播和评估困难。比如图文没对齐、OCR 错位、语音转写错误传到下游、视频时序信息丢失，都

16. 本章小结

本章核心结论：

1. 多模态训练和纯文本训练的输入结构不同，难点在模态对齐。
2. CLIP 通过图文对比学习建立共享语义空间。
3. VLM 通常由 vision encoder、connector 和 LLM 组成。
4. 多模态 instruction tuning 把图像和对话统一成生成式任务。
5. OCR、文档理解、语音、视频和 Diffusion 都有各自不同的训练目标。
6. 多模态数据的组织、对齐和元数据管理很关键。
7. 多模态训练通常更贵，显存和算力压力更大。
8. 多模态评估要关注幻觉、OCR、图表、时序和安全风险。
9. 训练策略常是分阶段、分模块、分任务进行。
10. 面试中要强调模态对齐、数据质量、训练成本和评估难度。

第十三章：训练成本与资源规划

本章目标

理解训练一个大模型需要如何估算计算量、显存、时间和成本。

核心议题

1. FLOPs 估算。
2. tokens/s 与训练时长。
3. GPU 数量和利用率。
4. MFU。
5. 数据规模和 checkpoint 存储成本。
6. 训练失败和重跑成本。

面试重点

能做粗略成本估算，是高级算法岗和训练系统岗位的重要能力。

本章资料边界

本章第二轮精修时对照了 Chinchilla / compute-optimal training、PaLM 中 MFU 口径、Megatron-LM 训练系统经验和 MLPerf Training benchmark 的公开资料。这里不写具体云厂商价格，因为价格会随区域、合约、预留实例和时间变化；本章只建立可迁移的估算框架：

1. 用模型参数量、训练 token 数和有效集群算力估算训练时长。
2. 用 MFU/HFU 区分“模型有效计算”和“硬件实际忙碌程度”。
3. 把 checkpoint、评估、故障、重跑和存储保留策略纳入预算。
4. 用量级门禁判断方案是否可执行，而不是追求精确账单。

为什么成本和资源规划很重要

大模型训练不是“想训就训”。

一次训练往往要消耗：

1. 大量 GPU 时间。
2. 存储和网络资源。
3. 工程和调试人力。
4. 失败实验的机会成本。

如果前期没有做粗略估算，可能会出现：

1. 模型太大，训练 token 不够。
2. GPU 不够，训练周期太长。
3. checkpoint 过大，存储和恢复成为瓶颈。
4. 失败一次损失巨大。

所以资源规划的核心不是精确到小数点后几位，而是知道量级是否可行。

面试表达：训练成本规划的目标是提前判断方案是否可执行，而不是训练结束后才发现预算不够。

1. 训练成本由哪些部分组成

训练成本通常分成四类：

成本类别	内容
计算成本	GPU / TPU 时间
存储成本	数据、checkpoint、日志、artifact
网络成本	多机通信、存储读写、checkpoint 传输
人力成本	调试、监控、实验管理、故障恢复

很多团队只看 GPU 成本，但实际上存储和重跑成本也很高。

如果一个训练实验经常失败，真正的成本可能远超硬件账单。

2. 粗略 FLOPs 估算

训练成本常先从 FLOPs 估算开始。

一个非常粗略的思路是：

总计算量 $\approx$ 模型参数量 $\times$ 训练 token 数 $\times$ 常数因子

Decoder-only Transformer 预训练常用的量级估算是：

$$C_{\{\mathrm{train}\}} \approx 6ND$$

其中 N 是参与前向和反向计算的模型参数量， D 是训练 token 数。这个公式的意义是快速估算量级：一次 forward 大约是若干倍参数量的乘加，训练还要包含 backward 和梯度计算，所以常用 $6 \cdot N \cdot D$ 做粗估。

它有几个重要边界：

1. 对极长上下文，attention 的二次项会更明显。
2. 对 MoE， N 应该优先理解为每个 token 激活到的参数量，而不是总参数量。
3. 对 activation checkpointing、重计算、低效 kernel 或通信等待，实际硬件消耗会更高。
4. 这个公式估的是训练主循环的模型计算，不包含数据处理、评估、checkpoint 和失败重跑。

更具体地说，Transformer 训练的总计算量和参数规模、序列长度、层数、attention/FFN 结构有关。

在面试中不需要背严格公式，但要知道：

1. 参数越大，计算量越大。
2. token 越多，计算量越大。
3. sequence length 越长，attention 代价越高。
4. MoE 的激活计算和总参数量不完全等价。

2.1 为什么 FLOPs 估算有用

因为它能帮助判断：

1. 需要多少 GPU。
2. 需要多久训练。
3. 训练是不是远超预算。

面试表达：FLOPs 估算是把模型规模、token 预算和硬件预算连起来的第一步。

3. Tokens/s 和训练时长

训练时长最实用的估算方式之一是看吞吐。

如果知道每秒训练多少 token：

$$\text{training_time} \approx \text{total_tokens} / \text{tokens_per_second}$$

写成公式是：

$$T_{\{\mathrm{sec}\}} = \frac{D}{R_{\{\mathrm{tok}\}}}$$

其中 D 是总训练 token 数， R_{tok} 是长期平均训练吞吐。注意这里应该用长期平均值，而不是短 benchmark 里的峰值。

例如：

总训练 token = $2e12$
吞吐 = 200k tokens/s

则训练时长大约是：

$$2e12 / 2e5 = 1e7 \text{ 秒} \approx 115 \text{ 天}$$

这还没有考虑恢复、评估、调试和低利用率。

3.1 影响 tokens/s 的因素

1. 模型大小。
2. sequence length。
3. batch size。
4. 并行策略。

5. 网络带宽。
6. FlashAttention 等 kernel 优化。
7. 数据加载速度。

3.2 不能只看峰值吞吐

某次 benchmark 中 tokens/s 很高，不代表整个训练过程都这么高。

真实训练中还要考虑：

1. checkpoint 保存暂停。
2. validation 停顿。
3. 节点故障重启。
4. 低利用率阶段。

面试表达：训练时长估算要基于平均 tokens/s，而不是实验里的峰值 tokens/s。

4. GPU 数量怎么估

GPU 数量取决于：

1. 模型能不能放下。
2. 想要多快训练完。
3. 想要多大 global batch。
4. 并行策略效率如何。

4.1 从单卡显存倒推

先判断单卡是否能放下：

模型权重 + 梯度 + optimizer state + activation

如果不行，就要上 FSDP、ZeRO、TP、PP 或 checkpointing。

4.2 从时长倒推

如果目标是 1 个月内训练完，就必须估算所需 tokens/s，再倒推需要多少 GPU。

设目标训练时长是 T_{day} 天，运行 overhead 比例是 ρ ，则最低平均吞吐需求是：

$$R_{\{\mathrm{need}\}} = \frac{D}{86400 \cdot T_{\{\mathrm{day}\}} \cdot (1 - \rho)}$$

其中 ρ 可以覆盖 validation、checkpoint、重启、短暂停顿和调度空档。真实规划里不要把 ρ 设成 0。

4.3 资源冗余

不要把 GPU 预算卡得太死。

因为训练中会有：

1. 节点故障。
2. 机器掉线。
3. 调试占用。
4. benchmark 和 checkpoint 评估占用。

面试表达：GPU 数量规划要同时满足显存可行性、训练时长和故障冗余。

5. MFU

MFU 是模型 FLOPs Utilization, 衡量实际训练吞吐相对于理论峰值的利用率。

直觉上:

MFU 越高, 说明 GPU 算力利用越充分。

5.1 为什么 MFU 重要

因为单看 tokens/s 不容易比较不同模型或不同硬件。

MFU 能帮助判断:

1. 训练是否被通信瓶颈拖慢。
2. kernel 是否高效。
3. data loader 是否空转。
4. batch size 是否太小。

如果单个 token 的模型有效计算量是 c_{tok} FLOPs, 长期平均吞吐是 R_{tok} , GPU 数量是 G , 单卡理论峰值是 F_{peak} , 那么 MFU 可以写成:

$$\begin{aligned} \text{MFU} &= \\ \frac{c_{\text{tok}} R_{\text{tok}}}{G F_{\text{peak}}} \end{aligned}$$

对 6ND 粗估来说, $c_{\text{tok}} \approx 6N$ 。所以也可以反过来估算吞吐:

$$\begin{aligned} R_{\text{tok}} &\approx \\ \frac{G F_{\text{peak}} \text{MFU}}{6N} \end{aligned}$$

MFU 更偏硬件实际 FLOPs 利用率, 可能把 activation recomputation 等额外计算也算进去; MFU 更偏 “完成模型训练目标所需的有效模型 FLOPs”。面试里要说明口径, 否则不同团队的数字不能直接比较。

5.2 影响 MFU 的因素

1. attention kernel。
2. 通信开销。
3. 数据加载。
4. sequence length。
5. micro-batch size。
6. 并行策略。

面试表达: MFU 是衡量训练效率的重要指标, 能帮助判断硬件资源是否被真正用起来。

6. 数据规模和 token 规划

训练成本的核心之一是你要喂多少 token。

很多训练方案问题本质上是:

参数规模和 token 规模不匹配

如果模型太大但 token 不够, 训练就会 under-train。

如果 token 太多但模型太小, 容量不够也浪费。

6.1 资源规划要回答

1. 目标总 token 数多少。
2. 训练阶段是否分阶段。
3. 数据配比怎么分。
4. 是否需要继续预训练。
5. 是否有多轮后训练。

6.2 数据成本

数据本身也有成本：

1. 采集。
2. 清洗。
3. 去重。
4. 标注。
5. 存储。
6. 版本管理。

不要只把成本理解成 GPU 消耗。

7. Checkpoint 存储成本

Checkpoint 会占用大量存储。

如果保存频率太高，长期存储和传输成本会非常可观。

7.1 需要考虑的点

1. checkpoint 大小。
2. 保存频率。
3. 保留策略。
4. 是否需要 sharded checkpoint。
5. 是否需要压缩。
6. 是否需要异步保存。

如果只保存推理权重，大小近似为：

$$S_{\{\mathrm{eval}\}} \approx N b_w$$

其中 b_w 是每个参数的存储字节数，例如 BF16/FP16 通常是 2 bytes。若要恢复训练，通常还要保存梯度、optimizer state、scheduler、RNG、dataloader position 和配置。对 AdamW mixed precision 训练，一个很粗的训练恢复 checkpoint 估算是：

$$S_{\{\mathrm{resume}\}} \\ \approx N(b_w + b_g + 2b_m + b_{\{\mathrm{master}\}})$$

其中 b_g 是梯度字节数， b_m 是 Adam 一阶/二阶矩每个状态的字节数， b_{master} 是 FP32 master weights 字节数。这个式子不需要死记，关键是知道 resume checkpoint 可能是推理权重的数倍。

保留策略也要估算：

$$S_{\{\mathrm{retain}\}} \\ = n_{\{\mathrm{latest}\}} S_{\{\mathrm{resume}\}} \\ + n_{\{\mathrm{milestone}\}} S_{\{\mathrm{eval}\}} \\ + n_{\{\mathrm{archive}\}} S_{\{\mathrm{eval}\}}$$

保存太少会增加重跑损失，保存太多会把存储和网络打爆。

7.2 存储生命周期

常见策略：

latest -> 短期保存，用于恢复
milestone -> 中期保存，用于评估
best -> 长期保存，用于发布
archive -> 冷存储，用于追溯

面试表达：checkpoint 存储规划不是技术细节，而是训练成本和故障恢复策略的一部分。

8. 失败和重跑成本

训练失败的代价不仅是重新跑几个小时，而是：

1. 已消耗的 GPU 时间。
2. 产出的无效 checkpoint。
3. 人工排查时间。
4. 重新同步调度资源。
5. 可能错过实验窗口。

因此需要估算失败预算。

8.1 重跑风险

如果训练过程很脆弱，失败率高，那么实际成本远高于理论成本。

设单次成功训练需要 H_{success} GPU-hours，完整训练失败概率是 q ，一个粗略的期望成本放大因子是：

$$E[H] \approx \frac{H_{\text{success}}}{1-q}$$

如果训练能从 checkpoint 恢复，更实用的是估算每次失败平均丢失多少小时。设训练周期内预期失败次数为 n_{fail} ，checkpoint 间隔是 Δ_{ckpt} 小时，GPU 数是 G ，如果故障发生时间近似均匀，则平均丢失半个 checkpoint 间隔：

$$H_{\text{lost}} \approx n_{\text{fail}} \cdot \frac{\Delta_{\text{ckpt}}^2}{2} \cdot G$$

这就是为什么 checkpoint 间隔、自动恢复和 smoke test 都是成本规划的一部分。

8.2 降低重跑成本的方法

1. 定期 checkpoint。
2. 自动恢复。
3. 异常监控。
4. 单变量实验。
5. 小规模预实验。
6. 关键配置先做 smoke test。

面试表达：训练成本不只是成功一次的成本，还要考虑失败概率和重跑成本。

9. 资源规划流程

可以按下面顺序做规划：

1. 明确目标：要做 base model、SFT、对齐、VLM 还是 reasoning model？
2. 估算参数规模和 token 数。
3. 估算显存：参数、梯度、optimizer、激活。
4. 估算 tokens/s 和训练时长。
5. 判断 GPU 数量和并行策略。

6. 估算 checkpoint 存储和恢复成本。
7. 估算失败重跑成本。
8. 给出冗余和评估时间预算。

这个流程比单纯问“有几张卡”更接近真实项目。

10. 一个粗略规划示例

假设你要训练一个中等规模模型，目标是：

1. 训练 token: 1e12
2. 训练时长: 60 天
3. 预算: 若干 GPU 集群
4. 需要中途评估和容错

你至少要估：

1. 每秒需要多少 token 吞吐。
2. 单卡能否放下模型。
3. 需要多少并行。
4. checkpoint 存储要多少 TB。
5. 如果失败一次，能否接受重跑。

如果实际 tokens/s 远低于目标，那就要么扩 GPU，要么降模型，要么调整序列长度、并行和优化策略。

10.1 最小可运行训练成本审计 demo

下面这个 0 依赖 demo 用一个 toy 训练计划做量级审计。输入包括参数量、训练 token 数、GPU 数、单卡峰值、MFU、目标训练天数、运行 overhead 和 checkpoint 策略；输出包括训练 FLOPs、长期 tokens/s、训练天数、GPU-hours、checkpoint 存储、失败重跑预期损失和门禁结果。

```
params = 7_000_000_000
train_tokens = 1_000_000_000_000
gpus = 128
peak_tflops_per_gpu = 1_000
mfu = 0.40
target_days = 14
runtime_overhead = 0.08

train_flops = 6 * params * train_tokens
peak_flops = gpus * peak_tflops_per_gpu * 1e12
effective_flops = peak_flops * mfu
tokens_per_s = effective_flops / (6 * params)
base_train_seconds = train_flops / effective_flops
base_train_days = base_train_seconds / 86400
required_tokens_per_s = train_tokens / (
    target_days * 86400 * (1 - runtime_overhead)
)

def gib(num_bytes):
    return num_bytes / 1024**3

weight_gib = gib(params * 2)      # BF16/FP16 inference weights.
grad_gib = gib(params * 2)        # BF16/FP16 gradients, rough estimate.
adam_gib = gib(params * 8)        # FP32 m and v.
master_gib = gib(params * 4)      # FP32 master weights.
```

```

resume_ckpt_gib = weight_gib + grad_gib + adam_gib + master_gib
inference_ckpt_gib = weight_gib
retained_storage_gib = 2 * resume_ckpt_gib + 10 * inference_ckpt_gib

```

```

gpu_hours_success = gpus * base_train_seconds / 3600
failure_rate_per_day = 0.05
expected_failures = failure_rate_per_day * base_train_days
checkpoint_interval_hours = 4
expected_lost_gpu_hours = (
    expected_failures * (checkpoint_interval_hours / 2) * gpus
)
planned_wall_days = (
    base_train_seconds / 3600 * (1 + runtime_overhead)
    + expected_lost_gpu_hours / gpus
) / 24

```

```

gates = {
    "throughput_meets_target": tokens_per_s >= required_tokens_per_s,
    "storage_under_500gib": retained_storage_gib <= 500,
    "planned_under_target": planned_wall_days <= target_days,
    "mfu_reasonable": 0.25 <= mfu <= 0.6,
}

```

```

print(f"train_flops={train_flops:.2e}")
print(f"effective_cluster_pflops={effective_flops / 1e15:.1f}")
print(f"tokens_per_s={tokens_per_s:.0f}")
print(f"required_tokens_per_s={required_tokens_per_s:.0f}")
print(f"base_train_days={base_train_days:.2f}")
print(f"planned_wall_days={planned_wall_days:.2f}")
print(f"gpu_hours_success={gpu_hours_success:.0f}")
print(f"resume_ckpt_gib={resume_ckpt_gib:.2f}")
print(f"retained_storage_gib={retained_storage_gib:.2f}")
print(f"expected_failures={expected_failures:.2f}")
print(f"expected_lost_gpu_hours={expected_lost_gpu_hours:.1f}")
print(f"gates={gates}")
print(f"gate_pass={all(gates.values())}")

```

这段 demo 的关键输出应该是：

```

train_flops=4.20e+22
effective_cluster_pflops=51.2
tokens_per_s=1219048
required_tokens_per_s=898608
base_train_days=9.49
planned_wall_days=10.29
gpu_hours_success=29167
resume_ckpt_gib=104.31
retained_storage_gib=339.00
expected_failures=0.47
expected_lost_gpu_hours=121.5
gates={'throughput_meets_target': True, 'storage_under_500gib': True, 'planned_under_target': True, 'mfu_reasonable': True}
gate_pass=True

```

读这段输出时要注意：base_train_days 只来自模型主循环，planned_wall_days 才把 checkpoint/评估 overhead 和期望故障损失纳入了计划；resume_ckpt_gib 远大于推理权重，是因为训练恢复需要 optimizer state 和 master weights；gate_pass=True 只说明这个 toy 方案在设定约束下量级可行，不代表真实集群一定能达到这个 MFU。

11. 常见资源规划错误

11.1 只看模型参数量

忽略 token 数、训练时长和激活显存。

11.2 只看峰值吞吐

忽略 checkpoint、评估和故障重启。

11.3 不考虑失败率

训练一旦不稳定，实际成本会成倍上升。

11.4 不考虑存储和网络

checkpoint 和数据流会成为瓶颈。

11.5 不留余量

所有资源卡满后，调试和容错空间很小。

12. 面试官会怎么问

问法 1：如果让你估算一个训练项目的成本，你怎么做？

可以这样答：

我会先明确模型规模、训练 token 数、序列长度和目标时长，然后估算参数、梯度、optimizer state 和激活显存，再结合同步策略估算通信成本。

问法 2：MFU 是什么？为什么重要？

可以这样答：

MFU 是模型 FLOPs Utilization，用来衡量实际训练计算相对于理论峰值的利用率。它可以帮助判断训练是不是被通信、数据加载、kernel 启动等开销所限制。

问法 3：checkpoint 成本为什么不能忽略？

可以这样答：

因为大模型 checkpoint 很大，尤其带 optimizer state 时体积会数倍增加。保存太频繁会浪费存储和带宽，保存太少又会增加故障重跑成本。

问法 4：为什么训练成本不只是 GPU 费用？

可以这样答：

因为还包括数据处理、存储、网络、调试、人力和失败重跑成本。一个训练系统如果频繁 OOM、NaN 或恢复失败，实际成本可能远高于 GPU 费用。

问法 5：怎么判断资源规划是否合理？

可以这样答：

看三件事：模型能否放下、能否在目标时间内训完、失败后是否可恢复。再结合 tokens/s、MFU、checkpoint 存储、数据吞吐和冗余度来综合判断。

13. 本章小结

本章核心结论：

1. 训练成本包括计算、存储、网络和人力。
2. FLOPs 和 token 数决定大致训练量级。
3. tokens/s 可以用于估算训练时长。
4. MFU 是衡量训练效率的重要指标。
5. GPU 数量要同时满足显存、时长和冗余。
6. 数据规模、checkpoint 存储和失败重跑都要计入成本。
7. 资源规划要从目标、模型、token、并行、存储和容错六个方面一起看。
8. 只看模型参数或峰值吞吐都会严重低估真实成本。
9. 面试中要能做量级估算，而不是只说“需要很多卡”。

第十四章：训练面试题

高频问题

1. 如果让你训练一个 7B 模型，你会怎么设计？
2. 训练 loss 突然 spike，你怎么排查？
3. 为什么训练 loss 降低不代表模型能力一定提升？
4. 如何检测 benchmark contamination？
5. 如何选择 batch size 和 learning rate？
6. ZeRO、FSDP、Tensor Parallel 各自解决什么问题？
7. 如何估算训练显存？
8. SFT 后模型变笨了怎么办？
9. 如何训练一个 VLM？
10. 如何控制大规模训练成本？

回答原则

训练类面试题要按“目标、数据、模型、优化、系统、评估、风险、成本”的顺序回答。

本章资料边界

本章第二轮精修时对照了前面第五册已核对过的训练流程、FSDP/ZeRO、TRL DPO/PPO、Chinchilla 训练预算和 MFU 相关资料。这里不新增复杂专题，而是把第五册的关键机制压缩成面试回答模板和自评脚本。

面试回答里的数字都应该被当成量级估算，而不是生产报价或固定 recipe。不同模型结构、tokenizer、序列长度、硬件、并行策略和训练框架都会改变真实结果。

如何回答训练类面试题

训练类面试题最忌讳只背一个点。

例如面试官问：

如果让你训练一个 7B 模型，你会怎么做？

低质量回答是：

我会用 Transformer、AdamW、数据清洗、分布式训练。

更好的回答应该有结构：

目标 -> 数据 -> tokenizer -> 架构 -> 训练目标 -> 优化器 -> 分布式 -> 监控 -> 评估 -> 风险 -> 成本

面试官真正想看的是你能不能把训练当成一个系统，而不是只知道几个术语。

回答中最好至少能写出三个量级公式：

$$C_{\{\mathrm{train}\}} \approx 6ND$$

$$B_{\{\mathrm{global}\}} = B_{\{\mathrm{micro}\}} \cdot A \cdot W$$

$$S_{\{\mathrm{AdamW}\}} \approx N(b_w + b_g + 2b_m + b_{\{\mathrm{master}\}})$$

其中 N 是模型参数量, D 是训练 token 数, B_{micro} 是单卡 micro-batch, A 是 gradient accumulation steps, W 是 data parallel world size。最后一个式子用于提醒面试官: 训练显存和 checkpoint 不能只看权重, 还要看梯度、Adam 一阶/二阶矩和 master weights。

1. 如果让你训练一个 7B 模型, 你会怎么设计?

回答框架

1. 先明确目标: 通用、代码、数学、领域还是 chat。
2. 定义数据规模和数据配比。
3. 选择 tokenizer 和模型架构。
4. 设计预训练目标和训练配置。
5. 选择分布式策略。
6. 建立监控、checkpoint 和评估。
7. 规划后训练和成本。

标准回答

我会先明确 7B 模型的目标能力, 比如是通用 base model、代码模型还是领域模型。目标会决定数据配比、tokenizer、上下文长度和

数据上, 我会构建多来源语料, 包括高质量网页、书籍、代码、数学、多语言和领域数据, 做清洗、去重、质量过滤和 benchmark cor

模型上, 一般会选择 decoder-only Transformer, 确定层数、hidden size、head 数、FFN size、RoPE、RMSNorm、SwiGLU、GQA 等配

训练上, 使用 next-token prediction, 优化器通常以 AdamW 为强基线, 配合 warmup、cosine decay、gradient clipping、BF16 mix

监控上要看 train loss、val loss、per-domain loss、grad norm、lr、tokens/s、MFU、NaN/Inf、checkpoint 状态。评估上要看 va

常见追问

1. 你怎么定数据配比?
2. 你怎么判断训练是否健康?
3. 你怎么估算训练成本?
4. 如果训练中 loss spike 怎么办?

2. 训练 loss 突然 spike, 你怎么排查?

回答框架

1. 定位第一次 spike 的 step。
2. 检查异常 batch。
3. 检查 grad norm 和 learning rate。
4. 检查 mixed precision。
5. 检查 checkpoint 恢复和分布式状态。
6. 做单变量修复。

标准回答

我不会一上来就调学习率, 而是先定位第一次 spike 的 step。然后保存该 step 的 batch, 检查 input length、decoded text、特殊

同时看 grad norm 是否同步爆炸、learning rate 是否刚到 peak、warmup 是否太短、gradient clipping 是否生效。如果 loss 进入 M

如果发生在 checkpoint 恢复后，还要确认 optimizer、scheduler、scaler、random state 和数据位置是否完整恢复。修复时一次只

常见追问

1. 如果 spike 后自己恢复，要不要管？
2. 如何判断是数据问题还是优化器问题？
3. 如果所有 rank 同时 NaN 怎么办？

3. 为什么训练 loss 降低不代表模型能力一定提升？

回答框架

1. train loss 只反映训练分布。
2. 可能存在重复、污染或过拟合。
3. 平均 loss 掩盖 domain 退化。
4. 下游能力和训练目标不完全一致。
5. 需要综合评估。

标准回答

训练 loss 只说明模型在训练数据上的 next-token prediction 更好，不代表真实能力一定提升。比如数据重复会让 train loss 下降

另外，预训练 loss 和指令遵循、事实性、安全性、推理能力并不完全等价。所以我会同时看 validation loss、per-domain loss、下游 benchmark、人工评测、regression eval 和 safety eval，不能只根据 train loss 判断模型变强。

4. 如何检测 benchmark contamination？

回答框架

1. 明确污染形式。
2. 做 exact match。
3. 做 near dedup。
4. 做 embedding 检索。
5. 对高风险样本人工抽检。

标准回答

Benchmark contamination 指评估集或其近似版本出现在训练数据中，导致评估分数虚高。我会先对 benchmark 题目、答案、解析和作图、MinHash、SimHash 或 embedding 检索做近似匹配。

对于代码、数学和考试题，还要检查题面改写、答案解析泄漏和相似模板。高风险命中需要人工抽检，并在评估报告中标注 contamin

5. 如何选择 batch size 和 learning rate？

回答框架

1. batch size 受显存、稳定性和吞吐限制。
2. learning rate 和 batch size 联动。
3. 需要 warmup 和 decay。
4. 用小规模实验搜索。

标准回答

Batch size 要看 global batch，而不是单卡 batch。global batch = micro batch $\times$ gradient accumulation $\times$ data parallel size。

Learning rate 要和 batch size、模型规模、数据质量和 optimizer 一起调。大模型训练通常需要 warmup，避免初期不稳定，再用 c
面试里可以顺手给出 global batch 的公式：

$$B_{\mathrm{global}} = B_{\mathrm{micro}} \cdot A \cdot W$$

如果 $B_{\mathrm{micro}}=2$ 、 $A=8$ 、 $W=16$ ，那么 global batch 是 256 个样本。LLM 训练里还应进一步换算成 token batch：
 $B_{\mathrm{global}} * \text{sequence_length}$ 。

6. ZeRO、FSDP、Tensor Parallel 各自解决什么问题？

标准回答

ZeRO 和 FSDP 主要解决训练状态显存问题。ZeRO-1 切 optimizer state, ZeRO-2 进一步切 gradient, ZeRO-3 连 parameter 也切。FSDP 类似全切分数数据并行，forward/backward 时按需 gather 参数，训练后再 shard 回去。

Tensor Parallel 切的是层内矩阵计算，比如 attention 和 MLP 的大矩阵，适合单层太大或单卡算力不够的模型。它通信更频繁，通

简单说，ZeRO/FSDP 切训练状态，TP 切模型层内计算。

常见追问

1. ZeRO-3 为什么通信更大？
2. FSDP 和 DDP 区别是什么？
3. TP 和 PP 怎么组合？

7. 如何估算训练显存？

回答框架

1. 参数。
2. 梯度。
3. optimizer state。
4. 激活。
5. buffer。

标准回答

训练显存不能只看模型权重。以 AdamW 为例，除了参数，还要保存梯度、一阶动量、二阶动量，可能还有 FP32 master weights。粗略

此外还要算激活显存，它和 batch size、sequence length、hidden size、layers 强相关。长上下文训练时 activation 可能非常大。
batch、FlashAttention、ZeRO/FSDP 和参数高效微调。

量级估算可以写成：

$$M_{\mathrm{state}} \\ \approx \\ N(b_w + b_g + 2b_m + b_{\mathrm{master}})$$

其中 b_w 是权重字节数， b_g 是梯度字节数， b_m 是 Adam moment 单个状态字节数， b_{master} 是 FP32 master weights 字节数。这个式子不包含 activation，所以真实训练显存还要再加上 activation、通信 buffer、临时 workspace 和框架开销。

8. SFT 后模型变笨了怎么办？

回答框架

1. 判断变笨的表现。
2. 检查 SFT 数据质量。

3. 检查数据配比和训练步数。
4. 检查学习率和 loss mask。
5. 做 regression eval。

标准回答

我会先定义“变笨”是哪些能力退化，比如数学、代码、事实问答、多语言还是格式遵循。然后检查 SFT 数据是否质量低、模板化、

SFT 的风险是过拟合指令风格，导致 base model 的通用能力退化。处理上可以降低 lr、减少 epoch、提升数据质量、混入通用 repla

9. 如何训练一个 VLM?

回答框架

1. 选择 vision encoder。
2. 设计 connector。
3. 接入 LLM。
4. 做图文对齐和 instruction tuning。
5. 做评估和安全检查。

标准回答

VLM 通常由 vision encoder、connector 和 LLM 组成。vision encoder 提取图像特征，connector 把视觉特征映射到 LLM hidden spa

训练上可以先做图文对齐或 connector 预训练，再做多模态 instruction tuning，覆盖 caption、VQA、OCR、图表理解、空间关系和

10. 如何控制大规模训练成本?

回答框架

1. 目标和规模匹配。
2. 数据和 token 预算。
3. 提升吞吐和 MFU。
4. 降低失败率。
5. 管理 checkpoint 和评估成本。

标准回答

我会先从目标倒推模型规模和训练 token，避免模型太大但 token 不够。然后优化训练吞吐和 MFU，比如合理选择 batch、并行策略、

显存上用 BF16、activation checkpointing、ZeRO/FSDP 等策略。流程上先做小规模 smoke test，降低失败重跑成本；训练中做好监

11. Base model、Instruct model、Chat model 有什么区别?

标准回答

Base model 主要通过预训练做 next-token prediction，学到语言、知识和通用模式，擅长续写但不一定会按指令回答。Instruct mo

12. DPO 和 PPO 有什么区别?

标准回答

PPO 是在线 RLHF 方法，通常需要 reward model、reference model 和 critic，通过模型采样回答、reward 打分和 KL 约束来更新 po

DPO 是离线偏好优化，直接用 prompt/chosen/rejected 偏好对训练模型，不需要显式 reward model 和在线 rollout。它简单稳定，但

DPO 的核心面试公式可以压缩成：

$$L_{\{\mathrm{DPO}\}} = -\log \sigma \left(\beta (\Delta_{\theta} - \Delta_{\mathrm{ref}}) \right)$$

其中 Δ_{θ} 是当前 policy 对 chosen/rejected 的 log probability 差, Δ_{ref} 是 reference model 的对应差值。这个公式的面试价值是说明: DPO 不是没有约束地提高 chosen 概率, 而是相对 reference 调整偏好。

13. 训练类面试回答 checklist

回答训练题时, 最后可以自检:

1. 是否先讲目标?
2. 是否讲数据和 tokenizer?
3. 是否讲模型架构和规模?
4. 是否讲训练目标和优化器?
5. 是否讲分布式和显存?
6. 是否讲监控和 debug?
7. 是否讲 checkpoint 和评估?
8. 是否讲成本和风险?
9. 是否讲 trade-off?

如果一个回答覆盖这些点, 就比较完整。

13.1 最小可运行面试回答自评 demo

下面这个 0 依赖 demo 用关键词和几个量级计算检查一个 “训练 7B 模型” 的回答是否覆盖关键维度。它不是替代人工判断, 而是把面试复盘变成可检查的 checklist。

```
answer = """
目标是训练一个通用 7B base model, 再进入 SFT 和偏好优化。
数据上做网页、代码、数学和多语言 mixture, 先清洗、去重并做 contamination 检测。
模型选择 decoder-only Transformer, 使用 RoPE、RMSNorm、SwiGLU 和 GQA。
训练目标是 next-token prediction, 优化器用 AdamW, 配 warmup、cosine decay、BF16、gradient clipping。
系统上根据显存选择 ZeRO/FSDP、TP 和 activation checkpointing。
监控 train loss、val loss、grad norm、lr、tokens/s、MFU、NaN、checkpoint。
评估包括 per-domain loss、代码、数学、安全、人工评估和 regression eval。
风险包括 loss spike、数据污染、能力退化、恢复失败和成本超预算。
成本上用 FLOPs、GPU hours、checkpoint 存储和失败重跑预算做量级估算。
""".lower()
```

```
rubric = {
    "target": ["目标", "base model"],
    "data": ["数据", "mixture", "清洗", "去重"],
    "contamination": ["contamination", "污染"],
    "model": ["decoder-only", "rope", "rmsnorm", "swiglu", "gqa"],
    "optimization": ["adamw", "warmup", "cosine", "bf16", "clipping"],
    "distributed": ["zero", "fsdp", "tp", "checkpointing"],
    "monitoring": ["train loss", "val loss", "grad norm", "mfu", "checkpoint"],
    "evaluation": ["per-domain", "代码", "数学", "安全", "regression"],
    "risk": ["loss spike", "能力退化", "恢复失败"],
    "cost": ["flops", "gpu hours", "存储", "重跑"],
}

coverage = {
    name: all(keyword in answer for keyword in keywords)
```

```

    for name, keywords in rubric.items()
}
score = sum(coverage.values())

micro_batch = 2
grad_accum = 8
data_parallel = 16
global_batch = micro_batch * grad_accum * data_parallel

params = 7_000_000_000
train_tokens = 1_000_000_000_000
train_flops = 6 * params * train_tokens
adamw_state_gib = params * (2 + 2 + 8 + 4) / 1024**3

gate_pass = (
    score >= 9
    and global_batch == 256
    and round(adamw_state_gib, 2) == 104.31
)

print(f"coverage={coverage}")
print(f"score={score}/10")
print(f"missing=[{name for name, ok in coverage.items() if not ok}]")
print(f"global_batch={global_batch}")
print(f"train_flops={train_flops:.2e}")
print(f"adamw_state_gib={adamw_state_gib:.2f}")
print(f"gate_pass={gate_pass}")

```

这段 demo 的关键输出应该是：

```

coverage={'target': True, 'data': True, 'contamination': True, 'model': True, 'optimization': True, 'distributed': True,
score=10/10
missing=[]
global_batch=256
train_flops=4.20e+22
adamw_state_gib=104.31
gate_pass=True

```

真正面试时不需要把脚本背出来，但要能像脚本一样自查：有没有目标、有没有数据、有没有模型和训练、有没有系统和评估、有没有风险和成本。如果缺一两项，回答就容易被追问打穿。

14. 本章小结

本章把第五册核心内容转化成面试回答模板。

核心结论：

1. 训练类面试要按系统链路回答，不要只背名词。
2. 设计训练方案要从目标、数据、模型、优化、系统、评估和成本展开。
3. Debug 题要先定位层级，再给修复方案。
4. 评估题要强调可信评估、污染检测和回归测试。
5. 分布式题要说清切了什么、省了什么、增加了什么通信。
6. 成本题要能做量级估算。
7. 后训练题要区分 SFT、PPO、DPO、reasoning RL 和安全对齐。
8. 多模态训练题要强调模态对齐、数据质量和评估难度。